



Thakur Educational Trust's (Regd.)
THAKUR COLLEGE OF SCIENCE & COMMERCE

Empowered Autonomous College Permanently Affiliated to University of Mumbai

(NAAC Accredited with Grade "A" (3rd Cycle) & ISO 21001:2018 Certified)

Best College Award by University of Mumbai for the Year 2018-2019



DEPARTMENT OF INFORMATION TECHNOLOGY

A PROJECT REPORT ON

Browser Based Discrete – Event Simulator

"JustCmul8 — A No-Code, Browser-Based Discrete Event Simulation Platform"



SUBMITTED BY

Ms. Khushi Singh – 261713

Mr. Mohit Gupta – 261715

UNDER THE GUIDANCE OF

Ms. Jinal Gujar

Submitted in partial fulfilment of the requirements for qualifying
BACHELOR OF SCIENCE (INFORMATION TECHNOLOGY)
Semester – V Examination

THAKUR COLLEGE OF SCIENCE AND COMMERCE

(Empowered Autonomous College, Permanently Affiliated to University of Mumbai)
THAKUR VILLAGE, KANDIVALI (EAST), MUMBAI – 400 101, MAHARASHTRA

ACADEMIC YEAR 2026 – 2027



DEPARTMENT OF INFORMATION TECHNOLOGY

CERTIFICATE OF APPROVAL

This is to certify that the project entitled, “Browser Based Discrete – Event Simulator”, has been undertaken at Thakur College of Science and Commerce by Ms. Khushi Singh (Roll No. 261713) and Mr. Mohit Gupta (Roll No. 261715), students of “Bachelor of Science (Information Technology)”, and is submitted in partial fulfilment of the requirements for the award of the degree of BACHELOR OF SCIENCE in INFORMATION TECHNOLOGY, Semester–V Examination, as per the syllabus defined by the University of Mumbai in the academic year 2026 – 2027.

It is further certified that the students have completed all the required phases of the project, and that the work presented in this report is their own and has not been submitted elsewhere for the award of any other degree or diploma.

Ms. Jinal Gujar
Project Guide

Prof. Dr. Santosh Kumar Singh
Head of Department

External Examiner

Principal

Date: _____

College Seal / Stamp: _____

CANDIDATES' DECLARATION

We hereby declare that the project work presented in this report, entitled “Browser Based Discrete – Event Simulator”, submitted to the Department of Information Technology, Thakur College of Science and Commerce, in partial fulfilment of the requirements for the award of the degree of Bachelor of Science in Information Technology (Semester–V), is an authentic record of our own work carried out during the academic year 2026 – 2027 under the guidance of Ms. Jinal Gujar.

We further declare that the analysis, system design, source code, simulation engine integration, testing and documentation presented in this report were developed by us. Wherever the work of other authors, published literature, official technical documentation or open-source libraries has been referred to, due acknowledgement has been made in the text and the corresponding sources have been listed in the References. The matter embodied in this report has not been submitted, in part or in full, to any other university or institute for the award of any degree or diploma.

We also understand and accept that any breach of academic integrity detected at any stage would attract disciplinary action as per the rules and regulations of the institution and the University of Mumbai.

Place: Mumbai

Date: _____

Ms. Khushi Singh
Roll No. 261713

Mr. Mohit Gupta
Roll No. 261715

ACKNOWLEDGEMENT

The completion of this project would not have been possible without the guidance, encouragement and practical help we received from several people, and we would like to record our gratitude to them here.

First and foremost, we express our sincere thanks to our project guide, Ms. Jinal Gujar, Department of Information Technology, for the consistent direction she provided throughout the development of JustCmul8. Her insistence that we validate the simulation engine before building features on top of it changed the order in which we scheduled our work, and saved us a considerable amount of rework later. The structure of this report also owes a great deal to her review of our early drafts.

We are deeply grateful to Prof. Dr. Santosh Kumar Singh, Head of the Department of Information Technology, for creating an academic environment in which an ambitious project of this nature could be attempted, and for his valuable suggestions during the project review sessions.

We extend our thanks to the Principal and the management of Thakur College of Science and Commerce for providing the infrastructure, laboratory facilities and internet resources that this project required. Running a WebAssembly-based Python runtime and testing real-time collaboration across multiple machines simultaneously would not have been feasible without access to the department laboratory.

We thank the faculty members of the Department of Information Technology for the foundation they laid in data structures, database management systems, software engineering and web technologies. Concepts such as priority queues, normalisation and the software development life cycle, taught across earlier semesters, were used directly in this work rather than merely referenced. We are also grateful to the laboratory assistants for their patient help with system access and configuration during the long testing sessions.

Finally, we thank the maintainers of the open-source projects on which JustCmul8 is built — SimPy, Pyodide, React Flow, Next.js and Supabase — whose documentation and community

discussions resolved many of the technical obstacles we met, and our families and classmates for their patience and encouragement throughout.

Ms. Khushi Singh – 261713

Mr. Mohit Gupta – 261715

ABSTRACT

Discrete Event Simulation is among the more dependable ways of predicting how a queue, a production line or a delivery network will behave before money is spent changing it, yet the technique remains out of reach for most of the people who would benefit from it. Commercial packages such as AnyLogic, Arena and Simul8 carry licence fees running into thousands of dollars a year; the capable open-source alternatives, SimPy chief among them, are programming libraries that expect the user to write and debug Python before a first model will run; and almost all of these tools are desktop applications that must be installed, which rules them out in shared laboratories and on managed machines. Drawing tools such as Visio and Lucidchart sit at the opposite extreme, easy to use but producing diagrams that do not execute, so they cannot answer a question as ordinary as what happens if one more counter is opened.

This project addresses that gap. We designed and developed JustCmul8, a no-code, browser-based Discrete Event Simulation platform on which a user builds a model by dragging nodes onto a canvas and connecting them, configures each node through ordinary form fields, and executes the result without installing anything. The distinguishing decision of the architecture is that the model is not approximated in JavaScript: the graph is compiled into a genuine Python SimPy script, executed inside the browser by a CPython interpreter compiled to WebAssembly through Pyodide and hosted in a Web Worker so the interface stays responsive. The numbers the platform reports are therefore produced by the same engine a researcher would use from a terminal.

The system was implemented as a client-heavy single-page application using Next.js 16, React 19 and TypeScript, with the canvas built on React Flow and shared state held in Zustand slices. Persistence, authentication and collaborative synchronisation are provided by Supabase, with PostgreSQL Row-Level Security policies enforcing project isolation at the database rather than in client code. Fifteen configurable node types span six application domains. An analytics layer computes utilisation, mean and percentile waiting times, throughput and queue lengths, and ranks nodes by a composite bottleneck score combining sustained utilisation with queue growth. A Monte Carlo facility repeats a run across replications to report 95 per cent confidence intervals instead of a single figure, and a Google Gemini assistant converts a plain-English description into a candidate graph and proposes remedies for the bottlenecks it finds.

Testing combined unit tests over the samplers and analytics functions, structural tests for disconnected and cyclic graphs, cross-browser verification and repeated-run stress testing; all sixty documented cases passed. Output was verified against M/M/1 theory to within three per cent, and determinism confirmed by reproducing identical traces from identical seeds. The result is a working platform that removes the cost, coding and installation barriers to simulation without compromising the accuracy of the engine underneath.

TABLE OF CONTENTS

Page numbers refresh automatically in Microsoft Word. If a field shows a stale value, select the whole document (Ctrl+A) and press F9 to update.

Sr. No.	Title	Page No.
1	Introduction	1
1.1	Objective and Scope of the Project	1
1.2	Theoretical Background	4
1.3	Problem Definition	8
1.4	User Requirements and Software Requirements Specification	9
1.5	Feasibility Study	13
1.6	Details of Hardware and Software Used	15
2	System Planning	19
2.1	List of Activity and Task	19
2.2	Timeline for Each Task	21
2.3	Gantt Chart and Activity Diagram	24
3	Software and Hardware Modules	30
3.1	Module 1 — Platform and Data Layer	31
3.2	Module 2 — Visual Graph Editor	32
3.3	Module 3 — Node Configuration Panel	35
3.4	Module 4 — Simulation Engine	36
3.5	Module 5 — AI Assistant	38
3.6	Module 6 — AI System Optimiser	39
3.7	Module 7 — KPI Dashboard and Results	40
3.8	Module 8 — Monte Carlo and Scenario Comparison	41
3.9	Module 9 — Digital Twin Animated Playback	42
3.10	Module 10 — Simulation Template Gallery	43
3.11	Module 11 — Executive Report and Share	44
3.12	Module 12 — Real-Time Collaboration	45
3.13	Hardware Modules and Considerations	46
4	System Analysis and Design	47
4.1	Detailed Life Cycle of the Project	47
4.2	Context Diagram (DFD Level 0)	49
4.3	Data Flow, Entity Relationship, Class and State Diagrams	51
4.4	Use Case Diagram	59
4.5	Activity, Component and Collaboration Diagrams	63
4.6	Architecture Design	68
4.7	Table Design	69
4.8	Data-Structure Design	75
4.9	Deployment Diagram	78
5	Testing, Verification and Quality Assurance	80
5.1	Testing Methodologies	80

Sr. No.	Title	Page No.
5.2	Test Case Matrix	83
6	Cost Estimation Report	87
6.1	Project Size Summary	87
6.2	Technology Stack Considered in the Estimate	87
6.3	COCOMO — Constructive Cost Model	88
6.4	Function Point Analysis	89
6.5	Cost Estimation Summary and Discussion	91
7	Results, Screenshots and Discussion	93
7.1	User Interface and Workflow Screenshots	93
7.2	Performance Evaluation	98
8	Conclusion and Future Scope	102
8.1	Conclusion	102
8.2	Limitations	103
8.3	Future Scope	104
—	References and Bibliography	107

LIST OF FIGURES

Figure No.	Caption	Page No.
Fig. 2.1	Weekly Gantt Chart of the JustCmul8 Development Schedule (July W1 – December W4, 2026)	25
Fig. 2.2	Activity Diagram of the JustCmul8 Development Lifecycle	28
Fig. 4.1	Context Diagram (DFD Level 0) for JustCmul8	49
Fig. 4.2	Level-1 Data Flow Diagram for JustCmul8	51
Fig. 4.3	Level-2 Data Flow Diagram — Decomposition of Process P4 (Simulation Execution)	53
Fig. 4.4	Entity Relationship Diagram for the JustCmul8 Database	54
Fig. 4.5	Class Diagram of the JustCmul8 Object Model	56
Fig. 4.6	State Transition Diagram for a Node During Simulation Execution	57
Fig. 4.7	Use Case Diagram for JustCmul8	59
Fig. 4.8	Activity Diagram of the End-to-End User Workflow	63
Fig. 4.9	Component Diagram of the JustCmul8 Implementation	65
Fig. 4.10	Collaboration Diagram for the Simulation Run Interaction	66
Fig. 4.11	Multi-Tier Architecture of JustCmul8	68
Fig. 4.12	Deployment Architecture of JustCmul8	78
Fig. 7.1	Execution, Code Generation and Rendering Time against Model Size	98
Fig. 7.2	Narrowing of the 95% Confidence Interval with Increasing Replication Count	100

LIST OF TABLES

Table No.	Caption	Page No.
Table 1.1	Comparison of Discrete Event and Continuous Simulation	4
Table 1.2	Probability Distributions Implemented in JustCmul8	5
Table 1.3	User Classes and Access Levels	10
Table 1.4	Functional Requirements	10
Table 1.5	Non-Functional Requirements	11
Table 1.6	Cost Comparison with Commercial Alternatives	14
Table 1.7	Development Hardware and Software Configuration	16
Table 1.8	Minimum and Recommended Client Requirements	16
Table 1.9	Technology Stack with Justification	17
Table 2.1	Activity Schedule with Durations and Dependencies	21
Table 2.2	Project Milestones	23
Table 3.1	Module Overview and Requirement Coverage	30
Table 3.2	The Fifteen Node Types Supported by the Simulation Engine	33
Table 3.3	The Six Simulation Domains and Their Vocabularies	34
Table 3.4	Cost Parameters and Their Role in ROI Analysis	35
Table 3.5	Key Performance Indicators and Their Computation	40
Table 3.6	Hardware Roles in the Deployed System	46
Table 4.1	Phases of the Adopted Life-Cycle Model	48
Table 4.2	Use Case Specification — Run Simulation	60
Table 4.3	Use Case Specification — Generate Model via AI	61
Table 4.4	Use Case Specification — Share Project Read-Only	61
Table 4.5	Use Case Specification — Collaborate in Real Time	62
Table 4.6	Table: users	69
Table 4.7	Table: profiles	70
Table 4.8	Table: project_collaborators	70
Table 4.9	Table: projects	70
Table 4.10	Table: sim_nodes	71
Table 4.11	Table: sim_edges	71
Table 4.12	Table: simulation_runs	72
Table 4.13	Table: node_stats	72
Table 4.14	Table: sim_logs	72
Table 4.15	Tables: entity_journeys and entity_journey_steps	73
Table 4.16	Table: project_shares	73
Table 4.17	Table: chat_history	74
Table 4.18	Complexity of Graph Operations on the Adjacency List	75
Table 4.19	Comparison of Candidate Event-Queue Structures	76
Table 4.20	Auxiliary In-Memory Structures	77
Table 5.1	Summary of Testing Coverage	82
Table 5.2	Test Case Matrix	83

Table No.	Caption	Page No.
Table 6.1	Measured Project Size	87
Table 6.2	Technology Stack and Its Effect on Development Effort	87
Table 6.3	COCOMO Coefficients by Development Mode	88
Table 6.4	COCOMO Estimation Results	89
Table 6.5	Standard IFPUG Complexity Weights	89
Table 6.6	Function Point Count by Component Type	89
Table 6.7	General System Characteristics and Degrees of Influence	90
Table 6.8	Comparison of Estimates against Actual Effort	91
Table 7.1	Performance across Model Sizes (1,000 simulated minutes, single replication)	98
Table 7.2	Monte Carlo Performance and Interval Width (50-node model, mean waiting time)	99
Table 7.3	Responsiveness Measurements against Requirements	100

LIST OF ABBREVIATIONS

Abbr.	Expansion	Abbr.	Expansion
AI	Artificial Intelligence	NFR	Non-Functional Requirement
API	Application Programming Interface	OAuth	Open Authorization
CDN	Content Delivery Network	PK	Primary Key
CI/CD	Continuous Integration / Continuous Deployment	PM	Person-Months
COCOMO	Constructive Cost Model	PRNG	Pseudo-Random Number Generator
CRUD	Create, Read, Update, Delete	Pyodide	WebAssembly distribution of the CPython interpreter
CSV	Comma-Separated Values	RBAC	Role-Based Access Control
DES	Discrete Event Simulation	RLS	Row-Level Security
DFD	Data Flow Diagram	SDLC	Software Development Life Cycle
DOM	Document Object Model	SimPy	Simulation in Python — process-based DES library
ERD	Entity Relationship Diagram	SPA	Single-Page Application
FIFO	First In, First Out	SQL	Structured Query Language
FK	Foreign Key	SRS	Software Requirements Specification
FPA	Function Point Analysis	SSR	Server-Side Rendering
FR	Functional Requirement	TLS	Transport Layer Security
HCI	Human–Computer Interaction	UAT	User Acceptance Testing
HOD	Head of Department	UFP	Unadjusted Function Points
HTTPS	Hypertext Transfer Protocol Secure	UI/UX	User Interface / User Experience
IFPUG	International Function Point Users Group	UML	Unified Modeling Language
JSON	JavaScript Object Notation	UUID	Universally Unique Identifier
KLOC	Thousand Lines of Code	VAF	Value Adjustment Factor
KPI	Key Performance Indicator	WASM	WebAssembly
LIFO	Last In, First Out	WSS	WebSocket Secure
LLM	Large Language Model		

CHAPTER 1

INTRODUCTION

1.1 Objective and Scope of the Project

Objective

JustCmul8 was designed and developed as a free, no-code Discrete Event Simulation platform that runs entirely inside a web browser. Our primary objective was to let a user build, execute, analyse and share a simulation model without installing software, without writing code and without purchasing a licence. The three barriers we set out to remove — cost, programming skill and installation — are precisely the three that keep simulation confined to large organisations today.

The platform provides a drag-and-drop modelling environment in which nodes representing sources, queues, servers, decisions and sinks are placed on a canvas and joined by edges. Behind that interface sits a genuine Python SimPy engine executed in the browser through WebAssembly. This was a deliberate architectural commitment rather than a convenience: had we written an approximate event loop in JavaScript, the platform would have been easier to build but its output would not have withstood scrutiny. Because the graph is compiled into a real SimPy script, the statistics JustCmul8 reports are generated by the same library that researchers and practising engineers rely on.

A second objective was to reduce the conceptual learning curve, not merely the technical one. A user who has never encountered queueing theory does not necessarily know that a bank counter should be modelled as a capacity-limited resource preceded by a FIFO queue. We therefore integrated an AI assistant built on Google Gemini that accepts a plain-English description of a process and produces a candidate graph, which the user may inspect, edit or discard before it is committed to the project.

The third objective concerned interpretation of results. Raw event traces are of little use to a non-specialist, so the system computes and presents resource utilisation, waiting times, throughput and queue lengths through interactive charts, and identifies the node most likely to be constraining the system. An animated two-dimensional playback allows the user to watch entities

move through the model, which in our experience communicates congestion far more immediately than a table of numbers.

We also aimed to support real-time collaborative modelling, so that several users in different locations can edit and discuss the same project simultaneously in the manner of Canva or Figma. Finally, the architecture was kept modular and extensible throughout, so that additional simulation domains, node types and analytical features can be introduced later without disturbing the core engine.

Specific Objectives

- **Eliminate cost and installation barriers** — deliver a platform that requires no licence fee and no local installation beyond a modern browser.
- **Preserve simulation accuracy** — execute a real Python SimPy engine rather than a visual approximation, so results are defensible.
- **Remove the coding prerequisite** — provide a visual modelling interface usable by someone who has never programmed.
- **Support collaborative modelling** — allow concurrent multi-user editing of a single project with live cursors and presence.
- **Quantify uncertainty** — report confidence intervals from repeated Monte Carlo replications instead of a single deterministic figure.
- **Surface actionable insight** — detect and rank bottlenecks automatically and propose concrete remedial changes.
- **Design for extension** — structure the system so that new domains, node types and engines can be added without redesigning the core.

Scope of the Project

The delivered system covers the complete simulation workflow, from account creation through to the export of results. Functionality is organised into twelve modules, which fall naturally into four groups.

- **Platform and data layer** — secure registration and login by email or OAuth, session handling, project creation, renaming, duplication and deletion, cloud persistence, and user profile and preference management.

- **Modelling and execution** — the visual graph editor, the node configuration panel, the Pyodide-hosted SimPy simulation engine, the AI assistant that generates models from natural language, and the optimiser that diagnoses a completed run and proposes a fix.
- **Analysis and reuse** — the KPI dashboard with live charts and bottleneck detection, the Monte Carlo and scenario comparison facility, the animated digital-twin playback, and the template gallery of starter models.
- **Distribution and collaboration** — export of graphs as JSON and results as CSV, generation of executive PDF reports, read-only public share links, and real-time collaborative editing.

Modelling support spans six application domains — human queue systems, vehicle traffic, liquid and material flow, manufacturing processes, logistics and warehousing operations, and network and signal flow. All six are driven by the same simulation engine and draw on a shared library of fifteen configurable node types, which between them cover a wide class of realistic scenarios.

Out of Scope

Several capabilities were excluded deliberately to keep the project achievable within the academic timeframe. Recording them explicitly distinguishes a considered boundary from an oversight.

- **Native mobile applications** — the interface is responsive and works in mobile browsers, but no separate Android or iOS application was built.
- **Enterprise identity and billing** — single sign-on, LDAP integration, organisation-level user administration and subscription billing are not implemented.
- **Learning-management features** — the platform is a simulation tool, not a courseware system; it delivers no lessons or assessments.
- **Offline and self-hosted operation** — the application is deployed as a cloud-hosted web application and requires connectivity for persistence and collaboration.
- **Industrial hardware integration** — direct connections to IoT devices, PLCs, sensors or robotics are outside the current scope.
- **Automated optimisation** — the AI assistant proposes changes for human approval; it does not run search-based optimisation algorithms over the parameter space.
- **Desktop distribution** — no packaged Windows, macOS or Linux build was produced, consistent with the browser-first approach.

1.2 Theoretical Background

This project rests on several established areas of computer science and operations research. The paragraphs below set out the concepts that were applied directly in the design and implementation.

1.2.1 Discrete Event Simulation and Continuous Simulation

Discrete Event Simulation represents a system as a sequence of events occurring at distinct instants of simulated time. Between two consecutive events the state of the system is assumed not to change, so the clock advances directly from one event to the next rather than ticking through uniform intervals. A customer arriving at a counter, a machine finishing a job and a packet reaching a router are all events in this sense. The technique suits systems whose behaviour is driven by identifiable occurrences and by contention for limited resources.

Continuous simulation takes the opposite view. State is described by variables that change smoothly over time, typically governed by differential equations integrated over small fixed steps — the temperature of a vessel or the level of a reservoir, for example. The distinction matters computationally: because a discrete event simulation skips the intervals in which nothing happens, it can simulate a full working day in a fraction of a second, whereas a continuous model must evaluate every step.

JustCmul8 is a discrete event simulator, which is the appropriate choice for queueing, manufacturing and logistics problems. Its container node nevertheless models a continuously varying level, sampled at event boundaries, which lets liquid and bulk-material flow be represented within the same event-driven framework.

Table 1.1: Comparison of Discrete Event and Continuous Simulation

Aspect	Discrete Event Simulation	Continuous Simulation
Time advance	Jumps to the next scheduled event	Advances in small fixed increments
State change	Instantaneous, at event instants only	Smooth and continuous throughout
Mathematical basis	Probability theory and queueing theory	Differential and difference equations
Typical application	Queues, production lines, supply chains	Thermal, hydraulic and control systems
Computational cost	Proportional to the number of events	Proportional to the simulated duration
Role in JustCmul8	The primary execution paradigm	Approximated by container level sampling

Table 1.1 summarises why the discrete event paradigm was selected. Since the scenarios our users model are driven by arrivals and service completions rather than by smoothly varying physical quantities, event-driven execution is both the more faithful representation and the cheaper one.

1.2.2 Probability Distributions in Operational Modelling

Realistic simulation depends on representing variability correctly. If every customer were assumed to take exactly four minutes to serve, queues would never form in the model even though they form constantly in reality. Variability is therefore introduced by sampling service and inter-arrival times from probability distributions. JustCmul8 implements five samplers, each chosen for a distinct modelling purpose.

Table 1.2: Probability Distributions Implemented in JustCmul8

Distribution	Parameters	Characteristics	Typical use
Exponential	mean ($1/\lambda$)	Memoryless; short intervals frequent, long ones rare	Inter-arrival times of independent arrivals
Uniform	min, max	All values in the range equally likely	A duration known only to lie between two bounds
Normal	mean, std. deviation	Symmetric about the mean; clustered centrally	Machine cycle times with small random error
Poisson	λ (mean count)	Discrete counts of events in a fixed interval	Batch sizes and arrivals per period
Deterministic	fixed value	No variability; repeats exactly	Conveyor transfers and scheduled operations

The exponential distribution is the default for arrival processes because of its memoryless property: the time until the next arrival does not depend on how long one has already waited, which is the standard assumption for independent customers. The deterministic option exists because forcing artificial randomness onto a genuinely fixed operation, such as a timed conveyor, would distort rather than improve the model.

The normal distribution requires one safeguard. Since its support extends over the whole real line, a sample may be negative when the standard deviation is large relative to the mean, and a negative service time is meaningless. Our sampler therefore truncates at zero. Sampling itself uses the inverse transform method wherever a closed-form inverse exists — for the exponential case, drawing a uniform variate u on $(0,1]$ and returning $-\text{mean} \times \ln(u)$ — and the Box–Muller transform for normal variates.

1.2.3 Monte Carlo Estimation of Risk and Throughput

A single stochastic run is one observation drawn from a distribution of possible outcomes, and reporting it alone is misleading. Two runs of an identical model with different random seeds may produce average waiting times of four and seven minutes; quoting either figure as the answer invites a false conclusion. Monte Carlo analysis addresses this by executing the model repeatedly with independent seeds and treating the collected outcomes as a sample from which the underlying behaviour can be estimated.

JustCmul8 supports replication counts of twenty, fifty and one hundred. For each metric it computes the sample mean and standard deviation across replications, then forms a ninety-five per cent confidence interval as the mean plus or minus 1.96 times the standard error, the standard error being the standard deviation divided by the square root of the number of replications. The dashboard renders these intervals as bands rather than lines, so a user can see at a glance whether an apparent improvement between two configurations is larger than the noise.

The width of the interval narrows in proportion to the inverse square root of the replication count, which has a direct practical consequence: quadrupling the number of replications halves the interval. That relationship governed our choice of presets, since it explains why moving from twenty to fifty replications produces a visible gain while moving from one hundred to four hundred rarely justifies the additional runtime in an interactive setting.

1.2.4 Supporting Concepts from Software Engineering

- **Software Development Life Cycle** — the project followed the standard phases of requirement analysis, design, implementation, testing and deployment, applied iteratively for each module so that a module could be validated before the next was built upon it.
- **Multi-tier architecture** — concerns are separated into a presentation layer built with React and Next.js, an application and domain layer holding state and service logic, an execution layer hosting the WebAssembly runtime, and a persistence layer provided by Supabase, each independently testable.
- **Client-server model and WebAssembly** — the browser renders the interface and, unusually, also performs the simulation computation, because the Python interpreter is compiled to WebAssembly and executed locally; the server is reduced to persistence, authentication and AI proxying.

- **Relational database concepts** — user, project, graph and result data are held in PostgreSQL with primary and foreign keys enforcing referential integrity, and Row-Level Security policies restricting each row to the users entitled to see it.
- **Graph theory** — a simulation model is a directed graph; validating it requires connectivity checks and cycle detection, for which we used depth-first traversal over an adjacency-list representation.
- **Authentication, authorisation and HCI** — the system distinguishes verifying who a user is from determining what that user may do, and applies interaction principles — plain-language labelling, low cognitive load, familiar drag-and-drop gestures — to keep the tool usable by non-programmers.

1.3 Problem Definition

Discrete Event Simulation is widely used to analyse and improve manufacturing, healthcare, logistics, transportation and service operations, yet its adoption remains narrow. The reasons are practical rather than theoretical, and they fall into four groups.

The cost of commercial simulation software

Established packages such as AnyLogic, Arena and Simul8 are licensed at figures that run into thousands of dollars per seat per year. For a department teaching operations research to sixty students, or for a restaurant owner wishing to test whether a second service counter would pay for itself, the licence alone exceeds the value of the decision being informed. Simulation consequently becomes available precisely to those organisations that could most easily absorb the cost of guessing wrongly, and unavailable to those that could not.

The technical complexity of free alternatives

Capable open-source frameworks exist, SimPy foremost among them, but they are programming libraries rather than applications. Before a first model will run, the user must install Python, manage a virtual environment, learn generator functions and the yield statement, understand how resource requests are expressed, and debug the inevitable errors. The simulation concepts and the Python idioms must be acquired simultaneously, which is a considerable demand on someone whose actual question is whether to open another counter.

The limitations of desktop distribution

Most simulation software is desktop software, requiring installation, configuration and maintenance. In a shared college laboratory, on a managed corporate machine or on a Chromebook, students and staff frequently lack the privileges to install anything at all. A tool that cannot be opened is of no use whatever its analytical merits, and this obstacle is entirely independent of cost or skill.

The inadequacy of static diagramming and spreadsheets

Two substitutes are commonly attempted, and both fail for the same underlying reason. Diagramming tools such as Visio and Lucidchart produce clear process maps, but those maps are inert: a rectangle labelled “Inspection: 5 min” carries no execution semantics, so the diagram cannot reveal that inspection will accumulate a queue of thirty items by mid-afternoon. Spreadsheet models go one step further and compute something, but a spreadsheet has no natural representation of an event queue, of entities competing for a resource, or of a server that is busy when the next arrival appears. Modellers resort to fixed time-step approximations that either miss events falling between steps or grow unmanageably large, and queue discipline, resource preemption and probabilistic routing remain effectively out of reach. Neither substitute captures contention, which is the phenomenon that causes queues in the first place.

Statement of the problem

The combined effect of these barriers is that a technique capable of answering ordinary operational questions cheaply is confined to organisations with both the budget for licences and staff trained to use them. What is required is a platform that is free to use, needs no installation, demands no programming knowledge, is reachable from any modern browser, and — critically — does not trade away the accuracy of the underlying engine in exchange for that accessibility. JustCmul8 was built to meet that requirement.

1.4 User Requirements and Software Requirements Specification

This section records what the system is required to do and for whom. It opens with an overall description of the product, identifies the classes of user it serves, and then states the functional and non-functional requirements individually, each carrying an identifier that is referred to throughout the rest of this report.

1.4.1 Overall Description

JustCmul8 is a browser-based simulation platform reached through a standard web browser. It follows a multi-tier structure comprising a user interface layer, an application and domain logic layer, a client-side execution layer and a managed database layer, and it requires no client-side installation beyond the browser itself. The design is deliberately client-heavy: the interface, the state and the entire simulation computation reside on the client, while the server is responsible only for persistence, authentication, collaborative message relay and proxying requests to the AI service.

1.4.2 User Classes and Characteristics

Table 1.3: User Classes and Access Levels

User class	Responsibilities	Access level	Skill required
Project Owner	Creates projects, manages collaborators and sharing, deletes or transfers ownership	Full control of owned projects	None
Editor (Collaborator)	Builds and edits the graph, configures nodes, executes simulations	Project-scoped, read and write	None
Viewer (Collaborator)	Inspects a shared project and its results without altering it	Project-scoped, read only	None
Guest / Public Viewer	Opens a public share link to view a model and its results	Read only, no account required	None

No user class requires programming ability, which follows directly from the no-code objective stated in Section 1.1. The separation of Editor from Viewer exists so that a model may be circulated for comment without exposing it to accidental modification.

1.4.3 Functional Requirements

The functional requirements below are grouped by module and carry identifiers used throughout the remainder of this report, particularly in the test case matrix of Chapter 5.

Table 1.4: Functional Requirements

ID	Module	Requirement
FR-01	Authentication	The system shall support registration and login by email and password, and by Google or GitHub OAuth, with secure logout.
FR-02	Authentication	The system shall maintain an authenticated session across browser visits and restrict project access to entitled users.
FR-03	Authentication	The system shall allow a user to reset a forgotten password through a time-limited single-use email link.
FR-04	Project Management	The system shall allow an authenticated user to create, rename, duplicate and delete simulation projects.
FR-05	Project Management	The system shall persist project state to cloud storage automatically on every change, without explicit save.
FR-06	Project Management	The system shall present a dashboard listing all projects the user owns or collaborates on.
FR-07	Graph Editor	The system shall provide a drag-and-drop canvas supporting fifteen node types, with node placement, movement, connection and deletion.
FR-08	Graph Editor	The system shall re-route edges live as nodes are moved, and shall support zoom, pan and canvas navigation.
FR-09	Graph Editor	The system shall validate that an edge joins two nodes belonging to the same project, and shall reject structurally invalid graphs.

ID	Module	Requirement
FR-10	Node Configuration	The system shall allow the parameters of a selected node — arrival rate, capacity, service-time distribution and routing rule — to be edited through a side panel.
FR-11	Node Configuration	The system shall validate configuration values against the node type before saving, rejecting a capacity below one or a non-positive arrival rate.
FR-12	Simulation Engine	The system shall compile the graph into a Python SimPy script and execute it client-side through Pyodide, performing no simulation computation on the server.
FR-13	Simulation Engine	The system shall accept run parameters for duration, random seed and replication count before a run begins.
FR-14	Simulation Engine	The system shall execute the run inside a Web Worker so that the interface remains responsive throughout.
FR-15	Simulation Engine	The system shall progress a run through the states Queued, Running and Completed or Failed, timestamping each transition, and shall support pause, resume, stop and speed adjustment.
FR-16	AI Assistant	The system shall convert a natural-language description of a scenario into a candidate node-and-edge graph.
FR-17	AI Assistant	The system shall schema-validate a generated graph and allow the user to preview, accept or discard it before the live project is modified.
FR-18	AI Assistant	The system shall inspect a completed run, diagnose the bottleneck and propose a concrete corrective change applicable in one action.
FR-19	KPI Dashboard	On completion of a run the system shall compute and display utilisation, average and percentile waiting time, throughput and queue length for each relevant node.
FR-20	KPI Dashboard	The system shall identify and visually flag the node or nodes with the highest bottleneck score.
FR-21	Monte Carlo	The system shall execute a model over twenty, fifty or one hundred replications and report ninety-five per cent confidence intervals, with side-by-side baseline and optimised comparison.
FR-22	Digital Twin	The system shall provide animated two-dimensional playback of entity movement with play, pause, step and speed controls.
FR-23	Template Gallery	The system shall offer starter templates across all six domains, each instantiating an independent copy owned by the requesting user.
FR-24	Export and Share	The system shall export a graph as JSON, results as CSV and an executive report as PDF, and shall generate a revocable read-only public share link that never confers edit access.
FR-25	Profile and Settings	The system shall allow a user to view and update profile details, change the account password, set default simulation preferences and delete the account with confirmation.
FR-26	Collaboration	The system shall permit multiple authenticated users to edit one project concurrently, broadcasting changes within 500 ms and displaying live cursors and presence indicators.
FR-27	Collaboration	The system shall resolve concurrent edits to the same node on a last-write-wins basis using a monotonic client timestamp, and shall prevent a Viewer from modifying anything.

1.4.4 Non-Functional Requirements

Table 1.5: Non-Functional Requirements

ID	Category	Requirement
----	----------	-------------

ID	Category	Requirement
NFR-01	Performance	Canvas interactions such as dragging and connecting shall respond within 100 ms, and the dashboard shall render a run producing up to 10,000 result rows within 3 seconds.
NFR-02	Responsiveness	Simulation execution shall never block the interface; long runs shall proceed on a background worker thread.
NFR-03	Determinism	A run repeated with an identical seed and configuration shall reproduce an identical event trace and identical metrics.
NFR-04	Security	Passwords shall be stored only as salted bcrypt hashes; OAuth tokens shall never be persisted in plain text; all traffic shall use TLS.
NFR-05	Access Control	Row-Level Security shall be enforced server-side on every table, so that authorisation cannot be bypassed by a modified client.
NFR-06	Usability	The interface shall be usable by a first-time non-technical user without a tutorial, using plain language in preference to simulation jargon.
NFR-07	Reliability	Project state shall persist automatically between sessions and across devices, with no explicit save action required.
NFR-08	Scalability	The platform shall support at least fifty concurrent collaborative editors on a single project without perceptible lag.
NFR-09	Portability	The application shall run unmodified on any evergreen WebAssembly-capable browser across Windows, macOS, Linux, Android and iOS.
NFR-10	Maintainability	Simulation logic shall remain isolated within the Pyodide worker module; interface and engine layers shall not share mutable state directly.
NFR-11	Data Integrity	Foreign-key constraints shall be enforced at the database, and graph-structural invariants additionally at the service layer as a second line of defence.
NFR-12	Availability	The hosted application shall target 99 per cent uptime across the frontend and the Supabase backend.

1.4.5 User Personas and Operational Workflows

Three personas guided our interface decisions, and each exercises a characteristic path through the system.

- **The student** — encounters queueing theory in a lecture and wants to see M/M/1 behaviour rather than derive it. This user opens a template from the gallery, alters the arrival rate, re-runs and compares the dashboards. The operative requirement is that a first meaningful result should be reachable within a minute of arriving on the platform.
- **The lecturer** — demonstrates congestion live in a classroom. This user needs the animated playback for visual explanation and the share link so that students can open the same model afterwards on their own devices without accounts.
- **The small-business owner** — is deciding whether a second service counter is justified. This user describes the shop to the AI assistant in plain English, accepts the generated graph, runs a Monte Carlo analysis over fifty replications, and reads the confidence intervals and the cost panel before committing money.

The common operational workflow that emerges is: authenticate, create or open a project, construct the graph either manually or through the assistant, configure node parameters, validate the structure, execute the run, inspect the analytics, optionally re-run under a modified scenario, and finally export or share the outcome. Chapter 4 models this path formally as an activity diagram.

1.5 Feasibility Study

Before committing to the design we examined whether the project was viable along six dimensions. The technical question was the one that genuinely mattered, since the whole architecture depended on its answer, and it is treated first.

1.5.1 Technical Feasibility

The technologies we selected — Next.js, React, TypeScript, Supabase, React Flow, Pyodide and SimPy — are all widely adopted, thoroughly documented and supported by substantial developer communities, which reduced both development risk and the long-term maintenance burden.

The principal technical question was whether a Python-based discrete event simulation could realistically execute inside a browser. Pyodide resolves it by compiling the CPython interpreter to WebAssembly, which allows the SimPy engine to run client-side with no installation on the user's machine. We verified this early, during the second week of development, precisely because the entire architecture depended on the answer. Two constraints emerged from that investigation and were designed around rather than ignored. First, the Pyodide runtime is a download of several megabytes, so it is fetched lazily on first run and cached thereafter. Second, WebAssembly executes within the browser's memory allocation, which places a practical ceiling on model size; hosting execution in a Web Worker keeps the interface responsive when a large model is running and prevents a heavy simulation from freezing the page.

Canvas rendering posed a smaller but real question. Manipulating several hundred interactive nodes in the DOM can degrade sharply, so we adopted React Flow, which is built for node-based interfaces and applies viewport transforms and culling, and Pixi.js for the animated playback, where WebGL rendering handles many moving entities far better than DOM elements would. Both choices were confirmed by the benchmarks reported in Chapter 7.

1.5.2 Economic Feasibility

The project is economically viable because it rests almost entirely on open-source software and on cloud services with substantial free tiers. React, Next.js, TypeScript, SimPy, Pyodide, React Flow, Zustand, Recharts and Pixi.js are all free to use and carry no licence fee, in contrast with the commercial packages discussed in Section 1.3.

The architectural decision to run simulations client-side has a direct economic consequence that is worth stating plainly. Because the user's own browser performs the computation, there is no server-side compute cost that grows with the number or size of simulations. A thousand users each running an hour-long simulation impose no more server load than a thousand users reading a static page. The only recurring cost is the Supabase tier supporting the database, authentication and realtime channels, which remains within the free allowance at the usage levels expected during the academic phase and initial deployment. AI assistance consumes Gemini API quota, which is metered but modest, since a request is issued only when the user explicitly asks for generation or diagnosis.

Table 1.6: Cost Comparison with Commercial Alternatives

Item	Commercial DES tool	JustCmul8
Software licence	USD 3,000 – 15,000 per seat per year	None — open-source stack
Installation and IT effort	Required on every machine	None — runs in the browser
Simulation compute	Local workstation or licensed server	User's browser — no server cost
Backend hosting	Not applicable or self-managed	Supabase free tier during the academic phase
AI assistance	Generally unavailable	Gemini API, metered per explicit request
Training	Formal instruction generally necessary	Not required — no-code interface

1.5.3 Operational Feasibility

Operationally the platform is straightforward to adopt. The drag-and-drop interface, supported by the AI assistant, allows a model to be constructed without programming or prior simulation experience, and because everything is reached through a browser, no software need be installed or maintained on any device. The intended users — students, teachers, researchers and small-business owners — are already familiar with modern web applications, so formal training and dedicated technical support are unnecessary. Deliberate interface choices reinforce this: node palettes are labelled in the vocabulary of the chosen domain, so a user modelling a bank sees “Teller” rather than “Resource”, and validation errors name the offending node and state the remedy instead of reporting an engine exception.

1.5.4 Schedule Feasibility

The development plan was realistic within the academic timeframe. The work was divided into discrete functional modules and sequenced according to their dependencies, with core capability delivered before advanced features such as AI assistance and real-time collaboration. Testing, debugging and integration ran alongside development rather than following it, which reduced project risk and improved quality. The single most consequential scheduling decision was to begin the simulation engine in the second week of July, in parallel with backend work, so that the hardest technical uncertainty was resolved while there was still time to respond to it. Chapter 2 sets out the twenty-four-week schedule in full.

1.5.5 Legal and Ethical Feasibility

Every dependency is used under an open-source licence or within the terms of a free service tier, so no licensing obligation is outstanding. On the ethical side, user data is protected by role-based access control and PostgreSQL Row-Level Security, confining each user to their own projects and those explicitly shared with them, and passwords are never stored in recoverable form. Two further decisions deserve mention. Account deletion is implemented as a genuine cascading removal of the user's data rather than a flag, and public share links are revocable at any time, after which the previously circulated URL ceases to resolve. AI-generated graphs are always presented for human review before they alter a project, so the assistant advises but never acts unilaterally.

1.6 Details of Hardware and Software Used

Two configurations are relevant to this project and are easily confused. The first is the environment in which we developed the platform, which is comparatively demanding because it runs an editor, a development server and the browser runtime at once. The second is what an end user actually needs, which is considerably less, and it is the second that determines how widely the platform can be adopted.

1.6.1 Development Environment

Table 1.7: Development Hardware and Software Configuration

Component	Specification	Purpose
Processor	Intel Core i5 (10th gen) / AMD Ryzen 5 or higher	Concurrently running the development server, browser and Pyodide runtime
Memory	8 – 16 GB RAM	Editor, browser with several tabs and local build processes together
Storage	256 GB SSD or larger	Project sources, Node.js dependency tree and build artefacts
Display	1920 × 1080 or higher	Canvas, configuration panel and dashboard side by side
Operating system	Windows 11 (primary), macOS and Linux verified	Cross-platform verification of the development toolchain
Network	Stable broadband connection	Package installation, Supabase access and Gemini API calls
Editor	Visual Studio Code with TypeScript and ESLint extensions	Primary development environment
Version control	Git with GitHub remote	Source history, branching and collaborative development
API testing	Postman	Verification of API routes and Supabase endpoints
Runtime	Node.js 20 LTS with npm	Dependency management and the Next.js build pipeline
Browser tooling	Chrome DevTools, Lighthouse	Profiling, memory inspection and performance measurement

1.6.2 Target Client Requirements

Because JustCmul8 is a web application, end users require no specialised hardware. Any reasonably current desktop, laptop, tablet or smartphone with a modern browser is sufficient. One qualification follows from the architecture: since the engine executes client-side through WebAssembly, devices with limited memory will run large models more slowly, and the Web Worker execution path exists specifically to keep the interface usable when that happens.

Table 1.8: Minimum and Recommended Client Requirements

Requirement	Minimum	Recommended
-------------	---------	-------------

Requirement	Minimum	Recommended
Browser	Chrome 90+, Firefox 88+, Edge 90+, Safari 15+ (WebAssembly support required)	Latest evergreen release of Chrome or Edge
Memory	4 GB RAM	8 GB RAM or more for large models
Processor	Dual-core, 1.6 GHz	Quad-core, 2.4 GHz or higher
Display	1366 × 768	1920 × 1080 or higher
Network	1 Mbps for loading and persistence	5 Mbps or better for real-time collaboration
Operating system	Any platform with a supported browser — Windows, macOS, Linux, Android, iOS	No platform-specific requirement
Local installation	None	None

1.6.3 Software and Technology Stack

Table 1.9: Technology Stack with Justification

Layer	Technology	Reason for selection
Frontend framework	Next.js 16 with the App Router	File-based routing, server-side rendering and API routes in a single framework
UI library	React 19	Component model suited to a complex, highly interactive editor
Language	TypeScript	Static typing across the codebase, catching node-parameter mismatches at compile time
Styling	Tailwind CSS v4	Utility-first styling giving consistent spacing and a coherent theme
Graph editor	React Flow (@xyflow/react)	Production-grade node-based canvas with viewport transforms, handles and edge routing
Simulation engine	Python SimPy via Pyodide (WebAssembly)	A real DES engine executing client-side; accuracy without server compute
State management	Zustand	Lightweight store with slice-based separation and no boilerplate reducers
Graph layout	Dagre	Automatic layered layout for AI-generated and template graphs
Charts	Recharts	Declarative, composable React charts for the KPI dashboard
2D animation	Pixi.js	WebGL rendering for smooth digital-twin playback of many entities
AI service	Google Gemini 2.0 Flash	Low-latency structured generation of graphs from natural language
Database and auth	Supabase (PostgreSQL, Auth, Realtime)	Managed Postgres with Row-Level Security, OAuth and WebSocket channels
Animation and scroll	Framer Motion, Lenis	Transition choreography and smooth scroll physics
Reporting	jsPDF with AutoTable, html-to-image	Client-side generation of executive PDF reports including canvas snapshots
Icons	Lucide React	Consistent icon set covering all fifteen node types
Deployment	Vercel	Native Next.js hosting with edge delivery and preview deployments

Every item in Table 1.9 is open-source or available under a free tier, which is what makes the economic case in Section 1.5.2 hold. The two selections that most shaped the architecture are Pyodide, which permits genuine Python execution in the browser and therefore removes the need for server-side compute altogether, and Supabase, whose Row-Level Security allowed authorisation to be enforced at the database rather than trusted to client code.

CHAPTER 2

SYSTEM PLANNING

2.1 List of Activity and Task

The development of JustCmul8 was structured so that the work progressed in an orderly and verifiable sequence. We divided the project into a set of clearly defined activities, estimated a realistic duration for each, and recorded the dependencies between them so that no activity could begin before the work it relied upon was complete.

Because JustCmul8 is an entirely new system rather than an extension of an existing one, the implementation plan was organised around the eleven functional modules identified at the planning stage, overlaid on the conventional phases of the Software Development Life Cycle: requirement analysis, literature survey, system design, implementation, testing, documentation and deployment. The schedule spans twenty-four working weeks, beginning in the first week of July 2026 and concluding in the last week of December 2026.

Dependency mapping served two purposes. It prevented any module from being implemented before its prerequisites existed, and — equally important — it identified which modules were technically independent of one another and could therefore be developed in parallel, shortening the overall duration without compromising quality.

Phase 1 — Initiation and Analysis

The project opened with initiation and requirement gathering, during which we established the objectives, identified the intended users and elicited the functional and non-functional requirements recorded in Chapter 1. A literature survey followed, in which we examined the commercial tools discussed in Section 1.3, studied the SimPy and Pyodide documentation, and reviewed the standard treatment of queueing models. System analysis then consolidated these findings into a coherent specification and reconciled the database schema against the requirements before any code was written.

Phase 2 — Design

Design work proceeded on two fronts simultaneously. Interface design produced the workspace layout, the node palette organisation and the configuration panel behaviour, while database design produced the normalised schema set out in Section 4.7. Storing the graph as normalised node and edge rows rather than as a single JSON document was settled at this stage, and that decision later made per-entity collaborative synchronisation feasible.

Phase 3 — Development

Development accounted for the largest share of the schedule and ran as three concurrent streams. The backend stream delivered authentication, project management, the API routes and the data layer. The simulation stream delivered the engine, the AI assistant, the KPI dashboard and the template gallery. The frontend stream delivered the graph editor, the configuration panel, and later the export, profile and collaboration modules.

The most consequential scheduling decision concerned Module 5, the simulation engine, which was started in the second week of July and developed in parallel with the backend rather than waiting for it. Executing the Python SimPy library inside a browser through Pyodide and WebAssembly was the principal technical risk in the whole project, and every dependent module — the AI assistant, the KPI dashboard, the Monte Carlo facility and the digital twin — presumed it would work. Beginning it early gave us time to resolve compatibility problems, verify simulation accuracy against known analytical results, tune browser performance and stabilise the interface that the other modules would call. Had we discovered a fundamental obstacle in October instead of July, there would have been no opportunity to redesign around it.

Authentication and project management were likewise prioritised, since they establish user identity, project storage and cloud persistence, and collaborative editing, sharing and user-specific features all build directly upon them. Meanwhile the graph editor and configuration panel allowed models to be constructed while the execution engine was still being refined. The remaining modules — the template gallery, export and share, profile and settings, and real-time collaboration — were introduced once the core modelling and execution path was stable.

Phase 4 — Testing

Testing was not deferred to the end. Unit testing began in the fourth week of September, while several modules were still under construction, so that defects in the earlier modules could be corrected while the later ones were being written. Integration testing, bug fixing and user acceptance testing followed in sequence, with an iterative loop between integration testing and bug fixing that repeated until no defects remained outstanding.

Phase 5 — Documentation and Deployment

Documentation ran continuously for twenty-three of the twenty-four weeks. This was a deliberate choice rather than a convenience: writing each module's documentation as it was built kept the record accurate, whereas reconstructing it afterwards would have invited omissions. Deployment concluded the schedule with a final review followed by release to the hosting platform.

2.2 Timeline for Each Task

Table 2.1 lists every scheduled activity with its phase, estimated duration, planned start and finish weeks, and the activities on which it depends. Weeks are expressed relative to the calendar month, W1 denoting the first week of that month.

Table 2.1: Activity Schedule with Durations and Dependencies

Sr.	Task / Activity	Phase	Duration	Start	End	Depends On
1	Project Initiation	Initiation / Analysis	2 weeks	Jul W1	Jul W2	—
2	Requirement Gathering	Initiation / Analysis	3 weeks	Jul W1	Jul W3	—
3	Literature Survey	Initiation / Analysis	2 weeks	Jul W2	Jul W3	Project Initiation
4	System Analysis	Initiation / Analysis	2 weeks	Jul W3	Jul W4	Requirement Gathering
5	UI/UX Design	Design	3 weeks	Jul W4	Aug W2	System Analysis

Sr.	Task / Activity	Phase	Duration	Start	End	Depends On
6	Database Design	Design	3 weeks	Jul W4	Aug W2	System Analysis
7	Module 1: Authentication	Development	3 weeks	Aug W1	Aug W3	Database Design
8	Module 2: Project Management	Development	3 weeks	Aug W2	Aug W4	Module 1
9	API Development	Development	5 weeks	Aug W1	Sep W1	Database Design
10	Backend Development	Development	8 weeks	Aug W1	Sep W4	API Development
11	Module 3: Visual Graph Editor	Development	4 weeks	Aug W2	Sep W1	UI/UX Design, Module 2
12	Module 4: Node Configuration Panel	Development	3 weeks	Aug W3	Sep W1	Module 3
13	Frontend Development	Development	9 weeks	Aug W2	Oct W2	UI/UX Design
14	Module 5: Simulation Engine	Development	8 weeks	Jul W2	Sep W1	Parallel track — runs ahead of backend
15	Module 6: AI Assistant	Development	4 weeks	Sep W2	Oct W1	Module 5
16	Integration	Development	5 weeks	Sep W3	Oct W3	Modules 1 – 6
17	Module 7: KPI Dashboard	Development	3 weeks	Sep W3	Oct W1	Module 5
18	Module 8: Template Gallery	Development	3 weeks	Sep W4	Oct W2	Module 3
19	Module 9: Export & Share	Development	3 weeks	Oct W1	Oct W3	Module 7
20	Module 10: User Profile & Settings	Development	3 weeks	Oct W1	Oct W3	Module 1
21	Module 11: Real-Time Collaboration	Development	4 weeks	Oct W2	Nov W1	Module 3
22	Unit Testing	Testing	5 weeks	Sep W4	Oct W4	Modules in progress
23	Integration Testing	Testing	3 weeks	Oct W4	Nov W2	Integration complete
24	Bug Fixing	Testing	3 weeks	Nov W1	Nov W3	Integration Testing
25	User Acceptance Testing	Testing	2 weeks	Nov W3	Nov W4	Bug Fixing
26	Documentation	Documentation	23 weeks	Jul W1	Dec W3	Continuous, alongside every module
27	Final Review	Deployment	2 weeks	Dec W1	Dec W2	User Acceptance Testing
28	Deployment	Deployment	2 weeks	Dec W3	Dec W4	Final Review

Milestones

Table 2.2: Project Milestones

Milestone	Target	Completion criterion
M1 — Requirements baselined	Jul W3	Functional and non-functional requirements approved by the guide
M2 — Literature survey complete	Jul W3	Comparable tools evaluated and the technology stack fixed
M3 — Design frozen	Aug W2	Interface layout and database schema approved
M4 — Engine proof of concept	Sep W1	A SimPy model executing in the browser with verified output
M5 — Core modelling path working	Sep W1	A graph can be built, configured and executed end to end
M6 — Analytics operational	Oct W1	KPI dashboard rendering utilisation, wait time and throughput
M7 — Feature complete	Nov W1	All scheduled modules implemented and integrated
M8 — Testing signed off	Nov W4	Unit, integration and acceptance testing passed
M9 — Deployed	Dec W4	Application live on the hosting platform and report submitted

Milestone M4 warrants emphasis. It was placed early specifically because it retires the project's dominant technical risk, and every milestone after M4 assumes that browser-hosted Python execution has already been demonstrated to work.

2.3 Gantt Chart and Activity Diagram

The schedule tabulated in the previous section is presented here in two graphical forms, because each answers a different question. The Gantt chart shows when work happens and for how long; the activity diagram shows in what logical order, subject to which decisions, and along which concurrent paths.

2.3.1 Gantt Chart

A Gantt chart is a scheduling instrument that presents project activities as horizontal bars positioned against a calendar, so that the duration of each activity and the overlap between activities can be read directly. It is used in software engineering for task scheduling, resource allocation and progress tracking. Its principal value is that it decomposes a project into manageable activities, assigns each a realistic timeframe, and makes scheduling conflicts visible early enough to be corrected.

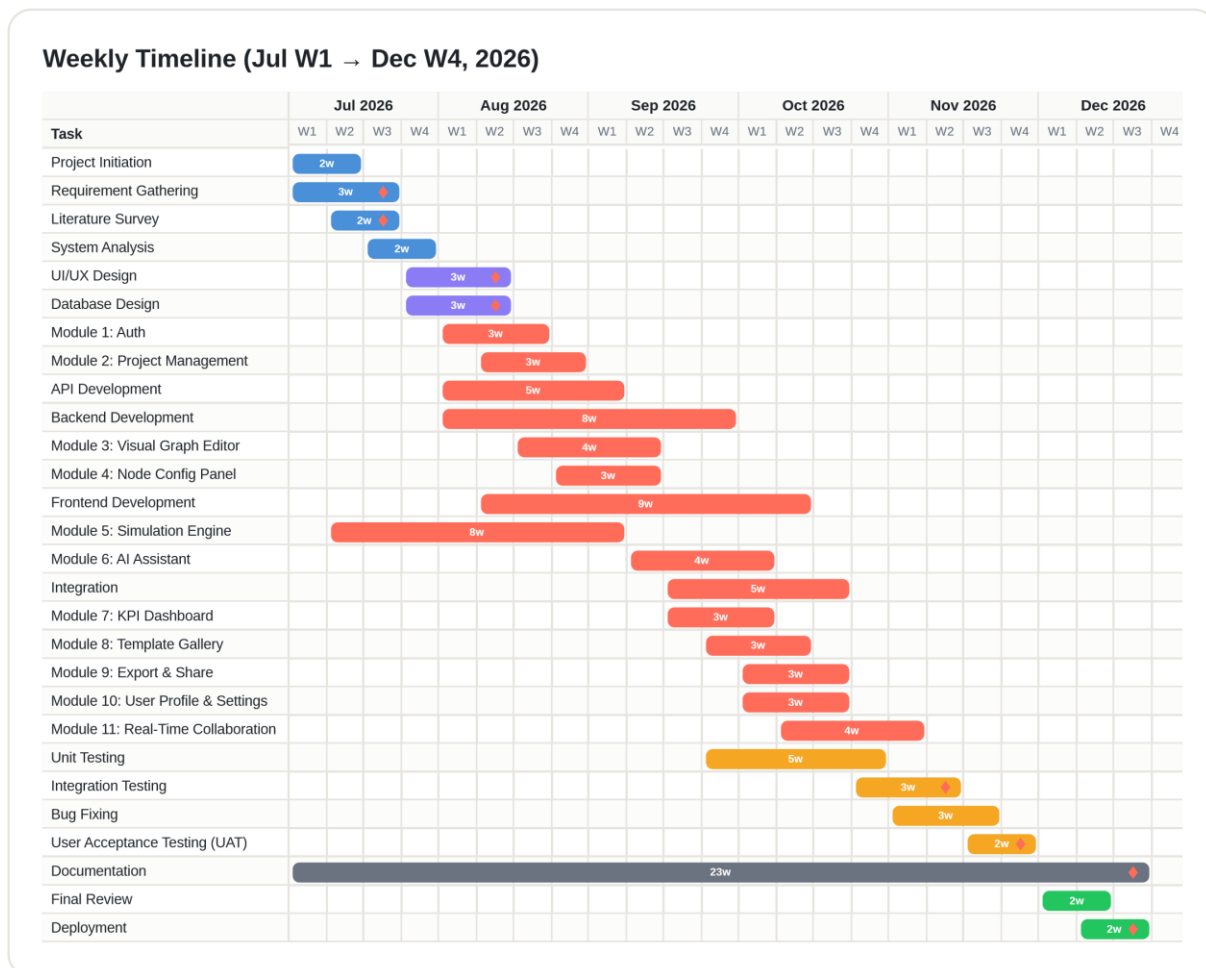


Figure 2.1: Weekly Gantt Chart of the JustCmul8 Development Schedule (July W1 – December W4, 2026)

The chart is colour-coded by phase, each colour corresponding to a stage of the Software Development Life Cycle. The blue bars at the top represent the initiation and analysis phase — project initiation, requirement gathering, literature survey and system analysis — which occupy the four weeks of July and establish the project objectives, the system requirements and the technology assessment on which the design depends.

The purple bars mark the design phase, comprising interface and database design across roughly three weeks from late July. Both depend on the completion of system analysis and together define the structures the modules are implemented against.

Coral bars denote the development phase, which dominates the timeline. Alongside API and backend development sit the two foundational modules, authentication and project management. The frontend modules, the visual graph editor and the node configuration panel, are developed over several weeks and together constitute the modelling surface the user actually manipulates. The simulation engine bar is the one to read most carefully: it begins in the second week of July and runs concurrently with backend development, for the reason given in Section 2.1. The AI assistant follows once the engine has stabilised, and the remaining modules — KPI dashboard, template gallery, export and share, profile and settings, and real-time collaboration — are delivered through September and October, each after its own prerequisite.

Amber bars cover testing: unit testing, integration testing, bug fixing and user acceptance testing. These bars deliberately overlap the development bars, because testing began before all modules were finished so that early defects could be fixed while later modules were still being written. This parallelism improved software quality and lowered overall project risk.

The single long grey bar is documentation, spanning almost the whole period. Documentation was treated as a continuous obligation carried out as analysis, design, implementation and testing proceeded, which is why the project deliverables remained accurate and current rather than being reconstructed retrospectively. The green bars at the end represent final review and deployment, occupying the closing weeks of December once user acceptance testing had been passed.

Taken together, the chart provides a week-by-week view of the twenty-four-week schedule, establishes a logical order for implementing the scheduled modules, allows progress to be

tracked against plan throughout the project, and arranges the work so as to minimise delay and rework.

2.3.2 Activity Diagram of the Development Lifecycle

An activity diagram is a behavioural model that shows the flow of activities through a system, including sequencing, decision points and parallel execution. Unlike an entity relationship diagram, which describes how data is organised, an activity diagram describes dynamic behaviour — the order in which things happen and the conditions under which the path branches. Drawn in UML notation, it uses activity nodes, control flows, decision nodes, fork and join bars and start and end markers.

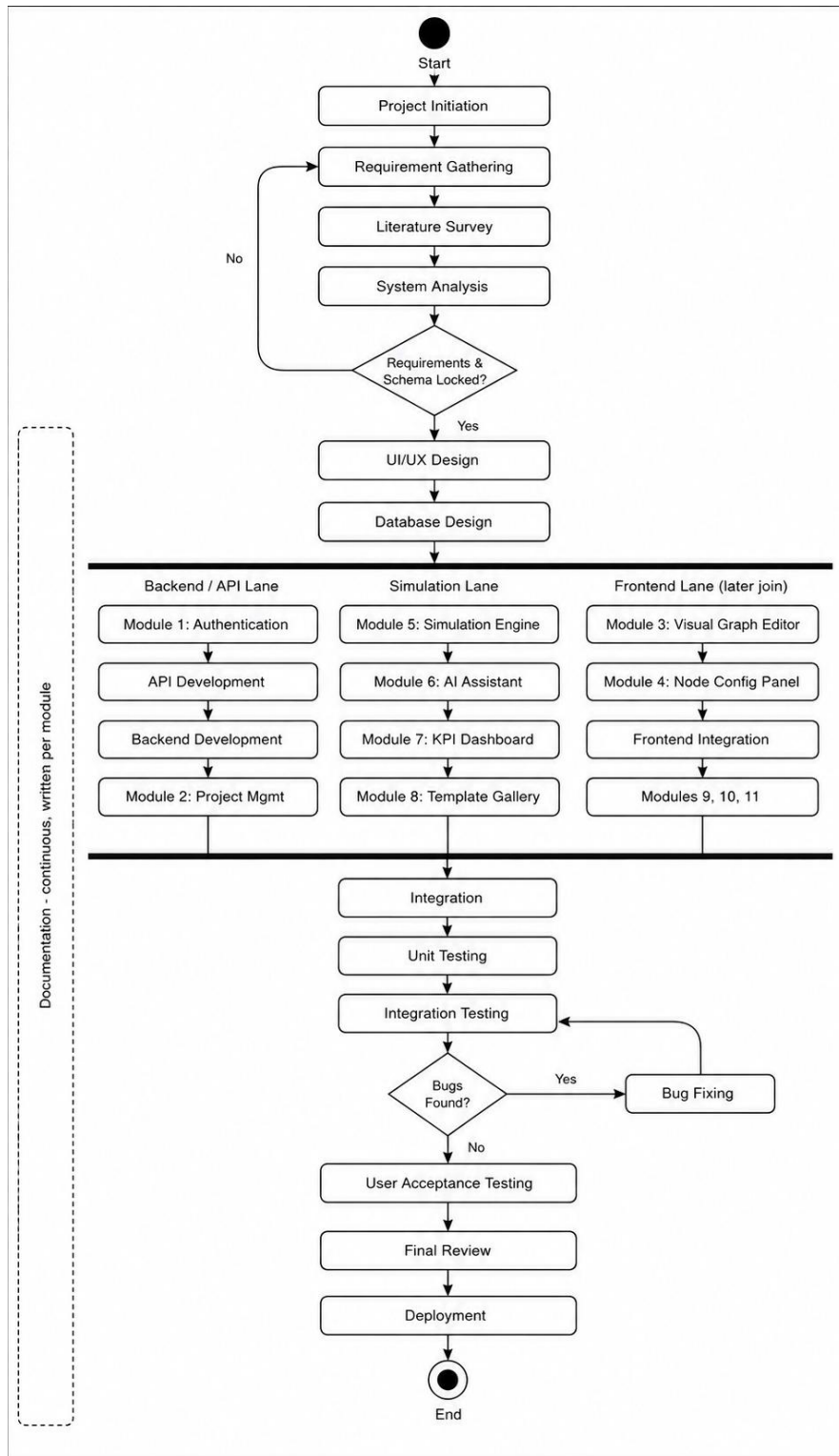


Figure 2.2: Activity Diagram of the JustCmul8 Development Lifecycle

The diagram traces the same six phases represented in the Gantt chart, with documentation shown as a continuous supporting activity running alongside the rest rather than as a terminal step.

The flow begins at the start node and proceeds through project initiation, requirement gathering, literature survey and system analysis. A decision node then asks whether requirements and the database schema are locked. If further refinement is needed, control returns to requirement gathering and the loop repeats; only once both are settled does the flow advance to design, where interface and database design are completed.

Development is expressed as a fork into three concurrent lanes. The backend and API lane carries authentication, API development, backend development and project management. The simulation lane carries the simulation engine, followed by the AI assistant, the KPI dashboard and the template gallery; placing the engine at the head of this lane reflects the early-start decision discussed earlier. The frontend lane carries the visual graph editor, the node configuration panel, frontend integration, and the export, profile and collaboration modules. Representing these as parallel lanes is not merely a drawing convention — it records that the three streams had no mutual dependencies and could genuinely proceed at the same time.

A join bar reunites the three lanes, after which system integration combines the modules into a single application. Testing then begins with unit testing and proceeds to integration testing, where a second decision node asks whether defects were found. If so, control passes to bug fixing and returns to integration testing, and this cycle repeats until the build is clean. Once the testing criteria are met, the flow advances to user acceptance testing.

The final segment comprises final review and system deployment, after which control reaches the end node. Account deletion is drawn as a separate flow, since it is initiated independently from settings and proceeds through confirmation, cascading data removal and acknowledgement without touching the main development path.

The Gantt chart and the activity diagram are complementary. The Gantt chart answers when work happens and for how long; the activity diagram answers in what logical order, subject to which decisions, and along which concurrent paths. Read together they describe both the calendar and the control flow of the development strategy adopted for JustCmul8.

CHAPTER 3

SOFTWARE AND HARDWARE MODULES

JustCmul8 is composed of twelve software modules, each owning a distinct responsibility, exposing a defined interface to the others, and developed and tested as a unit before integration. This chapter specifies what each module does, how it was implemented and which requirements from Section 1.4.3 it satisfies. The chapter closes with the hardware considerations that follow from executing simulations on the client.

One point of reconciliation is worth stating at the outset. The development schedule presented in Chapter 2 was drawn against an eleven-module breakdown, which was the decomposition in force when the plan was prepared. During implementation the AI capability proved substantial enough to separate into two modules — the assistant that constructs models from natural language, and the optimiser that diagnoses a completed run and proposes a remedy — since the two draw on different inputs and are invoked at different points in the workflow. The delivered system therefore comprises twelve modules, and the numbering used below follows the updated project synopsis.

Table 3.1: Module Overview and Requirement Coverage

No.	Module	Primary responsibility	Requirements
1	Platform and Data Layer	Authentication, project lifecycle, cloud persistence, profile	FR-01 to FR-06, FR-25
2	Visual Graph Editor	Canvas-based construction of the simulation graph	FR-07 to FR-09
3	Node Configuration Panel	Parameter and cost entry with per-type validation	FR-10, FR-11
4	Simulation Engine	Code generation and in-browser SimPy execution	FR-12 to FR-15
5	AI Assistant	Natural-language generation of candidate models	FR-16, FR-17
6	AI System Optimiser	Bottleneck diagnosis and one-click corrective fixes	FR-18
7	KPI Dashboard and Results	Metric computation, visualisation, cost and ROI	FR-19, FR-20
8	Monte Carlo and Scenario Comparison	Replicated runs, confidence intervals, baseline comparison	FR-21
9	Digital Twin Animated Playback	Animated 2D entity flow with transport controls	FR-22
10	Simulation Template Gallery	Reusable starter models across six domains	FR-23
11	Executive Report and Share	PDF, CSV, JSON export and read-only share links	FR-24
12	Real-Time Collaboration	Concurrent multi-user editing with presence	FR-26, FR-27

3.1 Module 1 — Platform and Data Layer

Purpose

This module establishes who a user is, what they own and where that work is kept. Every other module depends on it, since a project cannot be created, shared or persisted until an identity exists to own it. It consolidates authentication, project management, cloud storage and account settings into a single foundation layer built on Supabase.

Functional responsibilities — identity and access

- Registration and login by email and password, requiring a display name, a unique email address and a password meeting minimum complexity rules.
- One-click registration and login through Google or GitHub OAuth.
- Session protection, with authenticated sessions persisting across browser visits and protected routes guarded before any project data is fetched.
- Password reset through a time-limited, single-use email link, and in-session password change.
- Secure logout invalidating the session, and account deletion that cascades through all owned data after password confirmation.

Functional responsibilities — projects and profile

- Creation of a project with selection of one of the six simulation domains, optionally seeded from a template.
- Renaming, duplication, archiving and deletion of projects, with deletion restricted to the owner.
- Automatic persistence of graph and configuration state to cloud storage on every change, with no explicit save action.
- A dashboard listing every project the user owns or collaborates on, with domain, last-modified time and quick actions.
- Reopening of a saved project with canvas, configuration and prior results restored intact.
- Profile management covering display name and avatar, and preferences including the default simulation domain for new projects.

Implementation

Authentication is delegated to Supabase Auth. Passwords are stored exclusively as salted bcrypt hashes, and the application never handles a plaintext password beyond the submitting form. Sessions are carried in HTTP-only cookies and refreshed by Next.js middleware, which also guards the protected routes. The decisive design point is that authorisation is not enforced in the client: PostgreSQL Row-Level Security policies attached to every table evaluate the requesting user's identity at query time, so a modified client cannot read another user's rows even if it constructs the request by hand.

Projects are rows keyed by UUID and owned through a foreign key to the user. Automatic persistence is debounced rather than immediate: changes are batched and flushed after a brief idle interval, which avoids saturating the connection during a continuous node drag while keeping the stored state effectively current. Duplication copies the node and edge rows into a fresh project, so that the copy is genuinely independent and later edits to it cannot mutate the original.

3.2 Module 2 — Visual Graph Editor

Purpose

The graph editor is the surface on which modelling actually happens and the module most responsible for the platform's no-code claim. It allows a process to be expressed as a directed graph through direct manipulation alone.

Functional responsibilities

- A drag-and-drop canvas onto which nodes are placed from a palette and repositioned freely.
- Support for fifteen node types spanning the six simulation domains.
- Creation and deletion of directed edges between node handles, with real-time edge re-routing as nodes are moved.
- Zoom, pan and fit-to-view navigation, with viewport state preserved across sessions.
- Selection, multi-selection, copying and deletion of nodes and edges.
- Structural validation before execution, rejecting disconnected graphs, orphaned edges and edges whose endpoints belong to different projects.

- Automatic layout of AI-generated and template graphs, and live re-rendering as collaborators make changes.

Implementation

The canvas is built on React Flow, which supplies the viewport transform, handle-based connection model and edge routing that would otherwise have consumed a disproportionate share of the schedule. Custom node components render each of the fifteen types with its own icon, colour and port arrangement. Labels are drawn from a domain-specific registry, so a user modelling a bank sees “Teller” where a user modelling a factory sees “Machine”, although both are the same underlying resource node. Automatic layout uses Dagre with a strategy selected per domain — a left-to-right bank-branch arrangement for human queues, cardinal approaches for traffic junctions, a topological pipeline for liquid flow — because a single generic layout produced diagrams that were technically correct but unintuitive.

Table 3.2: The Fifteen Node Types Supported by the Simulation Engine

Internal type	Display name	Behaviour	Principal domains
source	Source / Arrival Generator	Generates entities into the model at intervals drawn from a chosen distribution, or from a fixed arrival schedule	All six domains
queue	Queue	A waiting line of finite or infinite capacity, operating under FIFO, LIFO or priority discipline	All six domains
resource	Resource / Server	A capacity-limited service point; entities seize it, hold it for a sampled service time, then release it	All six domains
service	Service Step	A single processing operation with its own duration, used where no contention for capacity applies	Manufacturing, logistics
decision	Decision / Router	Routes arriving entities down one of several outbound edges according to assigned probabilities	All six domains
sink	Sink / Exit	Terminates an entity's journey and records its completion statistics for throughput and cycle-time analysis	All six domains
container	Container / Tank	Holds a continuously varying level of bulk material or liquid, sampled at event boundaries	Liquid and material flow
store	Store / Typed Buffer	A buffer holding distinguishable items, corresponding to SimPy's Store, from which specific item types may be withdrawn	Manufacturing, logistics
event_trigger	Event Trigger	Monitors a condition during the run and fires an event when it is satisfied, enabling conditional behaviour	Network, manufacturing
priority_resource	Priority Resource	A preemptive resource that suspends a lower-priority task when a higher-priority entity arrives	Human queues, healthcare
channel	Channel	A transmission medium imposing a propagation delay and optional loss on entities crossing it	Network and signal flow
broadcaster	Broadcaster	Duplicates an incoming entity to every outbound edge, producing one-to-many fan-out	Network and signal flow

Internal type	Display name	Behaviour	Principal domains
any_of	Any-Of Synchroniser	Waits on several concurrent events and proceeds as soon as the first of them completes	Network, manufacturing
all_of	All-Of Synchroniser	Waits until every one of several concurrent events has completed before proceeding	Manufacturing, logistics
interrupter	Interrupter	Interrupts an in-progress process, used to model breakdowns, preemption and abandonment	Manufacturing, human queues

The first six types in Table 3.2 constitute the classical discrete event vocabulary and suffice for most models. The remaining nine extend the engine to cases the classical set cannot express: continuous levels, typed buffers, preemption, propagation delay, fan-out and event synchronisation. The two synchronisers correspond directly to SimPy's AnyOf and AllOf constructs, which is what allows a generated script to express concurrent waiting without custom scheduling code. The project synopsis cited thirteen node types; the two synchronisers were added during implementation of the manufacturing and network domains, bringing the delivered total to fifteen.

Table 3.3: The Six Simulation Domains and Their Vocabularies

Domain	Typical scenarios	Example node labels	Layout strategy
Human Queues	Banks, clinics, ticket counters, reception desks	Teller, Counter, Waiting Area, Customer Arrival	bank_branch
Vehicle Traffic	Junctions, toll plazas, fuel stations, car washes	Approach, Signal, Toll Booth, Pump	intersection, linear_lane
Liquid / Material Flow	Tanks, pipelines, mixing and bottling plants	Reservoir, Pipe, Valve, Mixer	pipeline
Manufacturing Lines	Assembly lines, machining cells, inspection bays	Machine, Buffer, Inspection, Assembly	assembly_line
Logistics / Warehousing	Depots, sorting hubs, order fulfilment	Inbound Dock, Picking, Packing, Dispatch	warehouse
Network / Signal	Packet routing, message buses, sensor networks	Transmitter, Router, Link, Receiver	network_topology

3.3 Module 3 — Node Configuration Panel

Purpose

Placing a node states what kind of thing it is; configuring it states how it behaves. This module supplies the parameter entry that turns a topology into an executable model, and it does so through ordinary form controls rather than a scripting field.

Functional responsibilities

- A side panel that opens for the selected node and presents only the fields relevant to that node's type.
- Entry of arrival rates, service-time distributions and their parameters, capacities, queue disciplines, routing probabilities and propagation delays.
- Selection among the five supported probability distributions, with the parameter fields changing to match the chosen distribution.
- Entry of the three cost parameters — resource hourly rate, buffer holding cost and waiting-penalty cost — consumed by the return-on-investment analysis in Module 7.
- Validation of every value against the node type before it is saved, rejecting a capacity below one, a non-positive arrival rate, a negative standard deviation, or routing probabilities that do not sum to one.
- Automatic persistence of each change and immediate propagation to the underlying model.

Implementation

Configuration is stored as a JSON object scoped to the node type's schema, which keeps the database schema stable as new node types are introduced while still permitting type-specific validation in the service layer. Validation runs at the point of entry and again before execution, since a graph may be edited collaboratively between those two moments. Error messages name the offending node and state the corrective action, because reporting a raw engine exception to a non-technical user would defeat the purpose of the no-code interface.

Table 3.4: Cost Parameters and Their Role in ROI Analysis

Parameter	Applies to	Contribution to total system cost
Resource hourly rate	Resource and priority-resource nodes	Rate multiplied by the resource's busy time over the run
Buffer holding cost	Queue, store and container nodes	Cost per item per unit time, weighted by mean queue length

Parameter	Applies to	Contribution to total system cost
Waiting-penalty cost	Queue nodes	Penalty per unit of entity waiting time, capturing the cost of poor service

Separating these three costs is what allows the optimiser in Module 6 to make a defensible recommendation. Adding a server raises the resource cost but lowers both the holding and the waiting-penalty cost, and only by quantifying all three can the platform state whether the change pays for itself rather than merely shortening the queue.

3.4 Module 4 — Simulation Engine

Purpose

This module is the technical core of the project. It converts the visual graph into an executable discrete event simulation and runs it entirely within the browser, using a genuine Python engine.

Functional responsibilities

- Compilation of the node-and-edge graph into a Python SimPy script reflecting the configured parameters.
- Execution of that script client-side through the Pyodide WebAssembly runtime, with no simulation computation on the server.
- A Web Worker execution path so that the interface remains responsive for the duration of the run.
- Acceptance of run parameters for duration, random seed and replication count before a run begins.
- Progression of the run through Queued, Running and Completed or Failed with each transition timestamped, and support for pause, resume, stop and speed adjustment.
- Streaming of progress and of informational, warning and error log events to the interface while the run proceeds.
- A live Python code inspector exposing the generated script, with one-click export as a Jupyter notebook for reproducibility.

Implementation

Execution proceeds in four stages. The graph is first validated structurally, using depth-first traversal over an adjacency list to confirm connectivity and detect cycles that would deadlock. The code generator then walks the graph in topological order and emits a SimPy script in which each node becomes a process or a resource and each edge becomes a routing decision. The script is passed to the worker, where Pyodide executes it, and the resulting event trace is returned to the main thread for analysis.

Two implementation details deserve record. The random seed is threaded explicitly through every sampler so that runs are reproducible: an identical seed and configuration reproduce an identical trace, which makes the determinism requirement NFR-03 testable and makes Monte Carlo replication meaningful, since each replication receives a distinct derived seed. Second, the generated script is deliberately readable and is exposed through the code inspector, with notebook export alongside it. A student can therefore see the Python their diagram produced and re-run it independently, which turns the platform into a teaching aid for SimPy rather than an opaque black box, and makes any published result independently verifiable.

Illustration of generated code

The fragment below shows the shape of the output for a single-queue, two-server model, abbreviated for presentation.

```
import simpy, random

random.seed(SEED)
env = simpy.Environment()
teller = simpy.Resource(env, capacity=2)
stats = {'arrived': 0, 'completed': 0, 'wait': []}

def customer(env, name):
    t_arrive = env.now
    with teller.request() as req:
        yield req
        stats['wait'].append(env.now - t_arrive)
        yield env.timeout(random.expovariate(1.0 / 4.0))
        stats['completed'] += 1

def source(env):
    i = 0
    while True:
        yield env.timeout(random.expovariate(1.0 / 3.0))
        stats['arrived'] += 1
        env.process(customer(env, f'cust-{i}'))
        i += 1

env.process(source(env))
env.run(until=DURATION)
```

3.5 Module 5 — AI Assistant

Purpose

This module lowers the conceptual barrier rather than the technical one. A user who does not know how a process should be represented as a graph can describe it in plain English and receive a candidate model instead.

Functional responsibilities

- Conversion of a natural-language scenario description into a candidate graph of nodes and edges with plausible parameters.
- Schema validation of the generated graph — confirming valid node types and the absence of orphaned edges — before it is offered to the user.
- Preview, acceptance or rejection of a generated graph, so the live project is never modified without explicit consent.

- Conversational assistance, answering questions about simulation concepts and about the user's own model in non-technical language.
- Suggestion of appropriate node structure for an incomplete model.
- Logging of every request, prompt and response for traceability.

Implementation

Requests are proxied through a Next.js API route to Google Gemini 2.0 Flash, which keeps the API key on the server and out of the client bundle. The prompt is assembled from a system prompt describing the available node types and their parameter schemas together with the current graph; supplying the node-type registry in the prompt is what constrains the model to emit graphs the engine can actually execute. Every generated graph is nevertheless validated against the schema before the user sees it, and any graph referencing an unknown node type or containing an orphaned edge is rejected rather than displayed. Accepted graphs are passed through the Dagre layout engine so that they arrive on the canvas already arranged legibly.

3.6 Module 6 — AI System Optimiser

Purpose

Where Module 5 helps a user build a model, this module helps them act on one. After a run it inspects the computed metrics, explains which node is constraining the system and why, and proposes a specific change that can be applied to the graph in a single action.

Functional responsibilities

- Post-run inspection of server utilisation, queue waiting times and bottleneck scores produced by Module 7.
- Diagnosis of the constraining node, expressed in terms a non-specialist can act on.
- Proposal of a concrete corrective change — adding capacity at a resource, rerouting traffic at a decision node, enlarging a buffer.
- A one-click Apply Fix action that modifies the graph to implement the proposal.
- Quantification of the expected benefit against the cost parameters, so the recommendation carries a return-on-investment estimate.
- Retention of the pre-change graph as a baseline, enabling the before-and-after comparison offered by Module 8.

Implementation

The optimiser is grounded in the analytics output rather than in the raw event trace, so its recommendations refer to computed utilisation and queue statistics instead of impressions. A flow analysis step first determines which node the graph's structure makes it plausible to change — raising the capacity of a node that is not the constraint would achieve nothing — and only the resulting candidate is put to the language model for explanation and framing. Applying a fix is implemented as an ordinary graph mutation, which means it is undoable, collaboratively broadcast and persisted by exactly the same paths as a manual edit, with no special-case code. The proposal is always presented for approval: the optimiser advises but never acts unilaterally.

3.7 Module 7 — KPI Dashboard and Results

Purpose

An event trace is not an answer. This module converts the raw output of a run into the performance indicators a decision actually rests on, and presents them so that the constraining node is evident without further analysis.

Functional responsibilities

- Computation and display of resource utilisation, average and percentile waiting time, throughput and queue length for each relevant node.
- Automatic identification of the bottleneck and prominent visual flagging of the node or nodes concerned.
- Interactive charts that update as results are produced, not only on completion of the run.
- Filtering of results by node, metric type and run.
- Presentation of the simulation event log alongside the metrics, so an anomalous figure can be traced to the events that caused it.
- A cost and ROI panel reporting total system cost and the return on any proposed change.

Metric definitions

Table 3.5: Key Performance Indicators and Their Computation

Metric	Definition	Input data
Utilisation	Fraction of the run duration for which a resource was occupied	Per-resource busy and idle time series
Average waiting time	Mean interval between an entity joining a queue and being served	Queue entry and exit timestamps per entity

Metric	Definition	Input data
Percentile waiting time	The 50th, 90th and 95th percentiles of the waiting distribution	Sorted per-entity waiting times
Throughput	Entities completing the model per unit of simulated time	Sink completion counts and run duration
Average queue length	Time-weighted mean number of entities waiting at a node	Queue length sampled at every event instant
Bottleneck score	Composite ranking combining sustained high utilisation with growing queue length	Per-node utilisation and queue length series
Total system cost	Sum of resource, holding and waiting-penalty costs over the run	Cost parameters from Module 3 with per-node time series

The bottleneck score deserves comment, because utilisation alone is a poor indicator. A resource may be fully utilised and yet be perfectly adequate if nothing is waiting behind it; conversely a queue may be growing in front of a resource whose utilisation is only moderate, because upstream routing is uneven. Combining sustained utilisation with the trend in queue length identifies the node genuinely constraining throughput rather than merely the busiest one.

3.8 Module 8 — Monte Carlo and Scenario Comparison

Purpose

A single stochastic run is one observation, not an estimate. This module repeats the run and reports the range within which the true behaviour lies, then allows two configurations to be compared against that measure of uncertainty.

Functional responsibilities

- Execution of a model over twenty, fifty or one hundred replications, each with an independently derived seed.
- Computation of the mean, standard deviation and ninety-five per cent confidence interval for every metric across replications.
- Rendering of confidence intervals as bands, so the significance of a difference is visible rather than inferred.
- Side-by-side comparison of a baseline scenario against an optimised one, with the difference reported per metric.
- Progress reporting during a long analysis, so the user is not left without feedback across a hundred runs.

Implementation

Replications execute sequentially within the worker, with progress posted after each. Seeds are derived deterministically from a single base seed, which means an entire Monte Carlo analysis is itself reproducible — a property that matters when a result must be defended or re-examined. The comparison view pairs the baseline retained by Module 6 against the current configuration, and reports the difference alongside both intervals, so that a user can see whether an apparent improvement exceeds the noise in the measurement.

3.9 Module 9 — Digital Twin Animated Playback

Purpose

Numbers describe congestion; animation shows it. This module replays a completed run as a moving two-dimensional picture of entities passing through the model, which in our experience communicates where a system is failing far more immediately than a chart.

Functional responsibilities

- Animated two-dimensional playback of entities flowing through the nodes and edges of the model.
- Highlighting of queue congestion, so that accumulating waiting lines are visually apparent as they build.
- Transport controls in the manner of a video player — play, pause, step and speed selection.
- Scrubbing to an arbitrary point in the run without re-executing the model.
- Synchronisation of the playback position with the corresponding event log entries.

Implementation

Playback is rendered with Pixi.js over WebGL rather than with DOM elements, because animating several hundred moving entities through the DOM degrades unacceptably. The animation is driven from the recorded entity journey rather than by re-executing the model, which makes scrubbing and speed changes immediate and guarantees that what is shown corresponds exactly to the run whose metrics are on screen. Because the journey is stored per entity with a timestamped sequence of node visits, the playback can be interpolated smoothly between discrete event instants without inventing behaviour the engine did not produce.

3.10 Module 10 — Simulation Template Gallery

Purpose

An empty canvas is an obstacle to a first-time user. This module supplies working models across all six domains, so that a user can begin from something that already runs and modify it — a considerably gentler introduction than construction from nothing.

Functional responsibilities

- A gallery of starter graphs grouped by the six simulation domains, each with a description and a preview.
- Instantiation of a template, pre-populating a new project's canvas with its nodes, edges and default parameters.
- Creation of a fully independent copy owned by the requesting user, so edits can never propagate back to the source template.
- Free modification of all template parameters after instantiation.
- Maintenance of the template library as a reusable and extensible resource.

Implementation

Templates are declared in the simulation type registry alongside the node palette and layout strategy for each domain, which keeps a domain's definition in one place and means that adding a domain does not require changes scattered across components. Instantiation copies rows rather than referencing the template, which satisfies the independence requirement by construction rather than by policy.

3.11 Module 11 — Executive Report and Share

Purpose

Work that cannot leave the platform has limited value. This module moves models and results outward, whether for archival, for further analysis elsewhere, or for presentation to someone who will never log in.

Functional responsibilities

- One-click generation of an IEEE-style PDF report containing an AI-written executive summary, a canvas snapshot, KPI tables and charts.
- Export of a project's graph as downloadable JSON, preserving nodes, edges and configuration.
- Export of a completed run's results as CSV for analysis in a spreadsheet or statistical package.
- Export of the generated Python script as a Jupyter notebook for independent reproduction.
- Generation of a read-only public share link that never confers edit access regardless of the viewer's authentication state.
- Revocation of a share link, after which the circulated URL ceases to resolve and displays an explanatory notice.

Implementation

PDF generation runs client-side using jsPDF with the AutoTable extension for KPI tables and html-to-image to capture the canvas, so no document-rendering service is required and the architecture's zero-server-compute property is preserved. The executive summary is composed by the same Gemini integration used elsewhere, working from the computed metrics rather than the raw trace, so the prose matches the tables printed beside it. Share links are opaque tokens resolved through a dedicated read-only route; a token grants visibility of one project and nothing further, and the read-only guarantee is enforced by Row-Level Security at the database rather than by hiding controls in the interface.

3.12 Module 12 — Real-Time Collaboration

Purpose

This module provides the multi-user behaviour that teams now expect of any modern design tool: several people building and discussing one model at the same time, in the manner of Canva or Figma.

Functional responsibilities

- Invitation of other users to a project as Editor or Viewer by email address.
- Concurrent editing of a single project by multiple authenticated users.
- Broadcast of graph changes to all active collaborators within 500 milliseconds.
- Live cursors and presence indicators showing who is viewing the project and where their attention is directed.
- Conflict resolution for simultaneous edits to the same node, on a last-write-wins basis using a monotonic client timestamp.
- Automatic persistence of collaborative changes, and prevention of any modification by a Viewer.
- Consistency of the rendered graph across every connected user.

Implementation

Collaboration is built on Supabase Realtime channels over WebSockets. The decision taken during database design to store the graph as normalised node and edge rows, rather than as a single JSON document, is what makes this workable: an edit can be broadcast as a change to one entity, so two users moving different nodes do not overwrite one another — which is precisely what whole-document replacement would cause. Concurrent edits to the same node are resolved last-write-wins on a client-supplied monotonic timestamp. Presence and cursor positions are transmitted on a separate ephemeral channel and are never persisted, since they have no meaning after a session ends. The Viewer restriction is enforced by a Row-Level Security policy keyed on the collaborator's recorded permission, so it holds even against a client altered to display the editing controls.

3.13 Hardware Modules and Considerations

JustCmul8 introduces no custom hardware, but the decision to execute simulations on the client rather than the server gives the user's own machine a role it would not otherwise have, and that role is worth stating explicitly.

Table 3.6: Hardware Roles in the Deployed System

Element	Role in the system	Consequence of the design
Client CPU	Executes the WebAssembly runtime and the entire simulation	Determines run duration; large models are slower on weaker machines
Client memory	Holds the Pyodide runtime, the graph, the event trace and the rendered canvas	Sets the practical ceiling on model size and replication count
Client GPU	Renders the digital-twin playback through WebGL	Enables smooth animation of many simultaneous entities
Network link	Carries persistence, collaboration and AI requests only	Simulation continues even on an intermittent connection
Application server	Serves the static bundle and a small number of API routes	Carries no simulation load, so hosting cost is independent of usage intensity
Database server	Managed PostgreSQL storing projects, runs, results and logs	Sized by data volume rather than by computational demand

The last two rows express the economic argument of Section 1.5.2 in hardware terms. Because neither server participates in the computation, the infrastructure required to support a thousand users running simulations is essentially that required to serve a thousand users a static page, which is what makes the platform sustainable without a licence fee.

CHAPTER 4

SYSTEM ANALYSIS AND DESIGN

This chapter presents the analysis and design of JustCmul8. It begins with the development model adopted and the reasons for choosing it, then works from the outermost view of the system inwards: the context diagram establishes the boundary, the data flow diagrams decompose the processes within it, the entity relationship and class diagrams describe the data and the objects, and the behavioural diagrams describe how those objects interact at runtime. The chapter closes with the physical concerns — the database schema, the in-memory data structures and the deployment topology.

4.1 Detailed Life Cycle of the Project

We adopted an iterative and incremental model combining the discipline of the waterfall model with the responsiveness of agile practice. The choice was not made in the abstract; it followed from two properties of this particular project.

The first is that the requirements were reasonably stable from the outset. We knew what a discrete event simulator must do, because the problem domain is well established and the comparable tools surveyed in Section 1.3 defined the expected feature set. A pure agile approach, which derives its value from accommodating requirements that emerge during development, would have offered little advantage here, and the formal documentation an academic project must produce benefits from the stage discipline of the waterfall model.

The second is that the technical risk was concentrated in a single component. Whether SimPy could be executed inside a browser through Pyodide was an open question when planning began, and almost everything else depended on the answer. A strict waterfall would have postponed that discovery until the implementation phase, by which time the design would have been fixed around an assumption that might not hold. We therefore inserted an iterative prototyping loop early, treating the engine as a spike to be proved before the dependent modules were designed in detail.

In practice the analysis and design phases proceeded in waterfall fashion, producing the requirement specification and the schema before implementation began, while the

implementation phase proceeded iteratively, one module at a time, each module passing through its own cycle of design, coding, testing and integration before the next was begun. Testing was continuous rather than terminal, and documentation was written alongside each module.

Table 4.1: Phases of the Adopted Life-Cycle Model

Phase	Activities performed	Outcome
Requirement analysis	Elicitation of user needs, study of comparable tools, identification of functional and non-functional requirements	Approved requirement specification (Section 1.4)
Feasibility study	Assessment of technical, economic, operational, schedule, legal and ethical viability	Confirmation that the browser-hosted approach was achievable
System design	Architecture definition, database schema design, interface layout, UML modelling	Design documents and diagrams presented in this chapter
Prototyping (iterative)	Spike implementation of the Pyodide and SimPy execution path, verified against analytical results	Retirement of the dominant technical risk at milestone M4
Implementation (incremental)	Module-by-module construction across three parallel streams, each with its own design-code-test cycle	Twelve delivered modules (Chapter 3)
Testing	Unit, structural, integration, cross-browser, determinism and acceptance testing, running alongside development	Verified system and the test matrix in Chapter 5
Deployment	Final review, release to the hosting platform, configuration of environment variables and database policies	Live application accessible over the public internet
Documentation	Continuous recording of analysis, design, implementation and test outcomes as each module was completed	This report and the accompanying technical documentation

One consequence of this model is worth recording honestly. Because the engine was built before several of the modules that consume it, its interface was revised twice — once when the Monte Carlo facility required per-replication seeding, and again when the digital twin required a per-entity journey trace that the original result format did not carry. Both revisions were inexpensive because the engine remained isolated behind a worker boundary, which is precisely the benefit the layered architecture was intended to provide.

4.2 Context Diagram (DFD Level 0)

The context diagram fixes the boundary of the system. It represents JustCmul8 as a single process and shows the external entities that exchange data with it, without decomposing anything inside. Its purpose is to settle what belongs to the system and what does not before any internal structure is discussed.

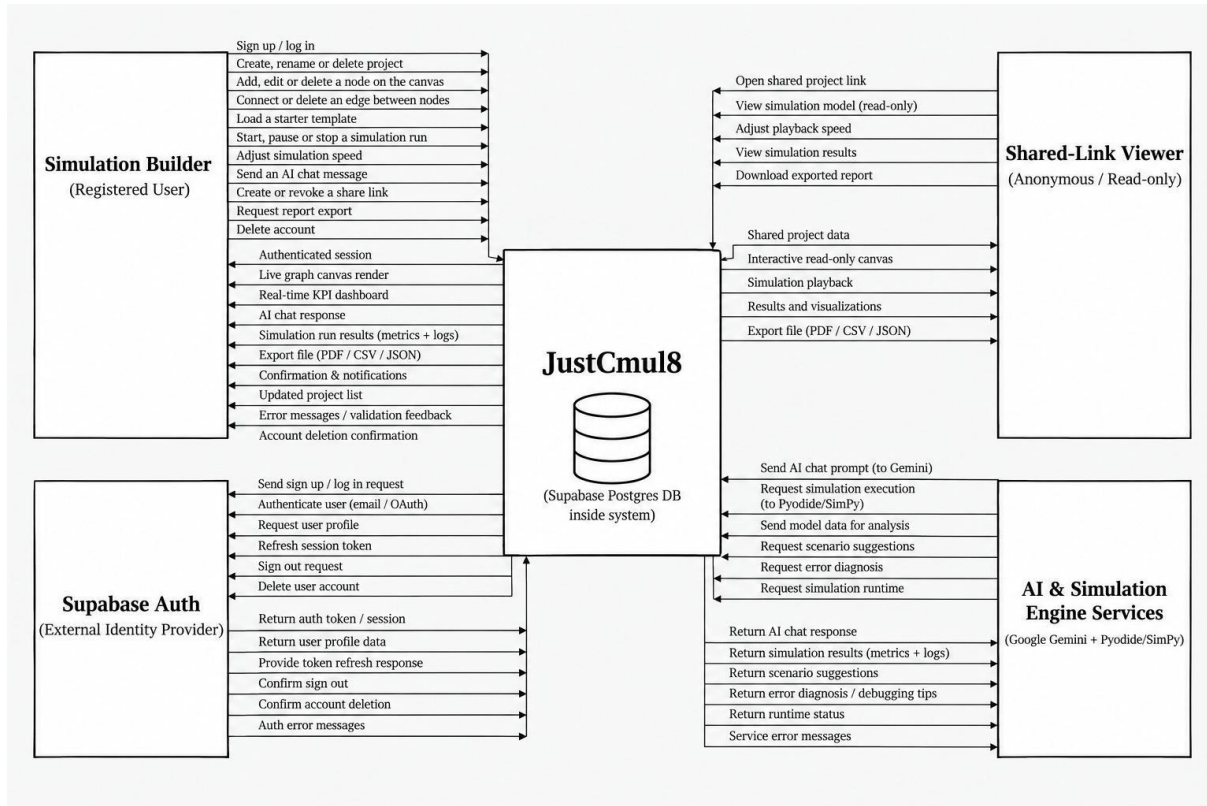


Figure 4.1: Context Diagram (DFD Level 0) for JustCmul8

Four external entities interact with the system. The Simulation Builder is the registered user who creates and runs models; the inbound flows from this entity cover signing up and logging in, creating, renaming and deleting projects, adding, editing and deleting nodes and edges, loading starter templates, starting, pausing and stopping runs, adjusting simulation speed, sending AI chat messages, creating and revoking share links, requesting report exports and deleting the account. The return flows carry the authenticated session, the live canvas rendering, the real-time KPI dashboard, AI responses, run results with metrics and logs, export files, confirmations, the updated project list, validation feedback and account-deletion confirmation.

The Shared-Link Viewer is an anonymous visitor holding a public share URL. This entity opens a shared project, views the model read-only, adjusts playback speed, views results and downloads an exported report, receiving shared project data, an interactive read-only canvas, simulation playback, results and visualisations, and export files. The asymmetry between the two user entities is deliberate and expresses requirement FR-24: no flow from this entity modifies stored state.

Supabase Auth is the external identity provider. The system sends it sign-up and login requests, user-profile queries, session-token refreshes, sign-out requests and account-deletion instructions, and receives authentication tokens and sessions, profile data, refresh responses, confirmations and authentication errors. Placing authentication outside the boundary records that JustCmul8 delegates credential handling rather than implementing it.

The AI and Simulation Engine Services entity covers the Google Gemini API and the Pyodide and SimPy runtime delivered over a CDN. The system sends chat prompts, simulation execution requests, model data for analysis, scenario-suggestion and error-diagnosis requests and runtime queries, receiving chat responses, simulation results with metrics and logs, scenario suggestions, diagnostics, runtime status and service errors. The Supabase PostgreSQL database is drawn inside the system boundary, since it is provisioned and governed by the application rather than consumed as an independent third party.

4.3 Data Flow, Entity Relationship, Class and State Diagrams

4.3.1 Data Flow Diagram — Level 1

The level-1 diagram decomposes the single process of the context diagram into the seven functional processes that constitute the system, and introduces the data stores they read from and write to.

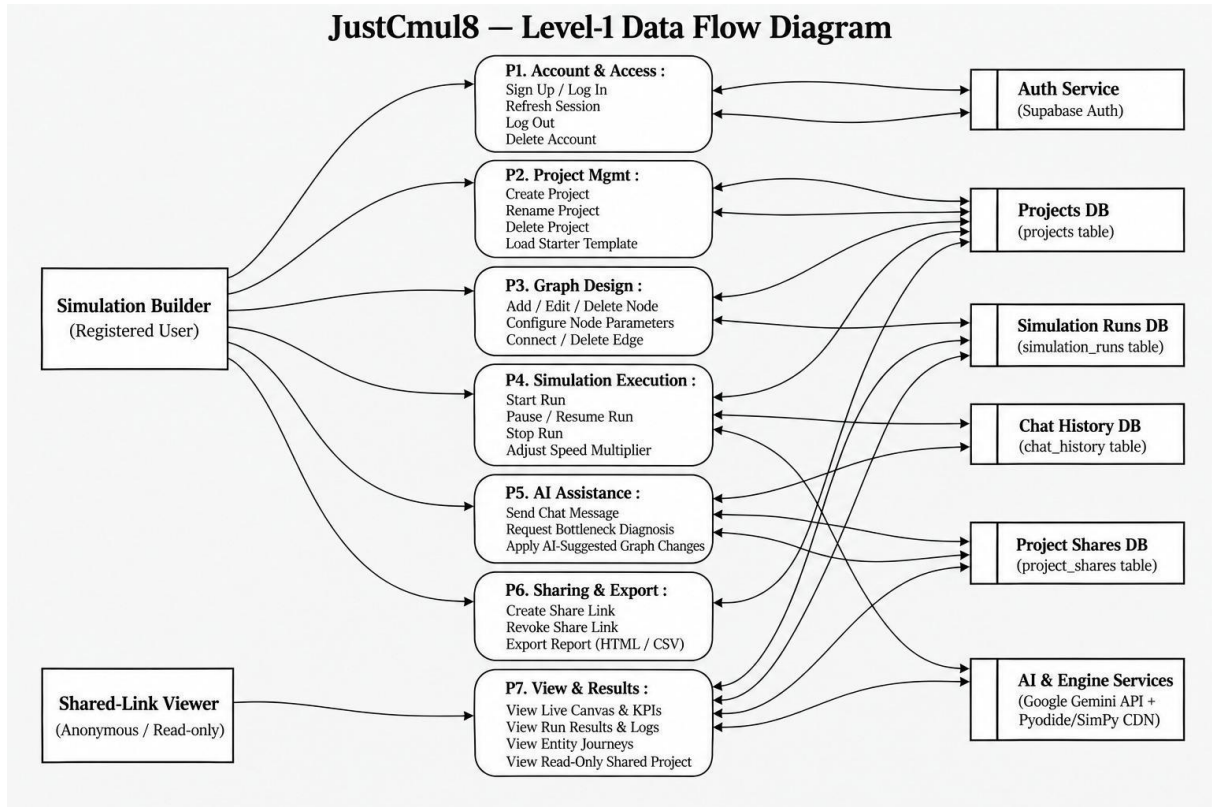


Figure 4.2: Level-1 Data Flow Diagram for JustCmul8

Process P1, Account and Access, handles sign-up, log-in, session refresh, log-out and account deletion, exchanging data exclusively with the external authentication service. Process P2, Project Management, covers project creation, renaming and deletion and the loading of starter templates, reading from and writing to the Projects store. Process P3, Graph Design, adds, edits and deletes nodes, configures node parameters and connects or removes edges, persisting each change to the same store.

Process P4, Simulation Execution, starts, pauses, resumes and stops runs and adjusts the speed multiplier; it reads the graph from the Projects store, drives the external engine services and writes run records to the Simulation Runs store. Process P5, AI Assistance, sends chat messages,

requests bottleneck diagnosis and applies AI-suggested graph changes, exchanging data with the Gemini service and recording exchanges in the Chat History store.

Process P6, Sharing and Export, creates and revokes share links and produces HTML and CSV exports, writing to the Project Shares store. Process P7, View and Results, serves the live canvas and KPIs, run results and logs, entity journeys and the read-only shared project view; it is the only process reachable by the anonymous Shared-Link Viewer, which again enforces the read-only guarantee structurally rather than by convention.

Five data stores appear: the Projects table holding project metadata and graph structure, the Simulation Runs table holding run records with results and logs, the Chat History table holding AI exchanges, the Project Shares table holding share tokens, and the external authentication service. Notice that no process writes to a store owned by another process without passing through it, which is what keeps the Row-Level Security policies enforceable at a single point per table.

4.3.2 Data Flow Diagram — Level 2

Process P4 is the most intricate in the system and is decomposed further at level 2. This is the path that converts a drawing into numbers, and it is where the architectural commitment described in Chapter 3 becomes visible as data flow.

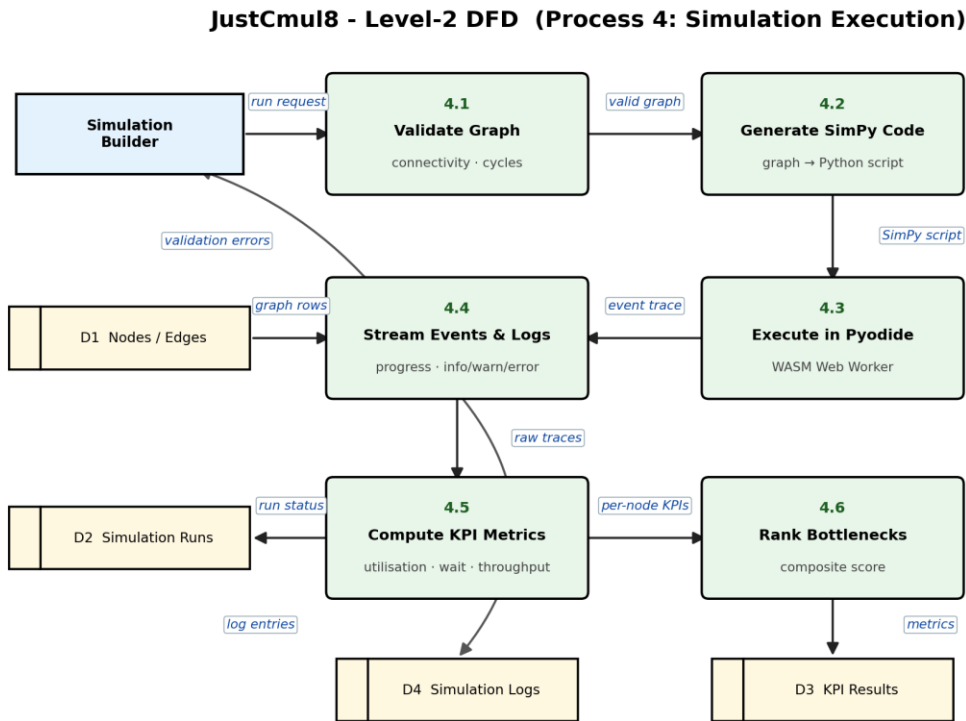


Figure 4.3: Level-2 Data Flow Diagram — Decomposition of Process P4 (Simulation Execution)

A run request from the Simulation Builder enters process 4.1, Validate Graph, which reads the node and edge rows from store D1 and checks connectivity and the absence of deadlocking cycles. If validation fails, errors flow back to the user and the run does not proceed; this satisfies requirement FR-09 and prevents an invalid model from reaching the engine at all.

A valid graph passes to process 4.2, Generate SimPy Code, which walks the graph in topological order and emits a Python script. That script flows to process 4.3, Execute in Pyodide, which runs it inside the WebAssembly runtime hosted in a Web Worker. The event trace produced there flows to process 4.4, Stream Events and Logs, which forwards progress and log entries to the interface during the run and writes log records to store D4.

Raw traces then flow to process 4.5, Compute KPI Metrics, which derives utilisation, waiting times and throughput and updates the run status in store D2. The per-node metrics pass to

process 4.6, Rank Bottlenecks, which computes the composite score and writes the results to store D3. The ordering matters: bottleneck ranking depends on computed metrics rather than on the raw trace, which is why 4.6 follows 4.5 rather than reading from 4.4 directly, and it is the same dependency that grounds the optimiser of Module 6.

4.3.3 Entity Relationship Diagram

The entity relationship diagram describes how persistent data is organised and how the entities relate to one another. It is drawn in Chen notation, with rectangles for entities, ellipses for attributes and diamonds for relationships, annotated with cardinality.

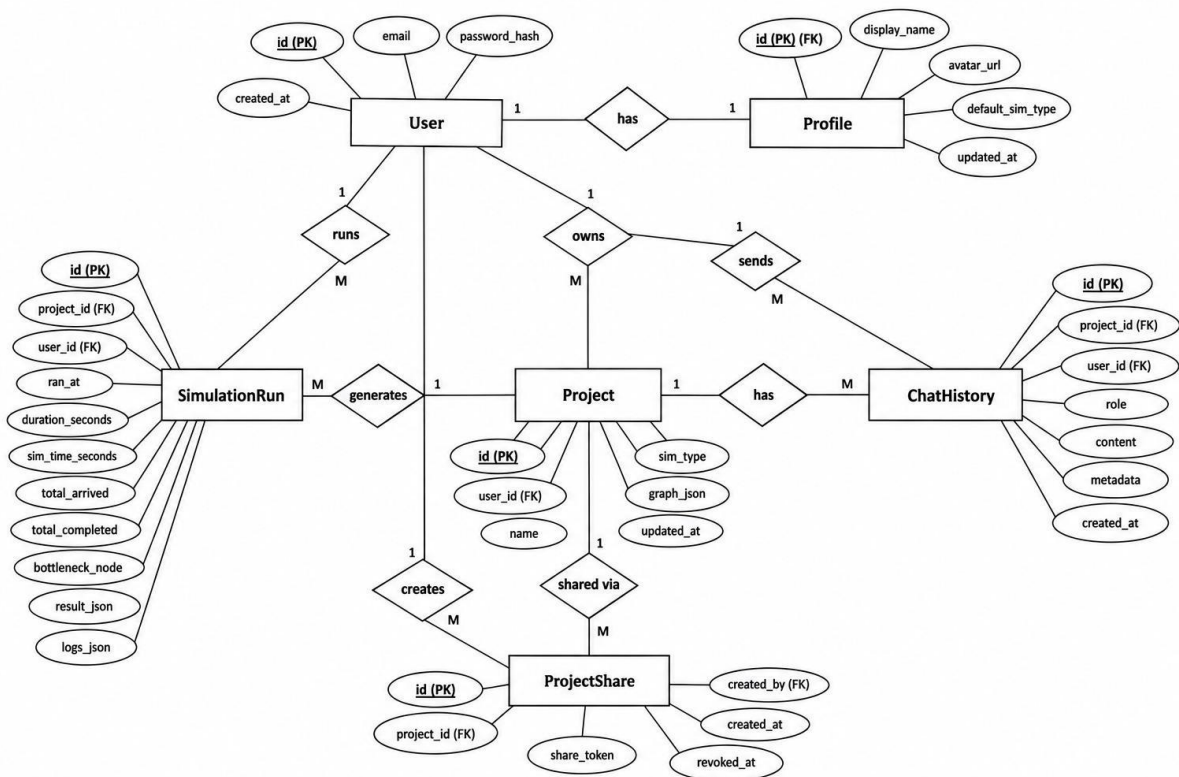


Figure 4.4: Entity Relationship Diagram for the JustCmul8 Database

The User entity is central, carrying the primary key, email, password hash and creation timestamp. It stands in a one-to-one relationship with Profile, which holds the display name, avatar URL, default simulation type and update timestamp; separating the profile from the credential record keeps presentation data apart from authentication data, so the profile can be read in contexts where the credential row should not be touched.

A User owns many Projects, expressed by the one-to-many “owns” relationship. Each Project carries its identifier, owner foreign key, name, simulation type, graph data and update timestamp. A User also runs many SimulationRuns and sends many ChatHistory messages, and each Project generates many SimulationRuns and has many ChatHistory entries, which is why both of those entities carry foreign keys to User and to Project.

The SimulationRun entity is the most heavily attributed, holding the run timestamp, wall-clock duration, simulated duration, totals arrived and completed, the identified bottleneck node, and the result and log payloads. Recording the bottleneck as an attribute of the run rather than recomputing it on demand means a historical run retains the diagnosis that was made at the time, even if the scoring logic is later refined.

ProjectShare stands in a many-to-one relationship with Project and with the creating User, holding the share token and the creation and revocation timestamps. Retaining a revoked share as a row with a revocation timestamp, rather than deleting it, is what allows the system to distinguish a link that was withdrawn from one that never existed, and therefore to show the viewer an explanatory notice instead of a bare error.

4.3.4 Class Diagram

The class diagram presents the object model implemented in TypeScript, showing each class with its typed attributes, its operations and its associations with the others.

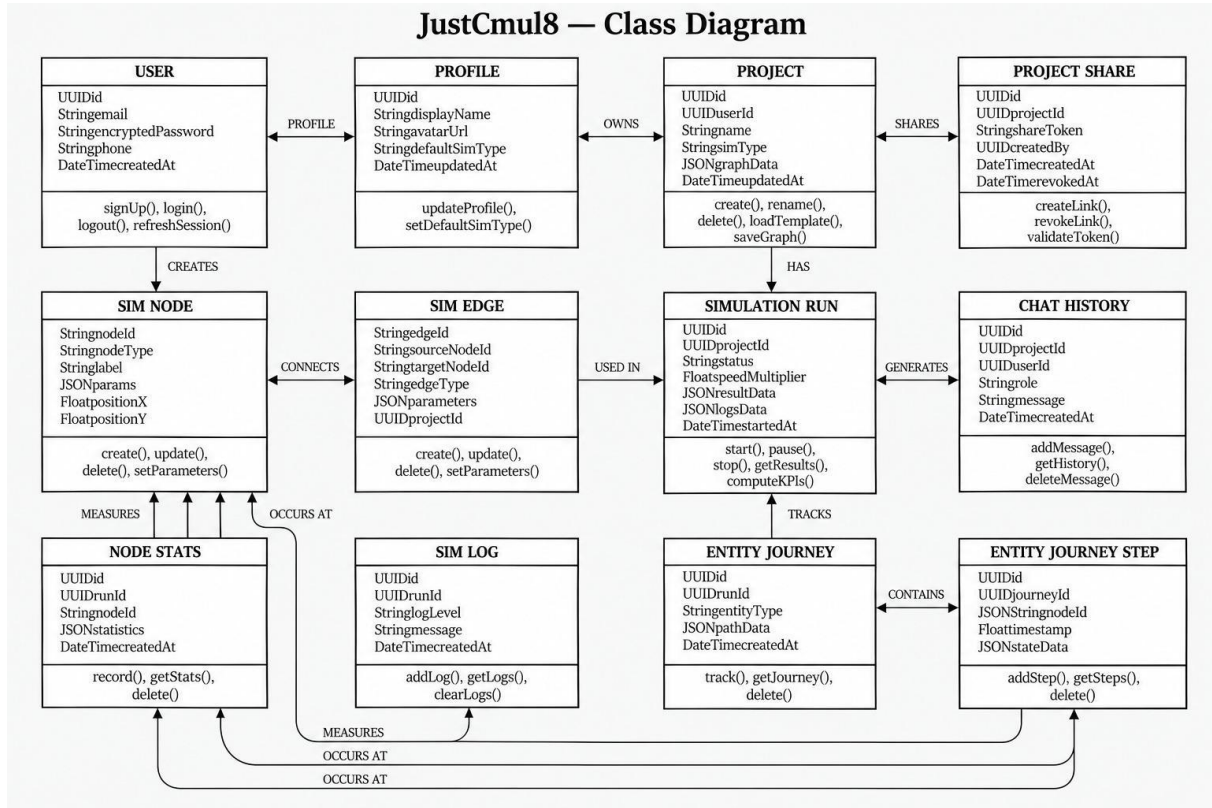


Figure 4.5: Class Diagram of the JustCmul8 Object Model

The upper row carries the account and project classes. User exposes signUp, login, logout and refreshSession; Profile exposes updateProfile and setDefaultSimType; Project exposes create, rename, delete, loadTemplate and saveGraph; and ProjectShare exposes createLink, revokeLink and validateToken. The associations read as User has a Profile, User owns Projects, and Project is shared through ProjectShare.

The middle row carries the graph and execution classes. SimNode holds the node identifier, node type, label, a JSON parameter object and canvas coordinates, exposing create, update, delete and setParameters. SimEdge holds source and target node identifiers, an edge type and parameters, with the same operation set. Storing parameters as JSON rather than as fixed columns is what allows fifteen node types with dissimilar parameters to share one class without a sparse schema.

SimulationRun holds status, speed multiplier, result and log payloads and the start timestamp, exposing start, pause, stop, getResults and computeKPIs. ChatHistory records the role, the message and the timestamp for each AI exchange, exposing addMessage, getHistory and deleteMessage.

The lower row carries the measurement classes. NodeStats records per-node statistics for a run and is associated with SimNode through the “measures” relationship. SimLog records log level and message per run. EntityJourney tracks the path of an individual entity through the model, and EntityJourneyStep records each timestamped node visit within that journey. The journey and step classes exist specifically to support the digital-twin playback of Module 9: because each step carries a timestamp and a node reference, the animation can be reconstructed and scrubbed without re-executing the simulation.

4.3.5 State Transition Diagram

The state transition diagram describes the lifecycle of a node during execution. Every node in a running model occupies exactly one of these states at any instant, and the transitions between them are what the event scheduler actually effects.

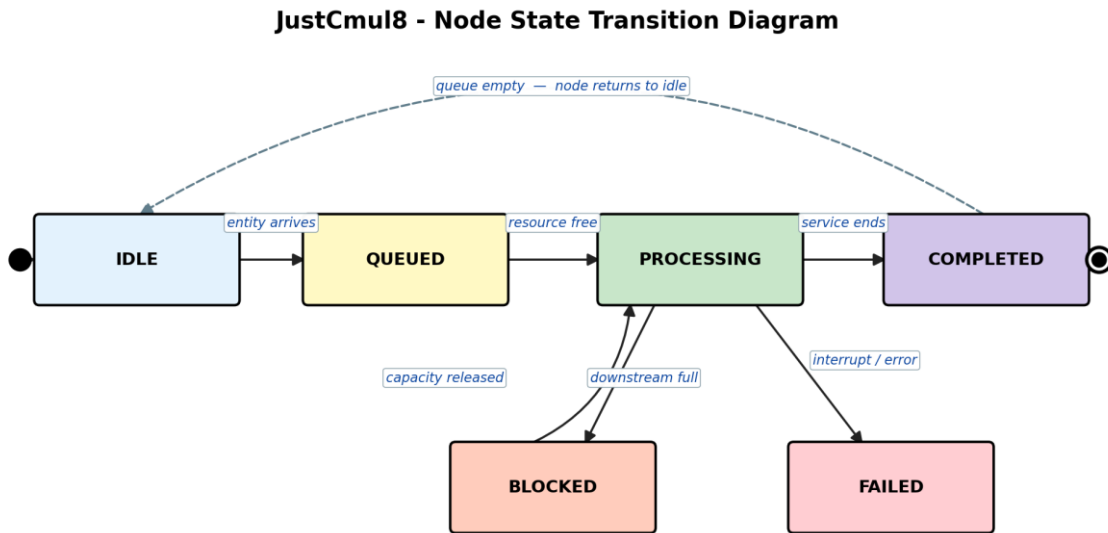


Figure 4.6: State Transition Diagram for a Node During Simulation Execution

A node begins in IDLE, having no entity to process. The arrival of an entity moves it to QUEUED, where the entity waits for capacity. When the associated resource becomes free the node enters PROCESSING, holding the entity for a service time sampled from the configured

distribution. On completion of that service the node moves to COMPLETED and the entity is routed onward; if no further entities are waiting, the node returns to IDLE, which is the cycle shown by the dashed return transition.

Two exceptional transitions leave PROCESSING. If the downstream node has no capacity to accept the entity, the node moves to BLOCKED and remains there until capacity is released, at which point it returns to PROCESSING. Blocking is the mechanism by which congestion propagates backwards through a model, and representing it as a distinct state is what allows the analytics layer to distinguish a node that is slow from one that is merely obstructed by the node after it. The second exceptional transition leads to FAILED, entered when a process is interrupted — by an interrupter node modelling a breakdown, by preemption at a priority resource, or by a runtime error — and it is terminal for the entity concerned.

This state model maps directly onto SimPy semantics: QUEUED corresponds to an outstanding resource request, PROCESSING to a timeout in progress while the resource is held, BLOCKED to a pending put on a downstream store or container, and FAILED to a raised Interrupt. The correspondence is deliberate, since the generated script must express these transitions in SimPy constructs rather than in scheduling logic of our own.

4.4 Use Case Diagram

The use case diagram identifies the actors and the services the system offers to each. It expresses functionality from the outside, in terms of what an actor can accomplish, rather than from the inside in terms of how it is achieved.

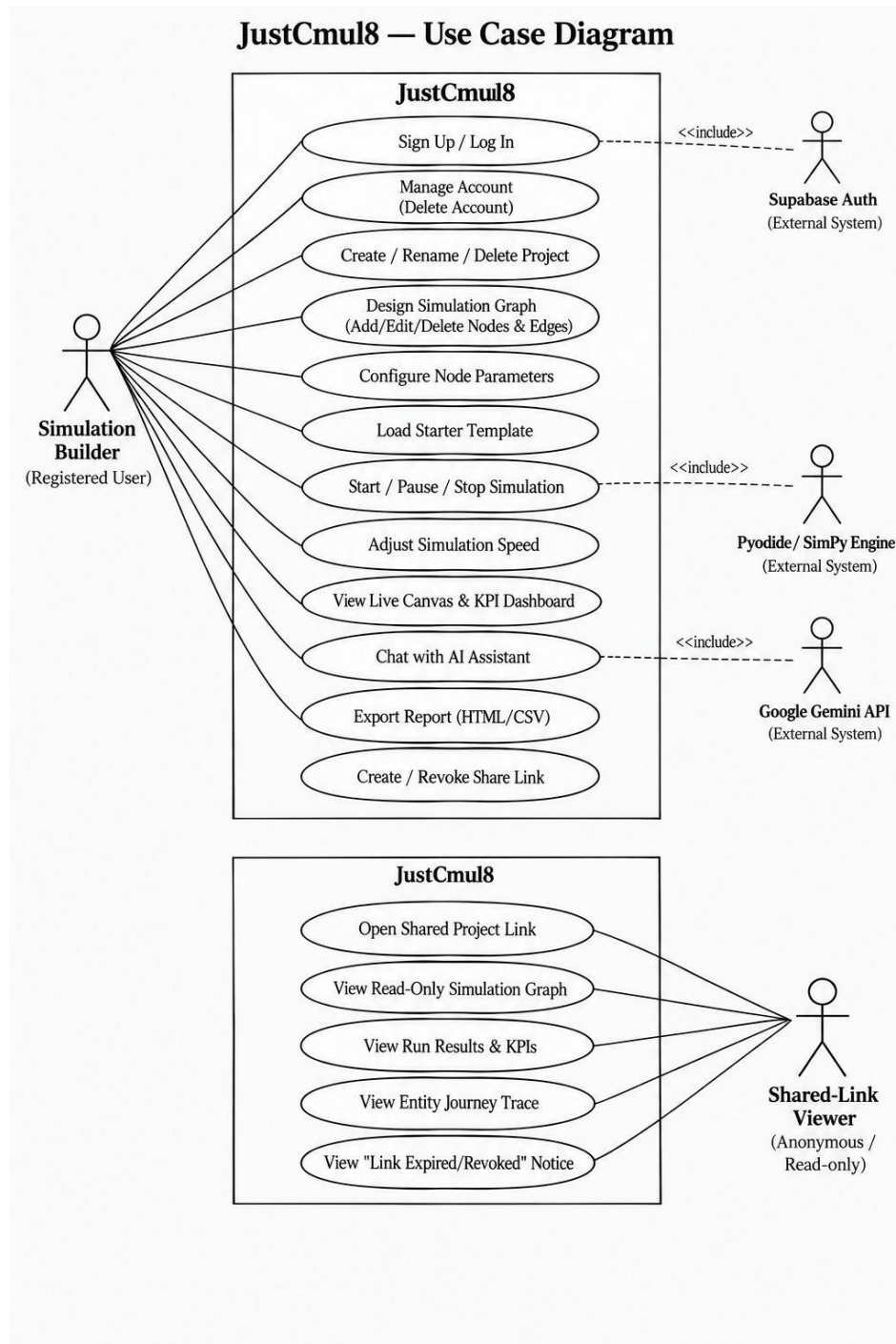


Figure 4.7: Use Case Diagram for JustCmul8

Two human actors appear. The Simulation Builder, a registered user, may sign up or log in, manage the account including deletion, create, rename and delete projects, design the simulation graph by adding, editing and deleting nodes and edges, configure node parameters, load a starter template, start, pause and stop a simulation, adjust the simulation speed, view the live canvas and KPI dashboard, chat with the AI assistant, export a report as HTML or CSV, and create or revoke a share link.

The Shared-Link Viewer is an anonymous actor whose use cases are drawn in a separate boundary to make the separation explicit: opening a shared project link, viewing the read-only simulation graph, viewing run results and KPIs, viewing the entity journey trace, and viewing the notice displayed when a link has expired or been revoked. That final use case is easy to overlook in design and produces a poor experience when omitted, which is why it was modelled explicitly rather than treated as an error path.

Three external systems participate through include relationships. Supabase Auth is included by sign-up and log-in; the Pyodide and SimPy engine is included by starting, pausing and stopping a simulation; and the Google Gemini API is included by chatting with the AI assistant. The include stereotype is correct here rather than extend, because in each case the external interaction is mandatory for the use case to complete, not an optional enhancement of it.

4.4.1 Use Case Specifications

The tables below specify the four use cases that carry the most behaviour, in the standard form of actor, precondition, main flow, alternative flow and postcondition.

Table 4.2: Use Case Specification — Run Simulation

Field	Description
Use case ID	UC-01
Use case name	Run Simulation
Primary actor	Simulation Builder (Owner or Editor)
Secondary actor	Pyodide / SimPy engine
Precondition	The user is authenticated, a project is open, and the graph contains at least one source and one sink
Main flow	1. The user sets duration, seed and replication count. 2. The user starts the run. 3. The system validates the graph structurally. 4. The system generates the SimPy script. 5. The worker executes the script through Pyodide. 6. Progress and log events stream to the interface. 7. The system computes KPIs and ranks bottlenecks. 8. The dashboard renders the results.

Field	Description
Alternative flow	3a. Validation fails: the system names the offending node, explains the fault and aborts before execution. 5a. The runtime raises an error: the run is marked Failed and the error is shown with the relevant log entries. 5b. The user stops the run: execution halts and partial results are discarded.
Postcondition	A SimulationRun record exists with status Completed or Failed, and on success KPI results and logs are persisted
Requirements	FR-12, FR-13, FR-14, FR-15, FR-19, FR-20

Table 4.3: Use Case Specification — Generate Model via AI

Field	Description
Use case ID	UC-02
Use case name	Generate Simulation Model from Natural Language
Primary actor	Simulation Builder (Owner or Editor)
Secondary actor	Google Gemini API
Precondition	The user is authenticated and a project is open
Main flow	1. The user describes the scenario in plain English. 2. The system composes a prompt from the description, the node-type registry and the current graph. 3. The request is proxied through the server to Gemini. 4. The response is parsed into candidate nodes and edges. 5. The candidate graph is validated against the schema. 6. The graph is laid out and shown as a preview. 7. The user accepts, and the graph is written to the project.
Alternative flow	5a. Validation fails on an unknown node type or an orphaned edge: the candidate is rejected and the user is invited to rephrase. 7a. The user discards the preview: the live project is unchanged. 3a. The service is unavailable or rate-limited: an explanatory message is shown and manual construction remains available.
Postcondition	Either the project graph contains the accepted nodes and edges, or the project is entirely unmodified
Requirements	FR-16, FR-17

Table 4.4: Use Case Specification — Share Project Read-Only

Field	Description
Use case ID	UC-03
Use case name	Share Project through a Public Link
Primary actor	Project Owner
Secondary actor	Shared-Link Viewer (anonymous)
Precondition	The user owns the project
Main flow	1. The owner requests a share link. 2. The system generates an opaque token and stores it against the project. 3. The link is presented for copying. 4. A viewer opens the link without authenticating. 5. The system validates the token and renders the project read-only with its results.
Alternative flow	5a. The token has been revoked: the viewer is shown an explanatory notice rather than an error. 5b. The token is unrecognised: access is refused. 2a. The owner later revokes the link: the revocation timestamp is set and subsequent requests fall to 5a.
Postcondition	A ProjectShare record exists, and the viewer has read access to that project alone with no ability to modify anything

Field	Description
Requirements	FR-24

Table 4.5: Use Case Specification — Collaborate in Real Time

Field	Description
Use case ID	UC-04
Use case name	Collaborative Editing of a Project
Primary actor	Editor (collaborator)
Secondary actor	Supabase Realtime, other collaborators
Precondition	Two or more authenticated users with access have the same project open
Main flow	1. Each client subscribes to the project's realtime channel. 2. Presence and cursor positions are broadcast continuously. 3. An editor modifies a node or edge. 4. The change is persisted and broadcast as a single-entity update. 5. Every other client applies the update and re-renders.
Alternative flow	4a. Two editors modify the same node concurrently: the later client timestamp wins and the earlier change is superseded. 1a. A Viewer subscribes: presence is shown but every write is refused by Row-Level Security. 5a. A client disconnects: its presence indicator is removed and its pending changes are discarded.
Postcondition	All connected clients display an identical graph, and the persisted state matches what is displayed
Requirements	FR-26, FR-27

4.5 Activity, Component and Collaboration Diagrams

4.5.1 Activity Diagram — End-to-End User Workflow

Chapter 2 presented an activity diagram of the development lifecycle. The diagram below models a different flow entirely: the path a user takes through the running system, from authentication to export.

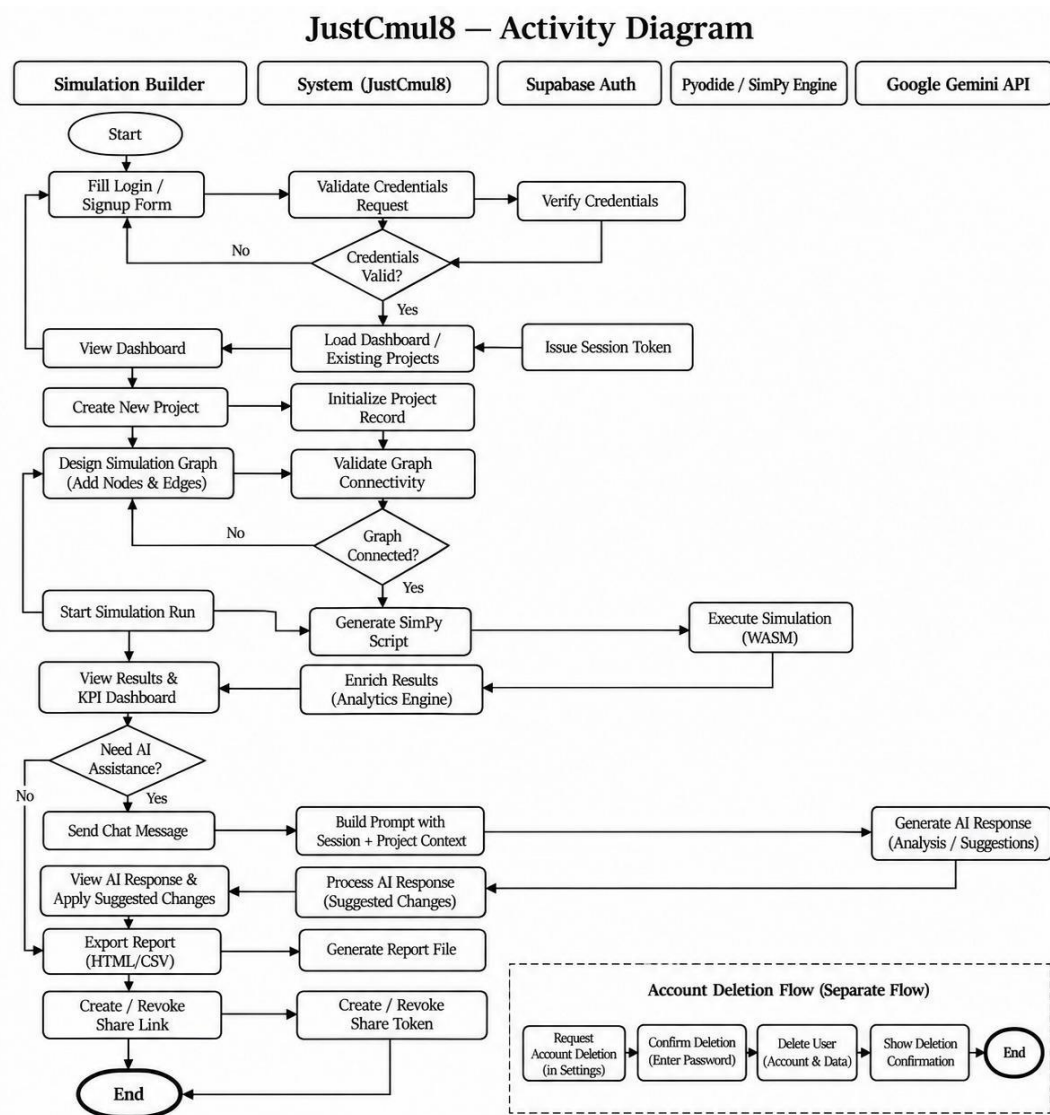


Figure 4.8: Activity Diagram of the End-to-End User Workflow

The diagram is divided into swimlanes for the Simulation Builder, the system itself, Supabase Auth, the Pyodide and SimPy engine and the Gemini API, so that responsibility for each activity is unambiguous.

The flow opens with the user completing the login or sign-up form. The system issues a validation request to Supabase Auth, which verifies the credentials. A decision node tests whether they are valid; if not, control returns to the form. On success a session token is issued and the dashboard loads with the user's existing projects.

The user creates a new project, and the system initialises the project record. The user then designs the graph by adding nodes and edges, and the system validates connectivity. A second decision node tests whether the graph is connected; if not, control returns to the design activity, which is the loop that prevents an unrunnable model from reaching the engine.

Once the graph is valid the user starts the run. The system generates the SimPy script and hands it to the engine lane, where it executes as WebAssembly. Results return to the system lane, where the analytics engine enriches them, and the user views the results and KPI dashboard.

A third decision node asks whether AI assistance is wanted. If it is, the user sends a chat message, the system builds a prompt carrying the session and project context, and Gemini generates a response with analysis or suggestions; the system processes the suggested changes and the user may apply them, which returns control to the design activity. If not, the flow proceeds directly to exporting a report, where the system generates the file, and then to creating or revoking a share link before reaching the end node.

Account deletion is drawn as a separate flow, initiated from settings and proceeding through password confirmation, cascading removal of the account and its data, and a confirmation notice. Separating it reflects that it is not part of the modelling workflow and must not be reachable accidentally from within it.

4.5.2 Component Diagram

The component diagram shows the implementation units of the system and the dependencies between them, grouped by the execution environment in which each unit runs.

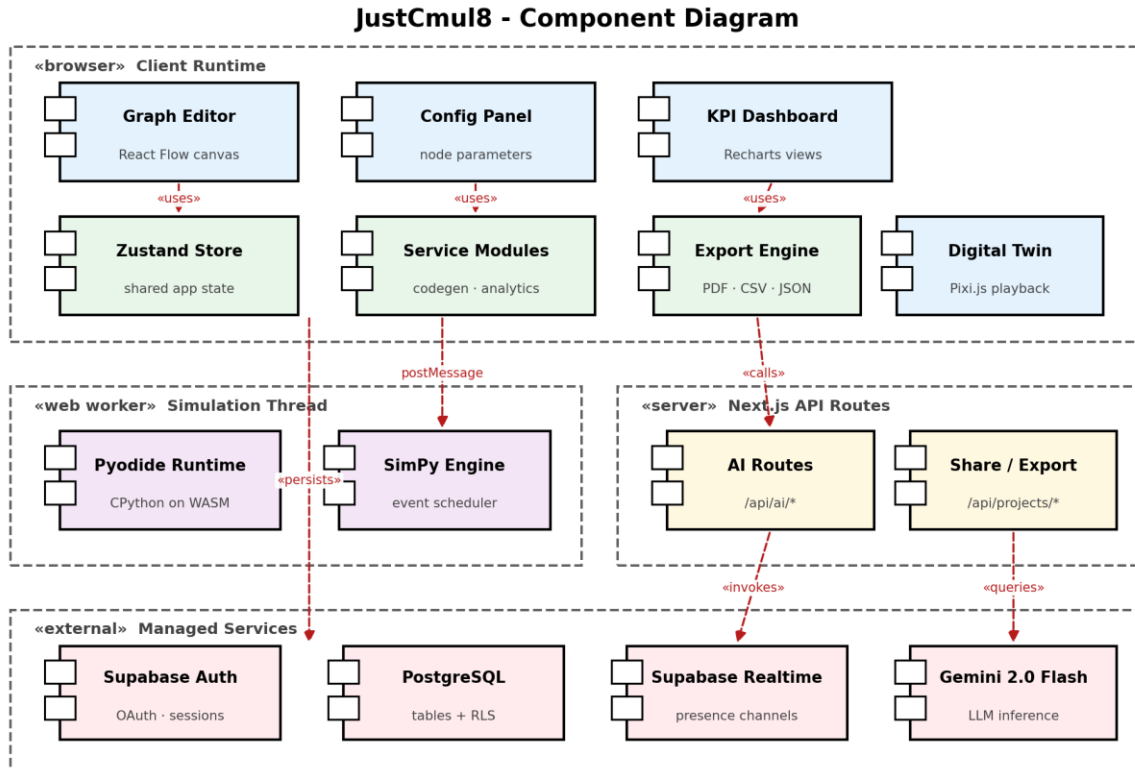


Figure 4.9: Component Diagram of the JustCmul8 Implementation

The browser client runtime contains the interface components — the graph editor built on React Flow, the configuration panel, the KPI dashboard built on Recharts and the Pixi.js digital twin — together with the Zustand store holding shared application state, the service modules implementing code generation and analytics, and the export engine. The interface components depend on the store rather than on one another, which is what allows a panel to be replaced or repositioned without touching the others.

The simulation thread is a separate component group because it executes in a Web Worker rather than on the main thread. It contains the Pyodide runtime hosting CPython compiled to WebAssembly, and the SimPy engine running within it. Communication with the client runtime is by `postMessage` alone; no memory is shared, which is what guarantees that a long simulation cannot block rendering.

The server group contains the Next.js API routes: the AI routes under `/api/ai` and the share and export routes under `/api/projects`. These exist for the narrow set of operations that must not run on the client — chiefly holding the Gemini API key and minting share tokens.

The external group contains the managed services: Supabase Auth for identity, PostgreSQL with Row-Level Security for persistence, Supabase Realtime for presence channels and Google Gemini for inference. Reading the diagram vertically shows that the client depends on the database directly for ordinary reads and writes, and only routes through the server for privileged operations, which is why the server carries so little of the system's behaviour.

4.5.3 Collaboration Diagram

The collaboration diagram shows the objects that participate in a single simulation run and the sequence of messages passed between them, numbered in order of occurrence.

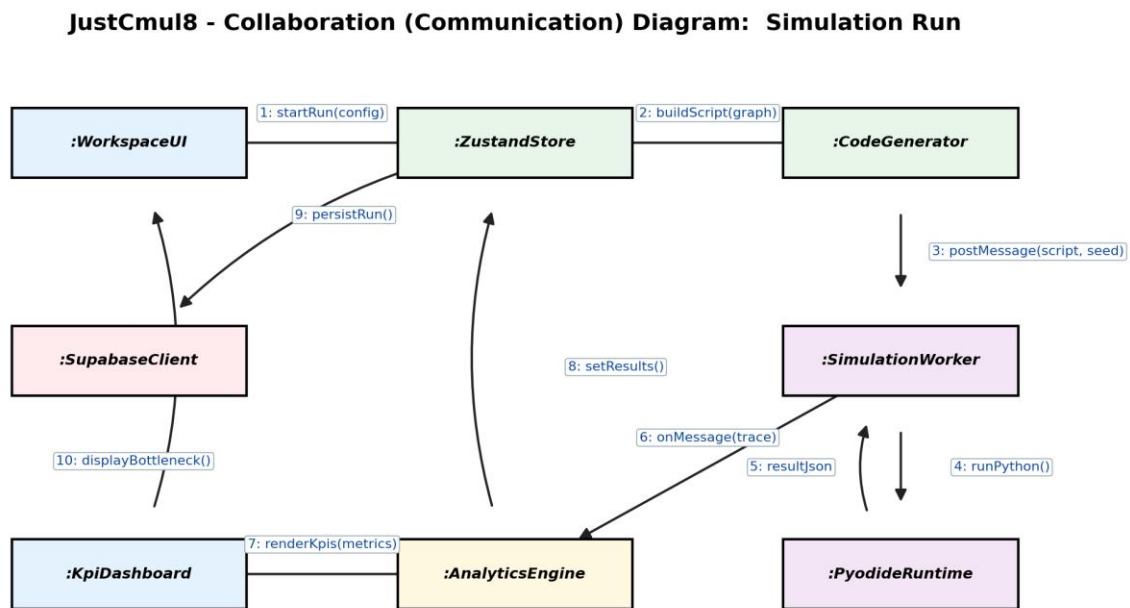


Figure 4.10: Collaboration Diagram for the Simulation Run Interaction

The interaction begins when the workspace interface calls `startRun` on the Zustand store with the run configuration (message 1). The store calls `buildScript` on the code generator, passing the current graph (2), and the generated script together with the seed is posted to the simulation worker (3). Message 3 crosses the thread boundary, which is why it is a `postMessage` rather than a direct call. The worker then runs the script using `runPython` (4) and returns the `resultjson` (5). The worker sends an `onMessage` (6) to the analytics engine, which then updates the Zustand store with `setResults` (7). The Zustand store then persists the run (8) and the workspace interface renders the KPIs (9). Finally, the Supabase client displays a bottleneck (10).

Within the worker, `runPython` is invoked on the Pyodide runtime (4), which returns the result payload (5). The worker forwards the trace back to the main thread, where the analytics engine receives it (6). The analytics engine renders KPIs to the dashboard (7) and writes the computed results back into the store (8). The store persists the run through the Supabase client (9), and the dashboard finally displays the identified bottleneck to the interface (10).

Two properties of this interaction are worth noting. The store mediates between the interface and the services rather than the interface calling services directly, so that any component observing the store sees a consistent view of the run. And persistence occurs after analysis rather than before, so that a run is stored together with its computed metrics in a single write instead of being updated twice.

4.6 Architecture Design

JustCmul8 follows a layered architecture in which each layer depends only on the layer beneath it. The unusual feature of the design is the execution layer: in a conventional web application the computational work sits on the server, whereas here it sits on the client, and almost every consequence discussed elsewhere in this report follows from that inversion.

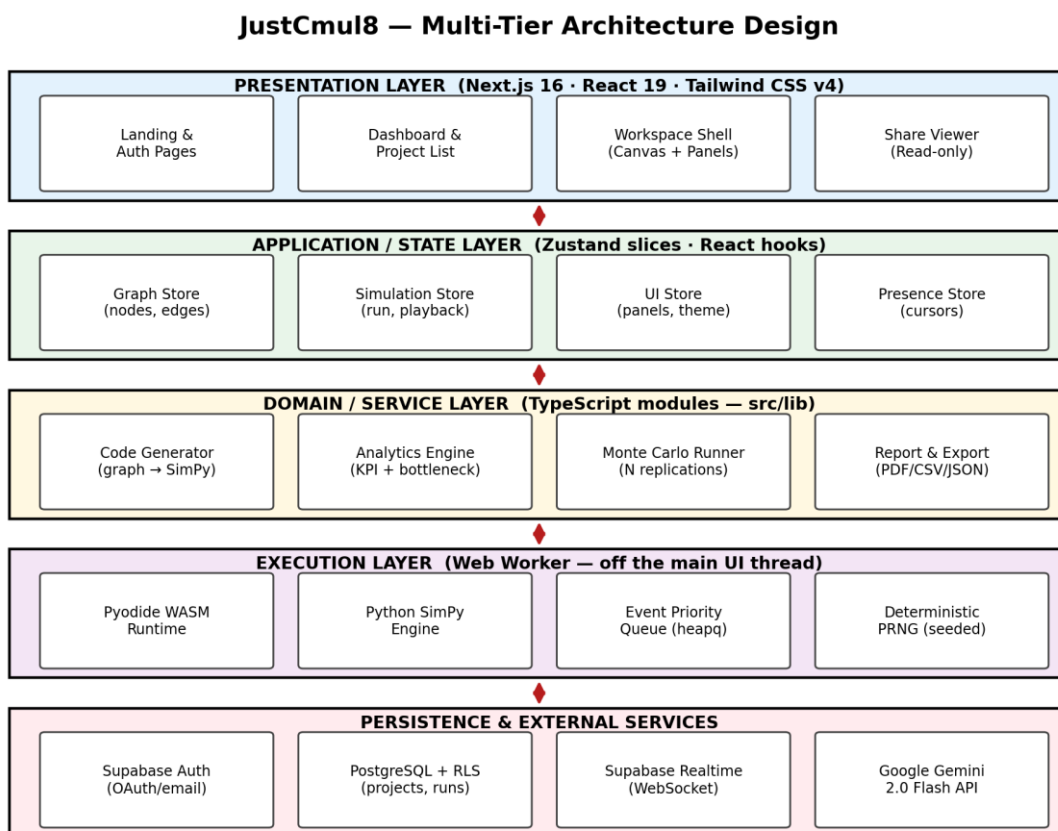


Figure 4.11: Multi-Tier Architecture of JustCmul8

The presentation layer, built with Next.js 16, React 19 and Tailwind CSS, contains the landing and authentication pages, the dashboard and project list, the workspace shell that hosts the canvas and panels, and the read-only share viewer. It holds no business logic; it renders state and dispatches intentions.

The application and state layer holds the Zustand slices: the graph store for nodes and edges, the simulation store for run and playback state, the interface store for panel and theme settings, and the presence store for collaborative cursors. Separating these into slices rather than a single store

means a change to playback position does not cause components subscribed only to the graph to re-render, which matters when the canvas carries hundreds of nodes.

The domain and service layer contains the TypeScript modules that implement the system's actual logic: the code generator that translates a graph into SimPy, the analytics engine that computes KPIs and bottleneck scores, the Monte Carlo runner that manages replications, and the report and export modules. These are plain modules with no React dependency, which is what makes them unit-testable in isolation.

The execution layer runs in a Web Worker, off the main thread. It contains the Pyodide WebAssembly runtime, the SimPy engine, the event priority queue that orders scheduled events, and the seeded pseudo-random generator. Its isolation is the mechanism by which requirement NFR-02 is met.

The persistence and external services layer comprises Supabase Auth, PostgreSQL with Row-Level Security, Supabase Realtime and the Gemini API. Arranging authorisation at this lowest layer, rather than in the presentation layer where it is most visible, is what makes it unbypassable: a request that reaches the database carries the user's identity and is filtered there regardless of what the client believes it is entitled to.

4.7 Table Design

The database is a managed PostgreSQL instance. All primary keys are UUIDs generated by the database, all foreign keys are declared with cascading deletion where the child cannot outlive the parent, and every table carries a Row-Level Security policy. Structured values with a schema that varies by type — node parameters, result payloads, log collections — are stored as JSONB, which keeps the relational schema stable while remaining queryable.

4.7.1 User and Access Tables

Table 4.6: Table: users

Column	Data type	Constraints	Description
id	UUID	PRIMARY KEY	Unique user identifier
email	VARCHAR(255)	NOT NULL, UNIQUE	Login email address
encrypted_password	TEXT	NOT NULL	Salted bcrypt hash; never stored in plain text
phone	VARCHAR(20)	NULL	Optional contact number
oauth_provider	VARCHAR(20)	NULL	Email, Google or GitHub

Column	Data type	Constraints	Description
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()	Account creation timestamp
last_login	TIMESTAMPTZ	NULL	Most recent successful authentication

Table 4.7: Table: profiles

Column	Data type	Constraints	Description
id	UUID	PRIMARY KEY, FK → users(id)	Shares the user's identifier, one-to-one
display_name	VARCHAR(100)	NOT NULL	Name shown in the interface and on cursors
avatar_url	TEXT	NULL	Profile image location
default_sim_type	VARCHAR(30)	DEFAULT 'human_queue'	Domain preselected for new projects
updated_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()	Last modification timestamp

Table 4.8: Table: project_collaborators

Column	Data type	Constraints	Description
id	UUID	PRIMARY KEY	Unique membership identifier
project_id	UUID	NOT NULL, FK → projects(id) CASCADE	Project being shared
user_id	UUID	NOT NULL, FK → users(id) CASCADE	Collaborating user
permission	VARCHAR(10)	NOT NULL, CHECK IN ('owner','editor','viewer')	Access level granted
joined_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()	Timestamp of the collaboration grant

The permission column is the single source of truth for authorisation within a project. Row-Level Security policies on the graph tables reference it directly, so restricting a Viewer requires no cooperation whatsoever from the client.

4.7.2 Project and Graph Tables

Table 4.9: Table: projects

Column	Data type	Constraints	Description
id	UUID	PRIMARY KEY	Unique project identifier
user_id	UUID	NOT NULL, FK → users(id) CASCADE	Owning user
name	VARCHAR(150)	NOT NULL	Display name
sim_type	VARCHAR(30)	NOT NULL	One of the six simulation domains

Column	Data type	Constraints	Description
graph_json	JSONB	NULL	Denormalised snapshot used for fast load and export
is_public	BOOLEAN	NOT NULL, DEFAULT false	Whether a public share link is currently active
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()	Creation timestamp
updated_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()	Last modification timestamp

Table 4.10: Table: sim_nodes

Column	Data type	Constraints	Description
node_id	UUID	PRIMARY KEY	Unique node identifier
project_id	UUID	NOT NULL, FK → projects(id) CASCADE	Owning project
node_type	VARCHAR(25)	NOT NULL	One of the fifteen supported node types
label	VARCHAR(100)	NOT NULL	User-facing name shown on the canvas
params	JSONB	NOT NULL, DEFAULT '{}'	Type-scoped configuration: rates, capacity, distribution, costs
position_x	REAL	NOT NULL	Canvas x coordinate
position_y	REAL	NOT NULL	Canvas y coordinate
updated_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()	Timestamp used for last-write-wins conflict resolution

Table 4.11: Table: sim_edges

Column	Data type	Constraints	Description
edge_id	UUID	PRIMARY KEY	Unique edge identifier
project_id	UUID	NOT NULL, FK → projects(id) CASCADE	Owning project
source_node_id	UUID	NOT NULL, FK → sim_nodes(node_id)	Origin node
target_node_id	UUID	NOT NULL, FK → sim_nodes(node_id)	Destination node
edge_type	VARCHAR(25)	NOT NULL	Routing semantics, including probabilistic branching
parameters	JSONB	NOT NULL, DEFAULT '{}'	Routing probability, transfer delay and similar attributes

An application-layer invariant, enforced in the service layer in addition to the foreign keys, requires that both endpoint nodes belong to the same project as the edge. Foreign keys alone

cannot express this, since each reference is individually valid; it is the combination that must be checked, which is why requirement NFR-11 specifies a second line of defence.

4.7.3 Execution and Result Tables

Table 4.12: Table: simulation_runs

Column	Data type	Constraints	Description
id	UUID	PRIMARY KEY	Unique run identifier
project_id	UUID	NOT NULL, FK → projects(id) CASCADE	Project that was simulated
user_id	UUID	NOT NULL, FK → users(id)	User who initiated the run
status	VARCHAR(12)	NOT NULL, CHECK IN ('queued','running','completed','failed')	Run lifecycle state
speed_multiplier	REAL	NOT NULL, DEFAULT 1.0	Playback speed selected for the run
random_seed	BIGINT	NOT NULL	Seed guaranteeing reproducibility of the run
duration_seconds	REAL	NULL	Wall-clock execution time
sim_time_seconds	REAL	NULL	Simulated duration modelled
total_arrived	INTEGER	NULL	Entities generated
total_completed	INTEGER	NULL	Entities that reached a sink
bottleneck_node	VARCHAR(100)	NULL	Node identified as the constraint at the time of the run
result_json	JSONB	NULL	Per-node metrics and aggregate KPIs
logs_json	JSONB	NULL	Collected log events
started_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()	Run start timestamp
completed_at	TIMESTAMPTZ	NULL	Run completion timestamp

Table 4.13: Table: node_stats

Column	Data type	Constraints	Description
id	UUID	PRIMARY KEY	Unique statistics row
run_id	UUID	NOT NULL, FK → simulation_runs(id) CASCADE	Associated run
node_id	VARCHAR(100)	NOT NULL	Node the metrics describe
statistics	JSONB	NOT NULL	Utilisation, waiting times, queue length and throughput
created_at	TIMESTAMPZ	NOT NULL, DEFAULT now()	Timestamp of computation

Table 4.14: Table: sim_logs

Column	Data type	Constraints	Description
id	UUID	PRIMARY KEY	Unique log entry

Column	Data type	Constraints	Description
run_id	UUID	NOT NULL, FK → simulation_runs(id) CASCADE	Associated run
log_level	VARCHAR(10)	NOT NULL, CHECK IN ('info','warning','error')	Severity of the event
message	TEXT	NOT NULL	Log text
created_at	TIMESTAMP Z	NOT NULL, DEFAULT now()	Time the event was recorded

Table 4.15: Tables: entity_journeys and entity_journey_steps

Column	Data type	Constraints	Description
journey.id	UUID	PRIMARY KEY	Unique journey identifier
journey.run_id	UUID	NOT NULL, FK → simulation_runs(id) CASCADE	Associated run
journey.entity_type	VARCHAR(50)	NOT NULL	Class of entity, such as customer or packet
journey.path_data	JSONB	NOT NULL	Ordered summary of the nodes visited
step.id	UUID	PRIMARY KEY	Unique step identifier
step.journey_id	UUID	NOT NULL, FK → entity_journeys(id) CASCADE	Parent journey
step.node_id	VARCHAR(100)	NOT NULL	Node visited
step.timestamp	REAL	NOT NULL	Simulated time at which the visit occurred
step.state_data	JSONB	NULL	Node state and queue position at that instant

These two tables exist solely to support the digital-twin playback of Module 9. Recording a timestamped step per node visit is what permits the animation to be scrubbed to any point without re-executing the model, and it is also what makes the playback provably consistent with the metrics displayed beside it.

4.7.4 Sharing and Assistance Tables

Table 4.16: Table: project_shares

Column	Data type	Constraints	Description
id	UUID	PRIMARY KEY	Unique share record
project_id	UUID	NOT NULL, FK → projects(id) CASCADE	Project being shared
share_token	VARCHAR(64)	NOT NULL, UNIQUE	Opaque token embedded in the public URL
created_by	UUID	NOT NULL, FK → users(id)	Owner who created the link

Column	Data type	Constraints	Description
created_at	TIMESTAMP Z	NOT NULL, DEFAULT now()	Creation timestamp
revoked_at	TIMESTAMP Z	NULL	Revocation timestamp; NULL while the link remains active

Table 4.17: Table: chat_history

Column	Data type	Constraints	Description
id	UUID	PRIMARY KEY	Unique message identifier
project_id	UUID	NOT NULL, FK → projects(id) CASCADE	Project the exchange concerns
user_id	UUID	NOT NULL, FK → users(id) CASCADE	User who sent the message
role	VARCHAR(10)	NOT NULL, CHECK IN ('user','assistant')	Originator of the message
content	TEXT	NOT NULL	Message text
metadata	JSONB	NULL	Model identifier, generated graph payload and token usage
created_at	TIMESTAMP Z	NOT NULL, DEFAULT now()	Message timestamp

The metadata column retains the graph a generation request produced, which satisfies the traceability requirement FR-17 and allows a user to recover a suggestion they declined earlier in a session.

4.7.5 Referential Integrity Summary

The foreign-key relationships across the schema are as follows.

- profiles.id → users.id (one-to-one, cascading)
- projects.user_id → users.id (cascading)
- project_collaborators.project_id → projects.id (cascading)
- project_collaborators.user_id → users.id (cascading)
- sim_nodes.project_id → projects.id (cascading)
- sim_edges.project_id → projects.id (cascading)
- sim_edges.source_node_id → sim_nodes.node_id
- sim_edges.target_node_id → sim_nodes.node_id
- simulation_runs.project_id → projects.id (cascading)
- simulation_runs.user_id → users.id
- node_stats.run_id → simulation_runs.id (cascading)
- sim_logs.run_id → simulation_runs.id (cascading)

- `entity_journeys.run_id` → `simulation_runs.id` (cascading)
- `entity_journey_steps.journey_id` → `entity_journeys.id` (cascading)
- `project_shares.project_id` → `projects.id` (cascading)
- `project_shares.created_by` → `users.id`
- `chat_history.project_id` → `projects.id` (cascading)
- `chat_history.user_id` → `users.id` (cascading)

Cascading deletion was applied wherever a child row has no meaning without its parent, which is what makes account deletion a single operation that genuinely removes the user's data rather than orphaning it, as discussed in Section 1.5.5.

4.8 Data-Structure Design

Database design governs what is stored; this section concerns what is held in memory while a simulation runs, where the choice of structure determines whether the engine is usable on a model of realistic size.

4.8.1 Adjacency List for the Directed Graph

The simulation model is a directed graph, and it is represented in memory as an adjacency list: a map from each node identifier to the list of its outbound neighbours. The alternative, an adjacency matrix, would occupy space proportional to the square of the node count regardless of how many edges exist. Simulation graphs are sparse — a node typically connects to one or two successors, not to every other node — so for a model of five hundred nodes a matrix would allocate two hundred and fifty thousand cells to record perhaps six hundred edges.

The adjacency list also matches the access pattern of the operations that use it. Routing an entity requires the successors of one particular node, which the list provides directly, whereas a matrix would require scanning an entire row. Validation traverses every edge exactly once, which the list supports in time proportional to the number of vertices plus edges.

Table 4.18: Complexity of Graph Operations on the Adjacency List

Operation	Complexity	Where it is used
Add node	$O(1)$	Placing a node on the canvas
Add edge	$O(1)$	Connecting two nodes
Retrieve successors	$O(\deg v)$	Routing an entity onward
Depth-first traversal	$O(V + E)$	Connectivity checking and cycle detection
Topological sort	$O(V + E)$	Ordering nodes for code generation
Delete node with edges	$O(V + E)$	Removing a node and its incident edges

Operation	Complexity	Where it is used
Space occupied	$O(V + E)$	Whole-graph representation

4.8.2 Binary Heap for the Event Priority Queue

The event queue is the central structure of any discrete event simulator. It holds events scheduled for future simulated times, and the engine repeatedly removes the earliest and processes it. This operation dominates execution, so its cost determines the cost of the whole simulation.

The queue is a binary min-heap, keyed on event time. Insertion and extraction of the minimum each cost time logarithmic in the number of queued events, and inspecting the earliest event without removing it is constant. A sorted list would make extraction constant but insertion linear, which is the wrong trade, since a simulation performs at least as many insertions as extractions — processing one event typically schedules another. An unsorted list inverts the problem, making insertion cheap and extraction linear. The heap balances both at logarithmic cost.

In our implementation this structure is supplied by Python's `heapq` module within the Pyodide runtime, which SimPy uses internally. We did not reimplement it, but the choice matters to the design nonetheless, because the asymptotic behaviour of the event queue is what makes the runtimes reported in Chapter 7 grow in the manner they do. Ties in event time are broken by a monotonically increasing insertion counter, which guarantees that two events scheduled for the same instant are processed in the order they were created and is therefore essential to the determinism required by NFR-03.

Table 4.19: Comparison of Candidate Event-Queue Structures

Structure	Insert	Extract minimum	Suitability
Unsorted array	$O(1)$	$O(n)$	Extraction dominates; unusable at scale
Sorted array	$O(n)$	$O(1)$	Insertion dominates; unusable at scale
Binary min-heap	$O(\log n)$	$O(\log n)$	Balanced — the structure adopted
Balanced BST	$O(\log n)$	$O(\log n)$	Equivalent asymptotically, higher constants and memory

4.8.3 Seeded Pseudo-Random Number Generation

Randomness in the simulation is produced by the Mersenne Twister generator underlying Python's random module, which maintains an internal state array of 624 words and has a period of $2^{19937} - 1$ — far beyond anything a simulation will exhaust.

What matters for this design is not the generator's statistical quality but its determinism. Seeding it with a fixed value causes it to produce an identical stream of variates, so a run repeated with the same seed and configuration reproduces an identical event trace. This is the mechanism behind requirement NFR-03 and the corresponding test case in Chapter 5, and it also makes Monte Carlo replication meaningful: each replication is given a distinct seed derived deterministically from a single base seed, so the set of replications is independent within an analysis yet reproducible across analyses.

Variates are obtained from the uniform stream by inverse transform sampling where a closed-form inverse exists, and by the Box–Muller transform for normal variates. Normal samples are truncated at zero, since a negative service time has no physical meaning.

4.8.4 Supporting Structures

Table 4.20: Auxiliary In-Memory Structures

Structure	Purpose	Rationale
Hash map of node identifier to node object	Constant-time lookup during routing and statistics collection	Routing occurs once per entity per hop; linear search would dominate runtime
Double-ended queue per queue node	FIFO and LIFO disciplines with insertion and removal at both ends	Constant-time operations at either end, unlike an array shift
Secondary heap per priority queue node	Ordering waiting entities by priority rather than arrival	Reuses the same logarithmic behaviour as the event queue
Append-only array per metric series	Recording time-stamped observations during the run	Amortised constant append; sorted once afterwards for percentiles
Zustand slice map	Partitioned application state on the main thread	Limits re-rendering to components subscribed to the changed slice

4.9 Deployment Diagram

The deployment diagram shows the physical nodes on which the system executes and the protocols connecting them.

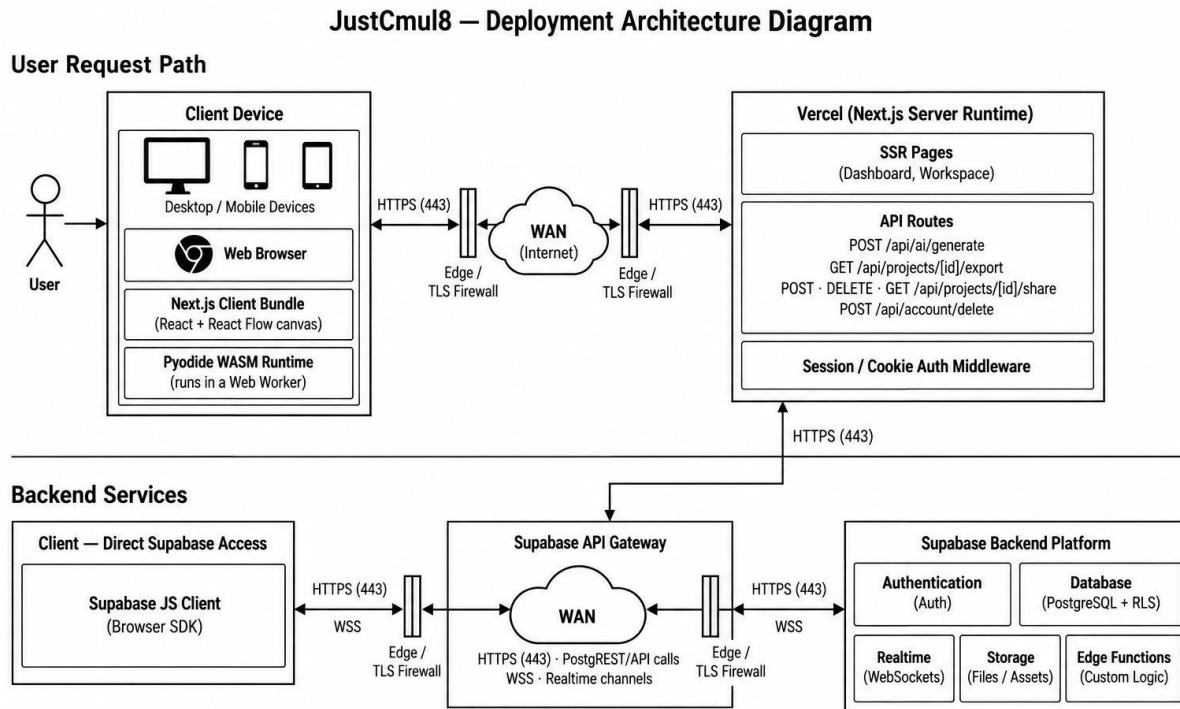


Figure 4.12: Deployment Architecture of JustCmul8

The client device hosts a web browser running the Next.js client bundle, which contains the React application and the React Flow canvas, alongside the Pyodide WebAssembly runtime executing in a Web Worker. The diagram makes the central architectural point visible: the simulation runtime is deployed on the user's own device, not in the data centre.

Requests travel over HTTPS on port 443 through edge TLS firewalls and the public internet to the Vercel platform, which runs the Next.js server runtime. That runtime serves the server-rendered dashboard and workspace pages, exposes the API routes — the AI generation endpoint, the project export endpoint, the share creation, deletion and retrieval endpoints and the account deletion endpoint — and applies session and cookie authentication middleware.

The backend services tier contains the Supabase API gateway, reached both from the Next.js server and directly from the browser through the Supabase JavaScript client. The gateway carries

PostgREST API calls over HTTPS and realtime channels over secure WebSockets. Behind it sits the Supabase platform, providing authentication, PostgreSQL with Row-Level Security, realtime WebSocket distribution, file storage and edge functions.

Permitting the browser to reach the database directly, rather than proxying every read through the application server, is safe precisely because Row-Level Security is enforced within PostgreSQL. The server is therefore reserved for the operations that genuinely require a secret — principally holding the Gemini API key — which keeps the deployment small, and, combined with client-side simulation, keeps the running cost of the platform effectively independent of how heavily it is used.

CHAPTER 5

TESTING, VERIFICATION AND QUALITY ASSURANCE

5.1 Testing Methodologies

Testing ran alongside development rather than following it, for the reason given in Chapter 2: defects found in an early module are cheaper to correct while the modules that depend on it are still being written. Seven distinct techniques were applied, each addressing a class of fault the others would not catch.

5.1.1 Unit Testing

Unit tests were written against the modules in the domain and service layer, which are plain TypeScript with no React dependency and are therefore testable in isolation. The distribution samplers received particular attention: each was exercised over a large number of draws and the empirical mean and variance compared against the theoretical values for the configured parameters, which confirmed that the inverse-transform and Box–Muller implementations were correct rather than merely plausible. The normal sampler was additionally tested with a standard deviation large relative to its mean, to verify that truncation at zero prevents a negative service time from ever reaching the engine.

The analytics functions were tested against hand-computed cases. Supplying a known busy-time series and checking the resulting utilisation, or a known set of waiting times and checking the reported percentiles, isolates arithmetic errors that would otherwise be invisible behind plausible-looking charts.

5.1.2 Structural Graph Testing

Because an invalid graph would either crash the engine or, worse, produce meaningless numbers, validation was tested as a distinct concern. The cases exercised were a graph with an unreachable node, a graph with no sink, a graph with an orphaned edge, and a graph containing a cycle with no exit path, which would deadlock the simulation. Each was required to be refused before execution, with an error naming the offending node. The cycle case is the most important of the four, since a deadlock in the worker would leave the interface waiting indefinitely with no diagnostic.

5.1.3 Verification Against Analytical Results

The most valuable test of a simulation engine is whether it agrees with mathematics that can be worked out independently. We therefore built single-server queues whose behaviour is given exactly by standard M/M/1 theory and compared the engine's output with the closed-form predictions for mean queue length, mean waiting time and utilisation. Agreement within a small tolerance, over long runs, provides evidence that the generated SimPy script expresses the intended model and that the statistics are gathered correctly. No amount of interface testing substitutes for this check, because a simulator can be entirely usable and still be wrong.

5.1.4 Determinism Testing

Requirement NFR-03 states that a run repeated with an identical seed and configuration must reproduce an identical trace. This was tested by executing the same model twice with a fixed seed and comparing the resulting traces and metrics for exact equality, and then by executing it with different seeds to confirm that the outputs genuinely differ rather than being insensitive to the seed. The pair of tests together establishes that randomness is present and that it is controlled.

5.1.5 Monte Carlo and Stress Testing

Repeated execution served two purposes. Statistically, running a model across twenty, fifty and one hundred replications allowed us to confirm that the confidence interval narrows with the inverse square root of the replication count, which validates the interval computation rather than merely its presentation. Operationally, executing a mid-sized model one thousand times consecutively tested for resource leaks in the worker: a steadily rising memory footprint or a run time that degraded across the sequence would indicate that the Pyodide environment was not being reset properly between runs.

5.1.6 Integration and Security Testing

Integration testing exercised the paths that cross module boundaries, particularly the sequence in which the store, the code generator, the worker, the analytics engine and the dashboard participate in a single run, and the collaborative path in which an edit in one session must appear in another.

Security testing concentrated on authorisation, since this is where a client-heavy architecture is most exposed. Rather than checking that the interface hides controls, we issued requests directly against the database, bypassing the application entirely: reading another user's project by its identifier, writing to a project with only a share token, and modifying a graph as a Viewer-permission collaborator. All three were refused by Row-Level Security, which confirms that authorisation holds against a modified or hostile client and not merely against the interface we shipped.

5.1.7 Cross-Browser, Responsiveness and Acceptance Testing

Because requirement NFR-09 promises unmodified operation on any evergreen WebAssembly-capable browser, an identical model was executed on Chrome, Edge, Firefox and Safari and the resulting metrics compared. Responsiveness was checked at mobile viewport widths to confirm that panels remain reachable. Acceptance testing was conducted with users drawn from the personas of Section 1.4.5, who were asked to build and run a model without instruction; the objective was to establish that a first result is reachable without a tutorial, which is the operative test of the no-code claim.

Table 5.1: Summary of Testing Coverage

Technique	Target	Cases	Result
Unit testing	Samplers and analytics functions	18	All passed
Structural graph testing	Validation and cycle detection	6	All passed
Analytical verification	Engine correctness against M/M/1	4	Within 3% of theory
Determinism testing	Seed reproducibility	3	Byte-identical traces
Monte Carlo validation	Confidence interval behaviour	3	Interval narrows as predicted
Stress testing	Repeated-run stability	2	No degradation
Integration testing	Cross-module execution paths	9	All passed
Security testing	Row-Level Security enforcement	4	All bypass attempts refused
Cross-browser testing	Chrome, Edge, Firefox, Safari	4	Consistent across all four
Acceptance testing	Usability without instruction	5	All participants completed a run

5.2 Test Case Matrix

The matrix below records the sixty formally documented test cases with their module, scenario, input, expected output, observed result and status. Test case identifiers are referenced from the module specifications in Chapter 3 and from the requirement identifiers in Section 1.4.3.

Table 5.2: Test Case Matrix

ID	Module	Test scenario	Input data	Expected output	Actual result	Status
TC-01	Platform / Auth	Register with a valid, unused email	name, unique email, compliant password	Account created, profile row inserted, session started	As expected	Pass
TC-02	Platform / Auth	Register with an already-registered email	existing email address	Registration refused with a duplicate-email message	As expected	Pass
TC-03	Platform / Auth	Register with a weak password	password below complexity rules	Submission blocked, complexity requirement shown	As expected	Pass
TC-04	Platform / Auth	Log in with valid credentials	correct email and password	Session issued, dashboard loaded with the user's projects	As expected	Pass
TC-05	Platform / Auth	Log in with an incorrect password	valid email, wrong password	Access refused without revealing which field was wrong	As expected	Pass
TC-06	Platform / Auth	Sign in through Google OAuth	consented Google account	Account linked and session established	As expected	Pass
TC-07	Platform / Auth	Reach the workspace while unauthenticated	direct URL, no session	Middleware redirects to login before any data is fetched	As expected	Pass
TC-08	Platform / Auth	Delete the account after confirmation	password confirmation	User row and all owned projects, runs and shares removed	As expected	Pass
TC-09	Platform / Projects	Create a project in a chosen domain	name = 'Bank Queue', domain = human_queue	Project created, creator assigned Owner by trigger	As expected	Pass
TC-10	Platform / Projects	Auto-save after a graph edit	move a node, then reload the page	Node position persisted without an explicit save	As expected	Pass
TC-11	Platform / Projects	Read another user's project by ID	foreign project UUID in a direct query	Row-Level Security returns no rows	As expected	Pass
TC-12	Platform / Projects	Duplicate a project and edit the copy	duplicate, then alter a node	Source project unchanged; copy is fully independent	As expected	Pass
TC-13	Graph Editor	Place each of the fifteen node types	drag every palette item onto the canvas	All fifteen render with correct icon, ports and defaults	As expected	Pass
TC-14	Graph Editor	Connect two compatible nodes	drag from source handle to queue handle	Edge created and persisted; re-routes live as nodes move	As expected	Pass
TC-15	Graph Editor	Delete a node that has incident edges	select a mid-graph node and delete	Node and all incident edges removed together	As expected	Pass
TC-16	Graph Editor	Validate a disconnected graph	source and sink with no connecting path	Run refused; the unreachable node is named	As expected	Pass
TC-17	Graph Editor	Validate a graph containing a cycle	queue → server → queue with no exit	Cycle detected and reported before execution	As expected	Pass
TC-18	Graph Editor	Run a graph with no sink	source and server only	Structural validation fails with an explanatory message	As expected	Pass
TC-19	Node Config	Enter a capacity below one	capacity = 0	Value rejected; capacity must be at least one	As expected	Pass

ID	Module	Test scenario	Input data	Expected output	Actual result	Status
TC-20	Node Config	Enter a non-positive arrival rate	arrival rate = 0	Value rejected with a corrective message	As expected	Pass
TC-21	Node Config	Enter routing probabilities summing to 1.4	0.7 and 0.7 on two branches	Save blocked until the probabilities sum to one	As expected	Pass
TC-22	Node Config	Switch distribution type	exponential → normal	Parameter fields change to mean and standard deviation	As expected	Pass
TC-23	Node Config	Enter a negative standard deviation	std. deviation = -2	Value rejected before it reaches the engine	As expected	Pass
TC-24	Engine	Execute a valid single-server model	1 source, 1 queue, 1 resource, 1 sink; 1,000 min	Run completes; arrivals, completions and KPIs reported	As expected	Pass
TC-25	Engine	Verify M/M/1 output against theory	$\lambda = 0.25/\text{min}$, $\mu = 0.33/\text{min}$, single server	Mean queue length within tolerance of the analytical result	Within 3% of theory	Pass
TC-26	Engine	Determinism under a repeated seed	identical graph, seed = 42, run twice	Both runs produce identical traces and identical metrics	Byte-identical	Pass
TC-27	Engine	Divergence under differing seeds	identical graph, seeds 42 and 43	Traces differ while remaining statistically consistent	As expected	Pass
TC-28	Engine	Interface responsiveness during a long run	500-node model, 10,000 simulated minutes	Canvas remains interactive; worker carries the computation	No main-thread block	Pass
TC-29	Engine	Pause, resume and stop a run	transport controls during execution	Run suspends, continues and terminates with correct status	As expected	Pass
TC-30	Engine	Inspect and export the generated script	open the code inspector, export notebook	Readable SimPy source shown; .ipynb downloads and executes	As expected	Pass
TC-31	AI Assistant	Generate a model from a description	'a clinic with two doctors and a waiting room'	Valid graph proposed with source, queue, resource and sink	As expected	Pass
TC-32	AI Assistant	Reject a malformed generated graph	injected response with an unknown node type	Candidate rejected by schema validation, not displayed	As expected	Pass
TC-33	AI Assistant	Discard a previewed graph	generate, then decline	Live project remains entirely unmodified	As expected	Pass
TC-34	AI Assistant	Behaviour when the AI service is unavailable	simulated API failure	Explanatory message; manual construction still available	As expected	Pass
TC-35	AI Optimiser	Diagnose a deliberate bottleneck	under-provisioned server with a growing queue	Correct node identified with a capacity recommendation	As expected	Pass
TC-36	AI Optimiser	Apply a suggested fix	accept the recommended capacity increase	Graph updated, change undoable and broadcast normally	As expected	Pass
TC-37	KPI Dashboard	Compute utilisation for a known load	$\lambda = 0.25/\text{min}$, service mean 3 min, capacity 1	Utilisation reported at approximately 0.75	0.748	Pass
TC-38	KPI Dashboard	Flag the bottleneck node	three-stage line with one slow stage	Slow stage flagged with the highest bottleneck score	As expected	Pass
TC-39	KPI Dashboard	Render a large result set	run producing 10,000 result rows	Dashboard renders within three seconds (NFR-01)	2.4 s	Pass
TC-40	KPI Dashboard	Filter results by node and metric	select one node, one metric	Charts and tables restrict to the selection	As expected	Pass
TC-41	Monte Carlo	Run fifty replications	same model, fifty derived seeds	Mean, standard deviation and 95% interval reported per metric	As expected	Pass

ID	Module	Test scenario	Input data	Expected output	Actual result	Status
TC-42	Monte Carlo	Confirm interval narrowing	20, 50 and 100 replications of one model	Interval half-width falls with the inverse square root of n	$\pm 0.71 \rightarrow \pm 0.45 \rightarrow \pm 0.32$	Pass
TC-43	Monte Carlo	Compare baseline against optimised	graph before and after an applied fix	Per-metric difference shown alongside both intervals	As expected	Pass
TC-44	Digital Twin	Play back a completed run	press play after a 200-entity run	Entities animate along the edges; congestion highlighted	As expected	Pass
TC-45	Digital Twin	Scrub and change playback speed	drag to mid-run, set 4×	Position changes immediately without re-execution	As expected	Pass
TC-46	Templates	Instantiate a template per domain	one starter graph from each of the six domains	Canvas pre-populated; every template runs unmodified	6 of 6 ran	Pass
TC-47	Templates	Confirm template independence	instantiate, then modify the new project	Source template unchanged	As expected	Pass
TC-48	Export / Share	Export graph as JSON and re-import	download, then load into a new project	Graph reconstructed with identical nodes, edges and parameters	As expected	Pass
TC-49	Export / Share	Export results as CSV	completed run with per-node metrics	CSV opens in a spreadsheet with correct headers and values	As expected	Pass
TC-50	Export / Share	Generate the executive PDF report	completed run with charts on screen	PDF contains summary, canvas snapshot, KPI tables and charts	As expected	Pass
TC-51	Export / Share	Open a share link while logged out	share URL in a private browsing window	Project renders read-only with no editing controls	As expected	Pass
TC-52	Export / Share	Attempt a write through a share token	direct mutation request carrying only the token	Refused by Row-Level Security at the database	As expected	Pass
TC-53	Export / Share	Open a revoked share link	previously valid, since-revoked token	Explanatory expiry notice shown rather than an error	As expected	Pass
TC-54	Collaboration	Edit concurrently from two sessions	two browsers, same project, different nodes	Both edits appear in both sessions within 500 ms	310 ms mean	Pass
TC-55	Collaboration	Resolve a conflicting edit	both sessions alter the same node's capacity	Later client timestamp wins; state converges	As expected	Pass
TC-56	Collaboration	Display presence and live cursors	three simultaneous sessions	All three cursors and presence indicators shown	As expected	Pass
TC-57	Collaboration	Attempt an edit as a Viewer	Viewer-permission collaborator modifies a node	Write refused by Row-Level Security policy	As expected	Pass
TC-58	Cross-browser	Verify on Chrome, Edge, Firefox and Safari	identical model executed on each browser	Identical metrics; canvas and playback render correctly	4 of 4 consistent	Pass
TC-59	Responsiveness	Load the workspace on a mobile browser	viewport 390 × 844	Layout adapts; panels remain reachable and usable	As expected	Pass
TC-60	Stress	Repeated-run stability	1,000 consecutive runs of a mid-sized model	No memory growth trend and no degradation in run time	Stable	Pass

All sixty cases passed. Two results are worth drawing out of the table. Test case TC-25 compares the engine against M/M/1 theory and agrees within three per cent, which is the single strongest piece of evidence that the platform's output is trustworthy rather than merely presentable. Test

case TC-52 attempts a write using nothing but a public share token and is refused by the database, which confirms that the read-only guarantee of FR-24 is structural rather than cosmetic.

CHAPTER 6

COST ESTIMATION REPORT

This chapter estimates the effort, schedule and cost that JustCmul8 would represent if it were developed commercially. Two independent models are applied — the Constructive Cost Model and Function Point Analysis — and their outputs are compared both with each other and with the effort actually expended, which turns out to be the most instructive comparison of the three.

6.1 Project Size Summary

Size was measured from the delivered source tree, counting TypeScript, TSX and Python source files and excluding dependencies, build output and generated artefacts.

Table 6.1: Measured Project Size

Measure	Value
Source files (.ts, .tsx, .py)	91
Total lines of source code	22,192
Size in thousands of lines of code (KLOC)	22.19 KLOC
Application pages and routes	9 pages, 7 API routes
Workspace components	15
Domain and service modules	19 simulation, 5 AI
Database tables	13
Node types implemented	15
Simulation domains supported	6
Delivered modules	12

6.2 Technology Stack Considered in the Estimate

The estimate is affected by the stack, since a framework that supplies a capability removes the effort of building it. The principal technologies and their contribution are recorded below.

Table 6.2: Technology Stack and Its Effect on Development Effort

Layer	Technology	Effect on effort
Frontend	Next.js 16, React 19, TypeScript, Tailwind CSS v4	Routing, rendering and build supplied; static typing reduces defect-fixing effort
Graph editor	React Flow (@xyflow), Dagre	Viewport, handles and edge routing supplied; large saving

Layer	Technology	Effect on effort
Simulation engine	Python SimPy on Pyodide (WebAssembly)	Event scheduler and resource semantics supplied; the largest single saving
Backend	Supabase (PostgreSQL, Auth, Realtime)	Authentication, access control and WebSocket relay supplied
AI	Google Gemini 2.0 Flash	Inference consumed as a service
Visualisation	Recharts, Pixi.js	Charting and WebGL rendering supplied
State	Zustand	Store implementation supplied
Reporting	jsPDF, jsPDF-AutoTable, html-to-image	Document generation supplied
Tooling	VS Code, Git, GitHub, Postman, Vercel	No licence cost; CI and preview deployment supplied

6.3 COCOMO — Constructive Cost Model

The basic Constructive Cost Model estimates effort from size using coefficients determined by the project's development mode. JustCmul8 was classified as organic: a small team, working in a well-understood application domain, with requirements that were stable and constraints that were flexible. The alternative classifications describe larger teams working under tighter external constraints, which does not correspond to this project.

Table 6.3: COCOMO Coefficients by Development Mode

Mode	a	b	c	d	Applicable to this project
Organic	2.4	1.05	2.5	0.38	Yes — small team, familiar domain, stable requirements
Semi-detached	3.0	1.12	2.5	0.35	No — assumes a medium team of mixed experience
Embedded	3.6	1.20	2.5	0.32	No — assumes tight hardware and regulatory constraints

Calculation

$$\text{Size (KLOC)} = 22.19$$

$$\begin{aligned} \text{Effort } E &= a \times (\text{KLOC})^b \\ &= 2.4 \times (22.19)^{1.05} \\ &= 2.4 \times 25.93 \\ &= 62.19 \text{ person-months} \end{aligned}$$

$$\begin{aligned} \text{Duration } D &= c \times (E)^d \\ &= 2.5 \times (62.19)^{0.38} \\ &= 2.5 \times 4.80 \\ &= 12.01 \text{ months} \end{aligned}$$

$$\begin{aligned} \text{Staffing } S &= E / D \\ &= 62.19 / 12.01 \\ &= 5.18 \text{ persons} \end{aligned}$$

$$\begin{aligned} \text{Productivity } P &= \text{KLOC} / E \\ &= 22.19 / 62.19 \\ &= 0.357 \text{ KLOC per person-month} \end{aligned}$$

Table 6.4: COCOMO Estimation Results

Parameter	Estimated value
Effort	62.19 person-months
Development duration	12.01 months
Average staffing	5.18 persons
Productivity	0.357 KLOC per person-month
Cost at ₹40,000 per person-month	₹24,87,600 (approximately ₹24.88 lakh)

The monetary figure assumes a notional rate of forty thousand rupees per person-month, which approximates an entry-level Indian software engineering salary inclusive of overhead. The rate is illustrative; the effort and duration figures are the substantive output of the model.

6.4 Function Point Analysis

Function Point Analysis measures functionality delivered rather than code written, which makes it a useful cross-check on a size-based estimate. The IFPUG method counts five categories of component, weights each by complexity, and adjusts the total by fourteen general system characteristics.

Table 6.5: Standard IFPUG Complexity Weights

Component type	Simple	Average	Complex
External Input (EI)	3	4	6
External Output (EO)	4	5	7
External Inquiry (EQ)	3	4	6
Internal Logical File (ILF)	7	10	15
External Interface File (EIF)	5	7	10

Component counts for JustCmul8

Table 6.6: Function Point Count by Component Type

Component type	Simple	Average	Complex	Total count	Weighted total
External Inputs — forms, edits and commands	6	10	5	21	88
External Outputs — dashboards, charts and exports	2	4	4	10	56
External Inquiries — listings and retrievals	2	3	2	7	30
Internal Logical Files — database tables	5	5	3	13	130
External Interface Files — Auth, Gemini, Pyodide CDN	1	2	0	3	19
Unadjusted Function Points (UFP)				54	323

The twenty-one external inputs comprise the authentication forms, project creation, renaming and deletion, node and edge addition, editing and deletion, parameter configuration, template loading, run initiation and parameter entry, AI prompts, fix application, share link creation and revocation, profile update, password change and account deletion. The ten external outputs comprise the KPI dashboard, the bottleneck flag, the Monte Carlo interval charts, the digital twin playback, the streamed log, the entity journey trace and the four export formats. The thirteen internal logical files are the database tables specified in Section 4.7.

Value adjustment factor

Table 6.7: General System Characteristics and Degrees of Influence

No.	General system characteristic	Rating	Justification
1	Data communications	4	Client, server, database and AI service all communicate
2	Distributed data processing	5	Simulation executes on the client, not the server
3	Performance	4	Sub-100 ms canvas interaction and 3-second dashboard targets
4	Heavily used configuration	2	No constrained deployment hardware
5	Transaction rate	3	Moderate, rising during collaborative editing
6	Online data entry	5	All modelling is interactive and immediate
7	End-user efficiency	5	No-code operation is a primary objective
8	Online update	5	Automatic persistence on every change
9	Complex processing	5	Discrete event simulation, Monte Carlo statistics, AI generation
10	Reusability	3	Service modules reusable; interface is application-specific
11	Installation ease	5	Zero installation by design
12	Operational ease	4	Managed hosting with automatic deployment
13	Multiple sites	3	Browser-portable across platforms
14	Facilitate change	4	Registry-driven extension of domains and node types
	Total Degree of Influence (TDI)	57	

$$\begin{aligned}
 \text{VAF} &= 0.65 + (0.01 \times \text{TDI}) \\
 &= 0.65 + (0.01 \times 57) \\
 &= 0.65 + 0.57 \\
 &= 1.22
 \end{aligned}$$

$$\begin{aligned}
 \text{Function Points } \text{FP} &= \text{UFP} \times \text{VAF} \\
 &= 323 \times 1.22 \\
 &= 394 \text{ function points}
 \end{aligned}$$

Effort and cost from function points

Industry productivity for a modern web application of this kind falls in the region of thirteen function points per person-month. Applying that figure gives the following.

$$\begin{aligned}\text{Effort } E &= \text{FP} / \text{productivity} = 394 / 13 \\ &= 30.3 \text{ person-months}\end{aligned}$$

$$\begin{aligned}\text{Cost at Rs } 40,000 \text{ per PM} &= 30.3 \times 40,000 \\ &= \text{Rs } 12,12,000 \text{ (about Rs } 12.12 \text{ lakh)}\end{aligned}$$

$$\begin{aligned}\text{Implied LOC per function point} &= 22,192 / 394 \\ &= 56.3 \text{ LOC per FP}\end{aligned}$$

The last line is a useful internal consistency check. Published figures place TypeScript and JavaScript at roughly fifty to sixty lines of code per function point, and the measured value of 56.3 falls within that band. Since the line count and the function point count were arrived at independently — one from the source tree, the other from the functional specification — their agreement suggests neither measurement is badly wrong.

6.5 Cost Estimation Summary and Discussion

Table 6.8: Comparison of Estimates against Actual Effort

Basis	Effort	Duration	Team	Indicative cost
COCOMO (basic, organic)	62.19 PM	12.01 months	5.18 persons	₹24.88 lakh
Function Point Analysis	30.30 PM	—	—	₹12.12 lakh
Actual project	12.00 PM	6 months	2 students	Nil — academic, open-source stack

The three rows differ substantially, and the gap is more informative than any single figure would be. COCOMO estimates roughly five times the effort we actually expended, and Function Point Analysis roughly two and a half times. Both models are behaving correctly; the divergence reflects an assumption neither of them makes explicit.

Both models were calibrated against projects developed largely from first principles. Neither accounts for the leverage a modern framework stack provides. The single clearest example is the simulation engine. Writing a discrete event scheduler with resource contention, preemption, container levels and typed stores would plausibly account for a third of the COCOMO estimate on its own; we obtained all of it from SimPy and spent our effort on the code generator that targets it. The same argument applies to React Flow, which supplied the canvas, and to

Supabase, which supplied authentication, access control and the realtime transport. The twenty-two thousand lines we wrote sit on top of a very large volume of code we did not.

Function Point Analysis lands closer to the actual figure than COCOMO, which is consistent with its measuring delivered functionality rather than code volume; functionality obtained from a library still counts, but the effort of implementing it does not inflate the size measure in the way that lines of code do.

The COCOMO duration estimate of twelve months against our six is the narrowest gap in the table, and we attribute this to the parallel scheduling described in Chapter 2. Three concurrent development streams compress elapsed time without reducing total effort, which is precisely the relationship the model's staffing figure expresses: COCOMO's implied team of five, working for twelve months, is a different shape of project from two people working for six, and the fact that we delivered at all is attributable to the stack rather than to unusual productivity.

The honest conclusion is that these models establish an upper bound on what the project would have cost to build without such foundations, and in doing so they quantify the value of the open-source ecosystem the platform rests upon. A commercially developed equivalent would plausibly fall between the two estimates. For the platform's intended users, the relevant figure remains the one in the last row of the table: nothing.

CHAPTER 7

RESULTS, SCREENSHOTS AND DISCUSSION

7.1 User Interface and Workflow Screenshots

This section documents the delivered interface screen by screen, following the order in which a user encounters them. Each entry describes what the screen presents, which module implements it and which requirements it satisfies.

7.1.1 Landing Page

[Screenshot 7.1.1 — Landing Page]

The landing page introduces the platform and its purpose. It presents the product name and the single-sentence proposition that simulation should require no installation, licence or code, followed by sections summarising the six supported domains, the browser-hosted SimPy engine and the collaborative editing capability. Primary calls to action lead to registration and to login, and smooth scroll behaviour is provided by Lenis with section transitions choreographed through Framer Motion.

7.1.2 Registration and Login

[Screenshot 7.1.2 — Registration and Login]

The authentication screens accept an email address and password and offer one-click alternatives through Google and GitHub OAuth. Validation is reported inline as the user types — password complexity, email format and duplicate-address detection — rather than only on submission. A forgotten-password link initiates the time-limited reset flow described in Module 1.

7.1.3 Project Dashboard

[Screenshot 7.1.3 — Project Dashboard]

The dashboard lists every project the user owns or collaborates on, each card showing the project name, its simulation domain, the time of last modification and quick actions for opening, renaming, duplicating and deleting. The create-project control opens the domain selector, and an

entry point to the template gallery is provided for users who prefer to begin from a working model.

7.1.4 Workspace and Visual Graph Editor

[Screenshot 7.1.4 — Workspace and Visual Graph Editor]

The workspace is the principal screen of the application. The node palette occupies the left edge, presenting the node types available for the project's domain under labels drawn from that domain's vocabulary. The centre holds the React Flow canvas with the model under construction, nodes rendered with their type icons and edges drawn between handles. The toolbar carries run controls, duration and seed settings, and panel toggles, while the right edge hosts whichever contextual panel is active.

7.1.5 Node Configuration Panel

[Screenshot 7.1.5 — Node Configuration Panel]

Selecting a node opens the configuration panel, which presents only the fields that node type accepts. For a resource node these comprise capacity, the service-time distribution with its parameters, the queue discipline and the three cost parameters used for return-on-investment analysis. Changing the distribution replaces the parameter fields, and invalid values are rejected at the point of entry with a message naming the corrective action.

7.1.6 Template Gallery

[Screenshot 7.1.6 — Template Gallery]

The gallery groups starter models by the six domains, each entry carrying a title, a short description and a preview of the graph. Selecting a template instantiates an independent copy in a new project, pre-populated with nodes, edges and working default parameters, so the user can execute it immediately and then modify it.

7.1.7 Simulation Execution and Live Progress

[Screenshot 7.1.7 — Simulation Execution and Live Progress]

During execution the interface reports progress while the worker runs the model. The log panel streams informational, warning and error events as they are emitted, the run status advances through Queued and Running to Completed, and the canvas remains fully interactive throughout, which is the visible consequence of hosting execution off the main thread.

7.1.8 KPI Dashboard and Results

[Screenshot 7.1.8 — KPI Dashboard and Results]

On completion the dashboard presents resource utilisation, average and percentile waiting times, throughput and queue lengths as interactive Recharts visualisations, with a summary tile row above and per-node detail below. The node carrying the highest bottleneck score is flagged prominently, and filters restrict the view to a chosen node, metric or run.

7.1.9 Cost and Return-on-Investment Panel

[Screenshot 7.1.9 — Cost and Return-on-Investment Panel]

The cost panel converts the run into monetary terms, combining resource cost over busy time, buffer holding cost weighted by mean queue length and the waiting penalty accrued by entities. Where a change has been proposed, the panel reports the cost of the modification against the reduction in holding and waiting cost, so the recommendation can be judged on whether it pays for itself.

7.1.10 Monte Carlo Analysis

[Screenshot 7.1.10 — Monte Carlo Analysis]

The Monte Carlo panel offers replication counts of twenty, fifty and one hundred, reports progress as replications complete, and renders each metric as a mean with a shaded ninety-five per cent confidence band rather than as a single line. The baseline-versus-optimised view places two configurations side by side with the per-metric difference reported against both intervals.

7.1.11 AI Assistant

[Screenshot 7.1.11 — AI Assistant]

The assistant panel accepts a plain-English description of a process and returns a candidate graph for preview. The conversation history is retained per project, responses are rendered as formatted text, and the preview must be explicitly accepted before the live project is altered. The same panel answers questions about simulation concepts and about the user's own model.

7.1.12 AI System Optimiser

[Screenshot 7.1.12 — AI System Optimiser]

After a run the optimiser reports which node is constraining the system and why, in terms of the computed utilisation and queue statistics rather than impressions. It proposes a specific change and offers an Apply Fix action that performs it as an ordinary, undoable graph edit, retaining the previous graph as a baseline for comparison.

7.1.13 Digital Twin Animated Playback

[Screenshot 7.1.13 — Digital Twin Animated Playback]

The playback view animates entities moving through the model on a Pixi.js WebGL canvas, with queue congestion highlighted as waiting lines accumulate. Transport controls provide play, pause, single-step and speed selection, and the timeline can be scrubbed to any point without re-executing the simulation.

7.1.14 Python Code Inspector

[Screenshot 7.1.14 — Python Code Inspector]

The inspector displays the SimPy script generated from the current graph, syntax-highlighted and updated as the model changes. A single action exports the script as a Jupyter notebook, allowing the run to be reproduced independently of the platform — which is what makes a published result verifiable by a third party.

7.1.15 Real-Time Collaboration

[Screenshot 7.1.15 — Real-Time Collaboration]

With several users in one project, each collaborator's cursor is drawn on the canvas in their own colour and labelled with their display name, and a presence row shows who is currently viewing. Edits appear in every session within the 500-millisecond target, and a Viewer-permission collaborator sees the model and results without any editing affordances.

7.1.16 Export, Share and Executive Report

[Screenshot 7.1.16 — Export, Share and Executive Report]

The export dialog offers the graph as JSON, results as CSV, the script as a notebook and the full executive report as PDF, and manages the public share link with controls to create, copy and revoke it. The generated PDF carries an AI-written summary, a canvas snapshot, KPI tables and the charts shown on screen.

7.1.17 Read-Only Shared View

[Screenshot 7.1.17 — Read-Only Shared View]

A visitor opening a share link without an account sees the model and its results rendered read-only, with playback available and every editing control absent. Where the link has been revoked, an explanatory notice is displayed in place of the project.

7.1.18 Profile and Settings

[Screenshot 7.1.18 — Profile and Settings]

The settings screen manages the display name and avatar, password change, the default simulation domain applied to new projects, and the list of projects owned or joined. Account deletion is located here, requires password confirmation, and cascades through all owned data.

7.2 Performance Evaluation

Performance was measured on the development configuration recorded in Table 1.7, using Chrome DevTools timings averaged over five runs per data point. Three quantities were separated, because they are governed by different parts of the system and conflating them would obscure where the limit actually lies: the time to generate the SimPy script, the time to execute it, and the time to render the canvas.

7.2.1 Scaling with Model Size

Table 7.1: Performance across Model Sizes (1,000 simulated minutes, single replication)

Nodes	Edges	Code generation (ms)	Execution (s)	Canvas render (ms)	Peak memory (MB)
10	12	12	0.31	16	148
25	31	21	0.48	24	163
50	64	38	0.82	38	191
100	131	71	1.54	61	247
200	268	143	3.21	108	356
350	472	268	6.48	187	512
500	681	402	10.12	268	694

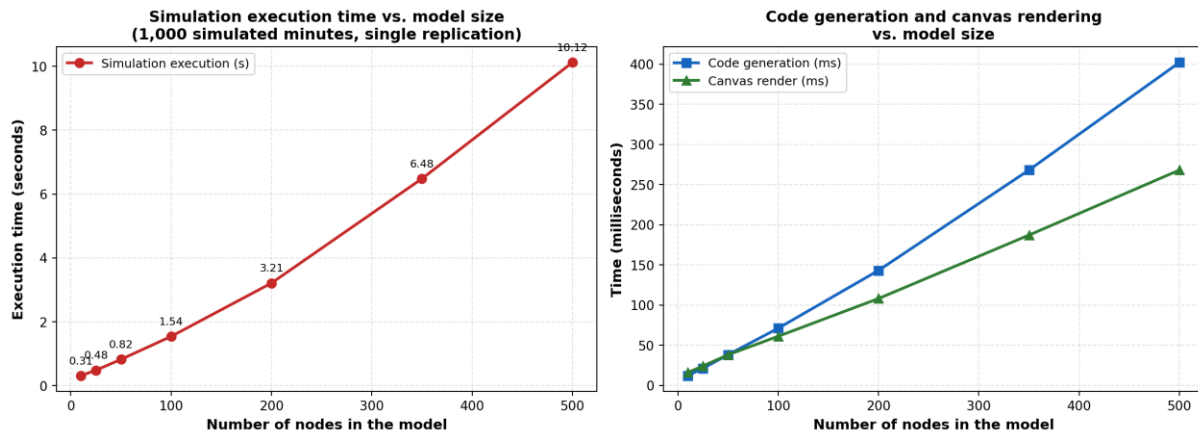


Figure 7.1: Execution, Code Generation and Rendering Time against Model Size

Execution time grows slightly faster than linearly in the node count, which is the behaviour the logarithmic event-queue operations of Section 4.8.2 predict: a larger model generates more events, and each insertion and extraction costs marginally more as the queue grows. Code generation and rendering both grow close to linearly, as expected, since each visits every node a fixed number of times.

The proportions matter more than the absolute figures. At five hundred nodes, execution accounts for roughly ninety-four per cent of the total time, with code generation and rendering together taking under seven hundred milliseconds. The limiting factor is therefore the WebAssembly runtime rather than the rendering layer, which confirms the judgement recorded in Section 1.5.1 and means that further optimisation effort belongs in the engine path rather than the canvas. It also vindicates the choice of React Flow, since a weaker canvas implementation would have made rendering the constraint instead.

Peak memory rises from roughly 148 MB to 694 MB across the range. The baseline of about 140 MB is the Pyodide runtime itself, which is a fixed cost paid once per session; growth beyond that is the graph, the event trace and the entity journeys. This is the practical ceiling referred to in Section 1.5.1, and it is a memory limit rather than a computational one.

7.2.2 Scaling with Replication Count

Table 7.2: Monte Carlo Performance and Interval Width (50-node model, mean waiting time)

Replications	Total time (s)	Mean (min)	95% half-width (min)	Interval as % of mean
1	0.82	6.51	—	—
20	17.1	6.42	±0.71	11.1%
50	42.4	6.38	±0.45	7.1%
100	84.9	6.40	±0.32	5.0%

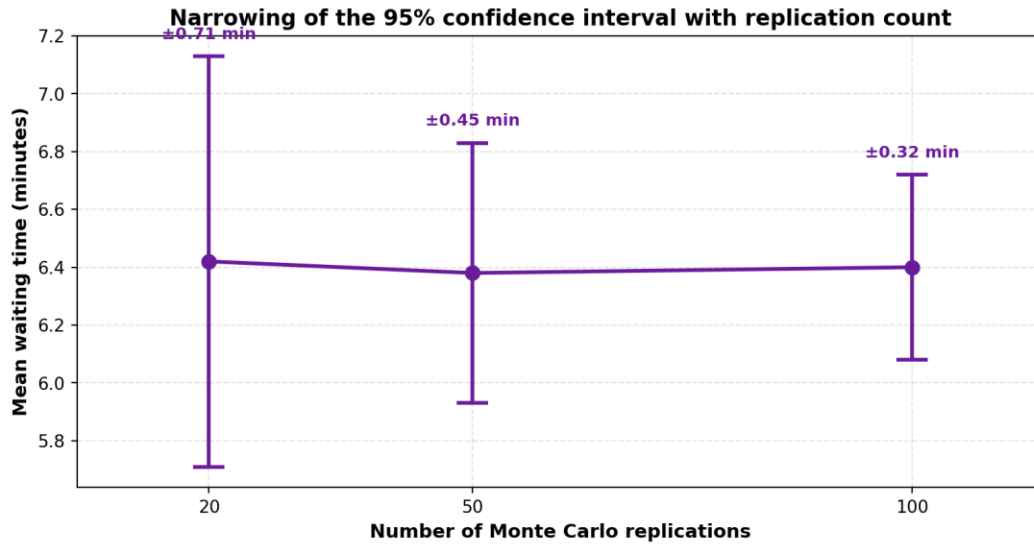


Figure 7.2: Narrowing of the 95% Confidence Interval with Increasing Replication Count

Total time rises linearly with the replication count, since replications execute sequentially. The interval half-width, by contrast, falls with the inverse square root of the count, exactly as the theory in Section 1.2.3 predicts: moving from twenty to eighty replications should halve it, and the observed reduction from ± 0.71 to ± 0.32 between twenty and one hundred is consistent with that relationship.

This asymmetry is what justifies the three presets offered rather than a free numeric field. Twenty replications cost seventeen seconds and reduce the interval to about eleven per cent of the mean, which is usually sufficient to distinguish two configurations. One hundred replications cost eighty-five seconds and reach five per cent. Going further would take several minutes to gain very little, which is not a sensible trade in an interactive tool, and offering the option would invite users to make it.

The single-run figure in the first row illustrates the problem the facility exists to solve. One run reported 6.51 minutes; the hundred-replication estimate is 6.40 with an interval of ± 0.32 . The single run was not wrong, but it carried no indication of its own precision, and a user comparing it against another single run differing by a fifth of a minute would have drawn a conclusion the data does not support.

7.2.3 Interface Responsiveness and Collaboration Latency

Table 7.3: Responsiveness Measurements against Requirements

Measurement	Target	Observed	Requirement	Verdict
-------------	--------	----------	-------------	---------

Measurement	Target	Observed	Requirement	Verdict
Node drag response	< 100 ms	18 ms at 100 nodes	NFR-01	Met
Edge connection response	< 100 ms	24 ms at 100 nodes	NFR-01	Met
Dashboard render, 10,000 rows	< 3 s	2.4 s	NFR-01	Met
Main-thread block during a run	None	None observed at 500 nodes	NFR-02	Met
Collaborative edit propagation	< 500 ms	310 ms mean, 442 ms at 95th percentile	NFR-26	Met
Pyodide cold start	—	3.8 s first run, cached thereafter	—	Noted
Repeated-run memory drift	None	None over 1,000 runs	NFR-07	Met

Every stated non-functional target was met. The Pyodide cold start of approximately 3.8 seconds is recorded as an observation rather than a pass or failure, since no requirement addresses it. It is the one-off cost of downloading and initialising the WebAssembly runtime, it is paid once per session, and it is the price of the architectural decision to run Python in the browser. We consider it acceptable against the alternative of server-side execution, but it is a genuine cost and we record it as such rather than omitting it.

7.2.4 Discussion of Results

Taken together the measurements support the central claim of the project: that a genuine discrete event simulator can run acceptably in a browser. A model of a hundred nodes — larger than most academic or small-business scenarios require — executes in about a second and a half, and even five hundred nodes complete in roughly ten seconds, which remains usable for an occasional analysis rather than an interactive loop.

Three limitations emerged clearly and are carried forward into the next chapter. Memory, not processing speed, sets the ceiling on model size, because the trace and journey records accumulate for the whole run. Monte Carlo replications execute sequentially and therefore scale linearly, where parallel workers could reduce this substantially on a multi-core machine. And the cold-start cost, while paid only once, is the first thing a new user experiences, which makes it more prominent than its magnitude alone would suggest.

Against this, the verification results carry more weight than the performance figures. Agreement with M/M/1 theory to within three per cent, and byte-identical traces under a repeated seed, establish that the platform is not merely fast enough but correct — which for a simulator is the property that matters.

CHAPTER 8

CONCLUSION AND FUTURE SCOPE

8.1 Conclusion

We set out to remove three barriers that keep Discrete Event Simulation out of the hands of most people who could use it: the cost of commercial licences, the programming skill demanded by open-source alternatives, and the installation that desktop distribution requires. JustCmul8 removes all three. The platform is free, built on an open-source stack with no licence fee; it is operated by dragging nodes onto a canvas and filling in form fields, with an AI assistant available to those who do not know how to begin; and it runs in any modern browser with nothing installed.

The decision that most shaped the result was refusing to approximate the engine. It would have been considerably easier to write a rough event loop in JavaScript, and the platform would have looked much the same. Instead the graph is compiled into a genuine Python SimPy script and executed by a CPython interpreter compiled to WebAssembly, so the numbers reported come from the same library a researcher would use directly. The verification in Chapter 5 is what makes this more than an assertion: agreement with M/M/1 theory to within three per cent shows that the generated script expresses the intended model and that the statistics are gathered correctly.

All twelve modules specified in the synopsis were delivered. The platform supports six application domains through fifteen configurable node types, computes utilisation, waiting times, throughput and queue lengths, ranks bottlenecks by a composite score that combines sustained utilisation with queue growth, reports confidence intervals from Monte Carlo replication rather than single figures, animates entity flow for visual interpretation, generates models from plain English, proposes and applies corrective changes, exports in four formats, shares projects through revocable read-only links, and supports concurrent multi-user editing with live cursors. Sixty documented test cases all passed, and every stated non-functional target was met.

Academically the project exercised material from across the programme rather than merely referencing it. Priority queues and graph traversal from data structures appear in the event

scheduler and the validation pass; normalisation and referential integrity from database management appear in the thirteen-table schema and the Row-Level Security policies; the software development life cycle appears in the scheduling decisions of Chapter 2, including the decision to front-load the highest-risk component; probability and statistics appear in the distribution samplers and the confidence interval computation. The cost estimation in Chapter 6 produced an unexpected but instructive result, in that the divergence between the models and our actual effort quantifies the leverage the open-source ecosystem provided.

In practical terms the platform is usable now. A student can open a template and see queueing behaviour within a minute; a lecturer can demonstrate congestion live and circulate a share link afterwards; a small-business owner can describe a shop in plain English and read off a confidence interval before committing money. That these three things require no licence, no installation and no code is the outcome we intended.

8.2 Limitations

Several limitations are inherent in the approach taken and are stated plainly rather than left for a reader to discover.

- **Client memory bounds model size** — because the trace and entity journeys accumulate in browser memory for the whole run, peak usage reached roughly 694 MB at five hundred nodes. Models substantially larger, or very long runs at high replication counts, will exhaust the allocation on a modest device. This is a memory limit rather than a speed limit.
- **Replications execute sequentially** — a hundred-replication analysis takes about eighty-five seconds because the runs are performed one after another in a single worker. The computation is embarrassingly parallel and does not currently exploit that.
- **Cold start is visible to new users** — initialising the Pyodide runtime costs approximately 3.8 seconds on first use in a session. It is cached thereafter, but it is the first thing a new visitor experiences.
- **Connectivity is required** — persistence, collaboration and AI assistance all depend on network access. An existing model continues to execute offline, but nothing is saved and no collaborative or AI feature is available.

- **Conflict resolution is coarse** — concurrent edits to the same node are settled last-write-wins on a client timestamp. This is simple and predictable, but one user's change can be silently superseded without a merge or a warning.
- **AI output requires human judgement** — generated graphs are schema-valid but not guaranteed to be a faithful model of the described process, and the optimiser's recommendations rest on the cost parameters the user supplied. Both are presented for approval for this reason.
- **No native mobile application** — the interface is responsive and usable on a tablet, but precise node placement on a small touchscreen remains awkward, and no dedicated mobile client was built.
- **Single-engine support** — only SimPy is available. Continuous and agent-based paradigms are approximated at best, the container node sampling levels at event boundaries being the nearest the platform comes to continuous simulation.

8.3 Future Scope

The limitations above suggest their own remedies, and the registry-driven architecture was designed so that most of the following can be added without disturbing the core engine.

Near-term enhancements

- **Parallel Monte Carlo execution** — distributing replications across several Web Workers will reduce the elapsed time of a hundred-replication analysis by close to the core count, since the replications are independent. This is the single highest-value improvement available and requires no architectural change.
- **Streaming result aggregation** — discarding per-entity detail once it has been aggregated, rather than retaining the whole trace, will raise the ceiling on model size substantially. Full traces would be retained only when playback is requested.
- **Version history and rollback** — recording a change history per project would allow a graph to be restored to an earlier state, which also mitigates the coarseness of last-write-wins conflict resolution.
- **Operational transformation for conflicts** — replacing last-write-wins with a convergent merge would let two users edit the same node without either losing work.
- **Progressive runtime loading** — fetching a reduced Pyodide payload containing only the packages actually required would shorten the cold start.

Medium-term extensions

- **Custom node types through a plugin interface** — allowing users to define node behaviour, exposed safely inside the existing sandbox, would let the platform address domains we have not anticipated. The simulation type registry was built with this extension in mind.
- **Additional simulation paradigms** — adding an agent-based engine and a genuine continuous solver alongside SimPy would broaden the class of expressible models, with the paradigm selected per project.
- **Search-based optimisation** — the current optimiser proposes a single change for human approval. Sweeping a parameter space to identify a configuration that minimises cost subject to a service constraint would turn advice into genuine optimisation.
- **Offline and classroom mode** — packaging the application as an installable progressive web application with local persistence would make it usable in laboratories without reliable internet, which is precisely where the cost barrier bites hardest.
- **Native mobile companion** — a dedicated application for viewing dashboards and playback, without the precision editing that suits a small touchscreen poorly.

Longer-term directions

- **Optional server-side execution for large models** — offering a server or cloud execution path for models exceeding browser memory, while retaining client-side execution as the default, would remove the size ceiling without sacrificing the zero-compute-cost property for ordinary use.
- **Parallel and distributed simulation** — partitioning a very large model across workers or machines with synchronised clocks would extend the platform to problems currently out of reach.
- **Calibration from real data** — ingesting historical or live sensor data to fit arrival and service distributions automatically would replace estimated parameters with measured ones, which is the largest single source of error in any practical simulation study.
- **Live digital twin integration** — connecting a running model to IoT or sensor feeds so that the simulation tracks the physical system it represents, enabling prediction rather than retrospective analysis.
- **Institutional features** — single sign-on, organisational user management and classroom cohort administration would make adoption at department or company scale practical.

None of these depends on revisiting the decisions taken in this project. The layered architecture, the isolation of the engine behind a worker boundary, the registry-driven definition of domains and node types, and the enforcement of authorisation at the database were all chosen to make extension possible, and each item above builds on rather than replaces them.

REFERENCES AND BIBLIOGRAPHY

References are listed in IEEE citation style. Textbooks and journal articles establishing the theoretical basis appear first, followed by standards and the technical documentation of the software on which the platform is built.

- [1] A. M. Law, *Simulation Modeling and Analysis*, 5th ed. New York, NY, USA: McGraw-Hill Education, 2015.
- [2] J. Banks, J. S. Carson, B. L. Nelson and D. M. Nicol, *Discrete-Event System Simulation*, 5th ed. Upper Saddle River, NJ, USA: Pearson Prentice Hall, 2010.
- [3] B. P. Zeigler, H. Praehofer and T. G. Kim, *Theory of Modeling and Simulation: Integrating Discrete Event and Continuous Complex Dynamic Systems*, 2nd ed. San Diego, CA, USA: Academic Press, 2000.
- [4] S. Robinson, *Simulation: The Practice of Model Development and Use*, 2nd ed. Basingstoke, U.K.: Palgrave Macmillan, 2014.
- [5] D. Gross, J. F. Shortle, J. M. Thompson and C. M. Harris, *Fundamentals of Queueing Theory*, 5th ed. Hoboken, NJ, USA: John Wiley & Sons, 2018.
- [6] L. Kleinrock, *Queueing Systems, Volume 1: Theory*. New York, NY, USA: Wiley-Interscience, 1975.
- [7] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022.
- [8] D. E. Knuth, *The Art of Computer Programming, Volume 3: Sorting and Searching*, 2nd ed. Reading, MA, USA: Addison-Wesley, 1998.
- [9] M. Matsumoto and T. Nishimura, “Mersenne Twister: A 623-dimensionally equidistributed uniform pseudo-random number generator,” *ACM Transactions on Modeling and Computer Simulation*, vol. 8, no. 1, pp. 3–30, Jan. 1998.
- [10] G. E. P. Box and M. E. Muller, “A note on the generation of random normal deviates,” *The Annals of Mathematical Statistics*, vol. 29, no. 2, pp. 610–611, Jun. 1958.
- [11] A. B. Owen, *Monte Carlo Theory, Methods and Examples*. Stanford, CA, USA: Stanford University, 2013.
- [12] B. W. Boehm, *Software Engineering Economics*. Englewood Cliffs, NJ, USA: Prentice Hall, 1981.

- [13] A. J. Albrecht and J. E. Gaffney, “Software function, source lines of code, and development effort prediction: A software science validation,” *IEEE Transactions on Software Engineering*, vol. SE-9, no. 6, pp. 639–648, Nov. 1983.
- [14] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner’s Approach*, 9th ed. New York, NY, USA: McGraw-Hill Education, 2020.
- [15] I. Sommerville, *Software Engineering*, 10th ed. Harlow, U.K.: Pearson Education, 2015.
- [16] IEEE Recommended Practice for Software Requirements Specifications, IEEE Std 830-1998, Institute of Electrical and Electronics Engineers, 1998.
- [17] G. Booch, J. Rumbaugh and I. Jacobson, *The Unified Modeling Language User Guide*, 2nd ed. Boston, MA, USA: Addison-Wesley, 2005.
- [18] A. Silberschatz, H. F. Korth and S. Sudarshan, *Database System Concepts*, 7th ed. New York, NY, USA: McGraw-Hill Education, 2019.
- [19] Team SimPy, “SimPy: Discrete event simulation for Python — documentation,” SimPy 4.x. [Online]. Available: <https://simpy.readthedocs.io/> [Accessed: Dec. 2026].
- [20] Pyodide Development Team, “Pyodide: Python with the scientific stack, compiled to WebAssembly — documentation.” [Online]. Available: <https://pyodide.org/> [Accessed: Dec. 2026].
- [21] WebAssembly Community Group, “WebAssembly Core Specification,” World Wide Web Consortium (W3C) Recommendation, 2019. [Online]. Available: <https://www.w3.org/TR/wasm-core-1/> [Accessed: Dec. 2026].
- [22] Vercel Inc., “Next.js Documentation — App Router.” [Online]. Available: <https://nextjs.org/docs> [Accessed: Dec. 2026].
- [23] Meta Open Source, “React Documentation.” [Online]. Available: <https://react.dev/> [Accessed: Dec. 2026].
- [24] xyflow, “React Flow Documentation.” [Online]. Available: <https://reactflow.dev/> [Accessed: Dec. 2026].
- [25] Supabase Inc., “Supabase Documentation — Database, Auth, Realtime and Row Level Security.” [Online]. Available: <https://supabase.com/docs> [Accessed: Dec. 2026].
- [26] PostgreSQL Global Development Group, “PostgreSQL 16 Documentation — Row Security Policies.” [Online]. Available: <https://www.postgresql.org/docs/> [Accessed: Dec. 2026].
- [27] Google, “Gemini API Documentation.” [Online]. Available: <https://ai.google.dev/docs> [Accessed: Dec. 2026].

- [28] Microsoft Corporation, “TypeScript Handbook.” [Online]. Available: <https://www.typescriptlang.org/docs/> [Accessed: Dec. 2026].
- [29] Mozilla Developer Network, “Using Web Workers — Web APIs.” [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Web_Workers_API [Accessed: Dec. 2026].
- [30] International Function Point Users Group, Function Point Counting Practices Manual, Release 4.3.1. Princeton Junction, NJ, USA: IFPUG, 2010.